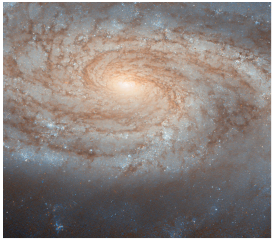
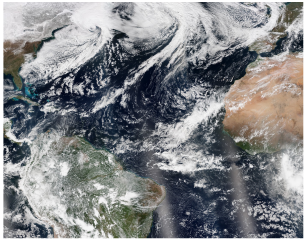
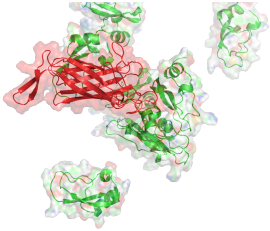
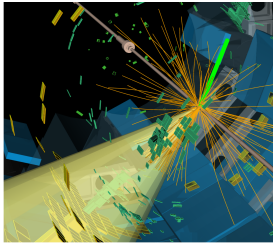





High performance scientific computing in C++

HPC C++ Course 2026

12 October – 15 October 2026 | Sandipan Mohanty | Forschungszentrum Jülich, Germany

Member of the Helmholtz Association

JÜLICH Forschungszentrum

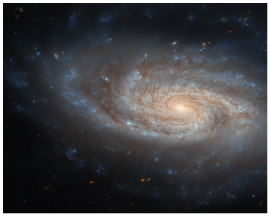
Chapter 1

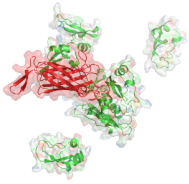
Introduction

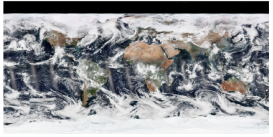
JÜLICH Forschungszentrum

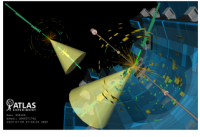
HPC and C++ in scientific computing

- Handle complexity and do it fast
- Reliablity: catch implementation logic errors before the program runs
- Efficient machine code based on the source: application return time may decide whether or not a research problem is even considered
  - Smart algorithms
  - Hardware aware translation of ideas into code
  - Profiling and tuning









JÜLICH Forschungszentrum

C++: elegant and efficient abstractions

- General purpose: no specialization to specific usage areas
- Compiler as a friend: in a large project, static type checking, data ownership control, const-ness guarantees and user defined compile time checks preclude a lot of possible errors
- No over simplification that precludes direct expert level use of hardware
- Leave no room for a lower level language
- You don't pay for features you don't use

C++ : high level and low level

- High level abstractions to facilitate fast development
- Direct access to low level features when you want them

Outline of themes

- A "quick" recap of C++20, C++23 and C++26
- Revisiting language fundamentals for high performance code
- Expression templates
- Explicit SIMD programming
- Multi-threaded programs using standard parallel algorithms and Intel (R) Threading Building Blocks
- Lessons from writing a matrix multiplication program
- Linear algebra with EIGEN
- GPU programming with NVidia CUDA and Thrust
- Introduction to single source heterogeneous computing using SYCL and OneAPI

The default C++ standard for code samples, examples exercises etc. is C++23, but a few examples will require older standards, and some others C++26.

Course material

- To download the material, use git, or the web browser. Here is the git repo: https://gitlab.jsc.fz-juelich.de/sdlbio-courses/hpcxx2026.git

```
cd your_working_directory
git clone https://gitlab.jsc.fz-juelich.de/sdlbio-courses/hpcxx2026.git
```

  The material will be updated every day during the course.

- Logging in to our system as described in the following slides and running the `setup.sh` script sets up your working directory for the course. Please read the information printed when you run the `setup.sh` script. It is important to work without interfering with others. Please download and work on the course material in **your own directory!**

High performance scientific computing in C++

Supercomputer access for the exercises

- Please login to the Jupyter-JSC system (https://jupyter-jsc.fz-juelich.de). If asked to choose whether to use JSC account or Helmholz account, choose JSC.
- After logging in, try to add a new jupyterlab. Choose JURECA as the system and training2652 as the project.
- For the partition choose **LoginNode**, and then start. Wait until the swirly things stop and you see the panel.
- What we will most need from there is the terminal, which should be at the bottom.
- In the terminal type this:

```
$ source $PROJECT/local/setup.sh
```

- After this, your paths should be set correctly. Test it using

```
$ g++ --version
$ clang++ --version
```

You should see GCC version 16.2 and Clang version 23.1.0, both compiled from source.

High performance scientific computing in C++

- **Necessary first step:** `source $PROJECT/local/setup.sh`, for every new terminal on JURECA for this course.
- It creates user specific working directories, downloads and updates course material and sets up the environment variables for compilers and libraries.
- After the script `setup.sh` is sourced, the following environment variables (EV) and additional shortcuts (SC) are available
  - `cxx2026`: (EV) Location of your private working area for the course (like `$HOME`)
  - `swhome`: (EV) Top level folder for software installations for compilers and libraries
  - `cdp`: (SC) Change directory to the top level of your private workspace (≡ `cd $cxx2026`)
  - `pathadd`: (SC) Prepend a new folder to PATH. E.g., `pathadd /x/y/z/bin`
  - `pathrm`: (SC) Remove a folder from PATH
  - `libpathadd`, `libpathrm`: (SC) Same as above, but for `LD_LIBRARY_PATH`, `LD_RUN_PATH`, `LIBRARY_PATH`
  - `incpathadd`, `incpathrm`: (SC) Same, but for `CPATH`, which is searched by the compilers for include files.
  - `cmpathadd`, `cmpathrm`: Same, but for `CMAKE_PREFIX_PATH`
  - `G`: (SC) Alias for `g++` using common options `-std=c++23 -pedantic -Wall -O3 -march=native`
  - `C`: (SC) Similar to `G`, but for Clang. It also uses Clang's own implementation of the standard library, `libc++`
  - `Gstdm` (SC). Generates pre-compiled module for the standard library, which can then be imported with a simple "import std;" line. For every new folder where you need to compile programs using "import std;" using the g++ compiler, you need to run this once. Alias for
    `g++ -std=c++23 -fmodules -fsearch-include-path -c bits/std.cc`

Some notes on the organisation of the course material

- When you run
  source $PROJECT/local/setup.sh
  a working directory with your user name will be created (if it was not already there) inside the project working area
- Type `cdp` to change directory to your private working directory.
  - Download course material in that directory as described earlier
  - Create a different folder, (e.g., `work`), where you do your modifications. Copy any examples you want to work on, to this directory, leaving the original archive unedited. How you manage things in `work` is up to you.
- The contents of these first few slides, since you may need to look them up later, are placed in the file `utilities.pdf`
- To test if things are working fine for you, go into the folder `test_your_setup` (or copy that into your working directory), and follow the instructions in the README files there. Finishing these steps ahead of the course will help save some time during the course!

Direct SSH connection to JURECA

- This is not strictly necessary for the course, but many people prefer to SSH connections outside the Jupyter session. This is also a useful backup option if Jupyter-JSC has an outage during the course!
- Follow the instructions in the page https://apps.fz-juelich.de/jsc/hps/jureca/clus ter/access.html#ssh-login to generate a suitable SSH key for JURECA. Certain key types are preferred, and the recommended key type is ed25519.

```
$ ssh-keygen -a 100 -t ed25519 -f ~/.ssh/id_ed25519_jsc
$
```

- Upload the generated **public key** to JuDoor
  - Login to JuDoor
  - Click on "Manage SSH-keys" next to JURECA
  - Select your ed25519 public key file using the "Browse" button. This file has a ".pub" extension.
  - Pay special attention to the "from clause". This goes in the box to the right of the public key filename box. If you are in a hurry, use the suggestion based on your current IP address, which is displayed in the from-clause box by pressing the downward arrow key to its right.
  - After filling in a suitable "from clause", click on the Start upload of SSH-keys button
- Wait

Direct SSH connection to JURECA

- Open SSH session on JURECA using your ed25519 key like this:

```
$  ssh -i ~/.ssh/id_ed25519_jsc <yourid>@jureca.fz-juelich.de
```

- ... or, make it a little easier for yourself:
  - Add a section for JURECA in your SSH config file `~/.ssh/config`

```
1  Host jureca??
2    Hostname %h.fz-juelich.de
3    Match Host jureca.fz-juelich.de,jureca??.fz-juelich.de
4    User yourid
5    IdentityFile ~/.ssh/id_ed25519_jsc
6    ServerAliveInterval 60
```

  - Open SSH session like this: `$ ssh jureca`
- After opening a new SSH session, set your project to training2652 and run the setup script:

```
$ jutil env activate -p training2652
$ source $PROJECT/local/setup.sh
$
```

What's wrong with this code?

```
1  #include <vector>
2  #include <iostream>
3
4  template <class T>
5  void fill(std::vector<T> v) {
6      for (auto& el : v) el = static_cast<T>(2.3);
7  }
8  auto main() -> int
9  {
10
11     std::vector a(10, 0);
12     std::vector b(10, 0.);
13     fill(a);
14     fill(b);
15     for (auto el: a) std::cout << el << "\n";
16     for (auto el: b) std::cout << el << "\n";
17
18 }
```

- (A) We forgot the template arguments when creating the `std::vector` objects `a` and `b`
- (B) The function `fill` is completely pointless, as it does not affect `a` or `b`
- (C) `fill` is OK for `b` which stores doubles, but misleading for `a` which stores `int`
- (D) Nothing. It will produce the intended result.

What's wrong with this code?

```
1  #include <iostream>
2  auto main() -> int
3  {
4    auto  high = 100'000'000.0f;
5    auto low = 99'999'990.0f;
6    auto sum = 0.f;
7    do {
8      sum + = 1;
9      high -= 1;
10   } while (high > low);
11   std::cout << sum << "\n";
12 }
```

- (A) Syntax error because of weirdly placed single quotes in lines 4 and 5
- (B) High will always remain greater than low
- (C) Do-while loop will run the loop body once too often
- (D) Nothing. It will produce the obvious answer

What is wrong with this code?

```
1  #include <iostream>
2  namespace A {
3  namespace B {
4      struct Cla {
5          double x {}, y {};
6      };
7  }
8  auto operator<<(std::ostream& os,
9                  const B::Cla& obj)
10         -> std::ostream&
11 {
12     os << obj.x << "\t" << obj.y << "\n";
13     return os;
14 }
15 }
16 auto main() -> int
17 {
18     A::B::Cla s { 3., 76. };
19     std::cout << s << "\n";
20 }
```

- (A) `operator<<` is not declared as a friend in the class `Cla`
- (B) We are not calling the output operator according to the code here:
  `A::operator<<(std::cout, s) << "\n";`
- (C) `operator<<` is defined in the `std` namespace, so, we have to declare it in the same namespace
- (D) There is nothing wrong. This is how the stream output operator is usually written.

Argument Dependent Lookup

Argument Dependent Lookup

```
1  namespace Surrounding {
2      struct AClass {};
3      void one_func(int x, int y) {
4          std::cout << "Surrounding::one_func(int, int) << "\n";
5      }
6      void another_func(int x, AClass y) {
7          std::cout << "Calling Surrounding::another_func...";
8      }
9  }
10 // Elsewhere...
11 Surrounding::AClass obj; // OK
12 Surrounding::one_func(1, 2); // OK
13 one_func(1, 2); // Error!
14 Surrounding::another_func(1, obj); // OK
15 another_func(1, obj); // OK!
```

Argument Dependent Lookup

- If a function call expression involves one or more arguments of class types, the search for the matching function includes functions defined in the namespaces surrounding each of those classes. This is called "Argument Dependent Lookup", or Koenig Lookup
- The functions considered have to be in the immediately surrounding namespace around a class
- Calling such functions is very similar to calling our mental model of a member function, e.g., `norm(x)` instead of `x.norm()`
- **Recommendation:** Write more functions using class type objects in the surrounding namespace instead of making them members!

Argument Dependent Lookup

Exercise 1.1:

The notebook `ADL.ipynb` demonstrates argument dependent lookup. This is an important class related idea. Please go through the notebook and try out your own variations!

Exercise 1.2:

The folder `examples/ADL` contains a series of small programs demonstrating ADL, similar to those in the notebook above. Try them, in addition to the notebooks, as they explore the topic further.

Numeric types

Floating point numbers

Area of a triangle of sides a, b and c...

- Heron's formula (Metrica, Heron of Alexandria, $\approx$ 60 CE)

$$
\begin{aligned}
s &= \frac{a+b+c}{2} \\
\Delta &= \sqrt{s \times (s-a) \times (s-b) \times (s-c)}
\end{aligned}
$$

JÜLICH Forschungszentrum

Floating point numbers

Area of a triangle of sides a, b and c...

- Heron's formula (Metrica, Heron of Alexandria, $\approx$ 60 CE)

$$\begin{aligned} s &= \frac{a+b+c}{2} \\ \Delta &= \sqrt{s \times (s-a) \times (s-b) \times (s-c)} \end{aligned}$$

- Kahan's formula (Miscalculating Area and Angles of a Needle-like Triangle, W. Kahan, 2000 CE, `http://http.cs.berkeley.edu/~wkahan/Triangle.pdf`)

$$\begin{aligned} a \geq b &\geq c \\ \Delta &= \frac{1}{4}\sqrt{(a+(b+c)) \times (c-(a-b)) \times (c+(a-b)) \times (a+(b-c))} \end{aligned}$$

Floating point numbers

```
1  const auto a = 5.0f;
2  const auto b = 4.0f;
3  const auto c = 3.0f;
4  std::cout << "Heron's formula = "
5        << area_heron(a,b,c) << "\n";
6  std::cout << "Kahan's formula = "
7        << area_kahan(a,b,c) << "\n";
```

```
Heron's formula = 6
Kahan's formula = 6
```

- Mathematically, both calculate the same thing

JÜLICH Forschungszentrum

Floating point numbers

```
const auto a = 100'000.000'00f;
const auto b =  99'999.999'79f;
const auto c =       0.000'29f;
std::cout << "Heron's formula = "
      << area_heron(a,b,c) << "\n";
std::cout << "Kahan's formula = "
      << area_kahan(a,b,c) << "\n";
```

```
|
|
```

- Mathematically, both calculate the same thing
- If the triangle becomes very long and thin though, weird things happen

Floating point numbers

```
1  const auto a = 100'000.000'00f;
2  const auto b =  99'999.999'79f;
3  const auto c =       0.000'29f;
4  std::cout << "Heron's formula = "
5        << area_heron(a,b,c) << "\n";
6  std::cout << "Kahan's formula = "
7        << area_kahan(a,b,c) << "\n";
```

```
Heron's formula = 0
Kahan's formula = 14.5
```

- Mathematically, both calculate the same thing
- If the triangle becomes very long and thin though, weird things happen

JÜLICH Forschungszentrum

Floating point numbers

```
1  const auto a = 100'000.000'00f;
2  const auto b =  99'999.999'79f;
3  const auto c =       0.000'29f;
4  std::cout << "Heron's formula = "
5        << area_heron(a,b,c) << "\n";
6  std::cout << "Kahan's formula = "
7        << area_kahan(a,b,c) << "\n";
```

```
Heron's formula = 0
Kahan's formula = 14.5
```

- Mathematically, both calculate the same thing
- If the triangle becomes very long and thin though, weird things happen
- Correct answer is 10.

Representation of floating point numbers

$$-1^{s} \times 1.mantissa \times 2^{exponent}$$

- It is enough to store the coloured parts. We win an extra bit of precision in the mantissa by skipping the 1 before the decimal point.

Representation of floating point numbers

$$-1^{s} \times 1.mantissa \times 2^{exponent}$$

- It is enough to store the coloured parts. We win an extra bit of precision in the mantissa by skipping the 1 before the decimal point.
- For a fixed exponent, there are 2^{23} different floating point numbers. $\Longrightarrow$ There are as many `float`s between 2^{-11} and 2^{-10} as there are between 1024 and 2048

Representation of floating point numbers

$$-1^{s} \times 1.mantissa \times 2^{exponent}$$

- It is enough to store the coloured parts. We win an extra bit of precision in the mantissa by skipping the 1 before the decimal point.
- For a fixed exponent, there are 2^{23} different floating point numbers. $\Longrightarrow$ There are as many `float`s between 2^{-11} and 2^{-10} as there are between 1024 and 2048
- By contrast, integral types have a uniform density throughout their range

Representation of floating point numbers

$$-1^{s} \times 1.mantissa \times 2^{exponent}$$

- Zero = all bits 0. One ?

JÜLICH Forschungszentrum

Representation of floating point numbers

$$-1^{s} \times 1.mantissa \times 2^{exponent}$$

- Zero = all bits 0. One ?
- Exponent is stored *shift-127* encoded. So, $1 \equiv$ [0][01111111][00000000000000000000000]

Representation of floating point numbers

$$-1^{s} \times 1.mantissa \times 2^{exponent}$$

- Zero = all bits 0. One ?
- Exponent is stored *shift-127* encoded. So, $1 \equiv$ [0][01111111][00000000000000000000000]
- To maintain our sanity, we will write it as $1 \equiv$ [0][(2^0)][00000000000000000000000]

Floating point numbers

- Mental exercise: we have two decimal numbers in scientific notation 9.78×10^{2}, and 1.0×10^{-1}. How will you add them ?

Floating point numbers

- Mental exercise: we have two decimal numbers in scientific notation 9.78×10^2, and 1.0×10^{-1}. How will you add them ?
- You shift the decimal point in one of them until the exponents are the same, and then add the mantissas: $9.78 \times 10^2 + 0.001 \times 10^2$. Digits in the smaller number are pushed to the right

Floating point numbers

- $1 \equiv [0][(2^0)][00000000000000000000000]$

JÜLICH Forschungszentrum

Floating point numbers

- $1 \equiv [0][(2^0)][00000000000000000000000]$
- What is the smallest representable n, with $n > 1$?

Floating point numbers

- $1 \equiv [0][(2^0)][00000000000000000000000]$
- What is the smallest representable n, with $n > 1$?
- $[0][(2^0)][00000000000000000000001]$ with the mantissa changing by $2^{-23} \approx 0.00000011920928955078125$

Floating point numbers

- $1 \equiv [0][(2^0)][00000000000000000000000]$
- What is the smallest representable n, with $n > 1$?
- $[0][(2^0)][00000000000000000000001]$ with the mantissa changing by $2^{-23} \approx 0.00000011920928955078125$
- What is 2.0 ? $[0][(2^1)][00000000000000000000000]$. What if you add these two ? What information about the smaller number can we retain ?

JÜLICH Forschungszentrum

Floating point numbers

- What is the smallest representable n, with $n > 1$?
- [0][(2^0)][00000000000000000000001]. Mantissa changes by $2^{-23} \approx 0.000000119209289550781 3$.
- This quantity depends on the floating point type. In C++, you can retrieve it `std::numeric_limits<T>::epsilon()`

Floating point numbers

- What is the smallest representable n, with $n > 1$?
- [0][(2^0)][00000000000000000000001]. Mantissa changes by $2^{-23} \approx 0.000000119209289550781 3$.
- This quantity depends on the floating point type. In C++, you can retrieve it `std::numeric_limits<T>::epsilon()`
- Two quantities with exponent 0 can not be distinguished in this representation, if they differ by less than `epsilon`

Floating point numbers

- What is the smallest representable n, with $n > 1$?
- [0][(2^0)][00000000000000000000001]. Mantissa changes by $2^{-23} \approx 0.00000011920928955078125$.
- This quantity depends on the floating point type. In C++, you can retrieve it `std::numeric_limits<T>::epsilon()`
- Two quantities with exponent 0 can not be distinguished in this representation, if they differ by less than `epsilon`
- In an expression like `(big+small)-big`, if `big` and `small` differ by more than 23 in exponent, all information about `small` is lost, and we get a 0. $2^{23} = 8388608$.

Floating point numbers

- Floating point numbers with all bits in the exponent field at 0, are said to be “denormalised” (remember the shift-127 encoding)
- Not enough bits to represent such small quantities.
- All exponent bits being 1 indicate some special “numbers”:
  - $\pm\infty$: all mantissa bits 0.
  - NaN : at least one mantissa bit non-zero.

Exercise 1.3:

In `examples/floating_fun.cc`, there is a small program "simulating" a calculation involving some large quantities adding up to 0. Eight numbers are stored in an array of floats, and their sum evaluated and printed. The calculation is repeated by permuting the indexes of the array, so that the numbers are added in all possible orders. Observe the output!

Exercise 1.4: std::numeric_limits

What is epsilon for **`float`** and **`double`** on your computer ? Find out by writing a small C++ program and printing out the values from `std::numeric_limits`. Look up the documentation of `numeric_limits`. What other information can you get about numeric types from that header ?

Float: $[1 - \text{bit}][8 - \text{bits}][23 - \text{bits}]$

Maximum	3.40282e+38
Minimum	1.17549e-38
Lowest	-3.40282e+38
Epsilon	1.19209e-07
Rounding error	0.5

Double: $[1 - \text{bit}][11 - \text{bits}][52 - \text{bits}]$

Maximum	1.79769e+308
Minimum	2.22507e-308
Lowest	-1.79769e+308
Epsilon	2.22045e-16
Rounding error	0.5

New floating point types in C++23

Name	typeid	Min	Max	Epsilon
double	d	2.2250738585072014e-308	1.7976931348623157e+308	2.220446049250313e-16
std::float64_t	DF64	2.2250738585072014e-308	1.7976931348623157e+308	2.220446049250313e-16
float	f	1.1754944e-38	3.4028235e+38	1.1920929e-07
std::float32_t	DF32_	1.1754944e-38	3.4028235e+38	1.1920929e-07
std::float16_t	DF16_	6.1035156e-05	65504	0.0009765625
std::bfloat16_t	DF16b	1.1754944e-38	3.3895314e+38	0.0078125

- Two different 16 bit floating point numbers introduced
- `std::float64_t` and `std::float32_t` with different **`typeid`**s compared to built in **`double`** and **`float`**

JÜLICH Forschungszentrum

C++20

Important refreshing of the language, similar to C++11.

- **Concepts**
- **Ranges**
- **Modules**
- **Coroutines**
- `auto` function parameters to implicitly declare function templates
- Explicit template syntax for lambdas
- Class non-type template parameters
- `try ... catch` and virtual functions in `constexpr` functions
- `consteval` and `constinit`
- `<=>`

- `<span>`
- `<ranges>`
- `<concepts>`
- `std::atomic<double>`
- `constexpr` algorithms
- `std::assume_aligned`
- `constexpr` numeric algorithms

C++23

(Interesting changes are mostly concentrated in the standard library.)

- Multi-dimensional subscript operators
- Deducing `this`
- Static `operator()` and `operator[]`
- `[[assume(expr)]]`

- `import std;`
- `<expected>`
- `ranges::to`, `views::zip`
- `<stacktrace>`
- `std::byteswap`
- `std::mdspan`
- Formatting ranges and containers
- `<print>`
- `std::forward_like`
- `std::generator`: synchronous coroutine generator

C++26

- **Contracts**
- **Reflection**
- Expansion statements
- Pack indexing
- Structured binding improvements

- `<debugging>`
- `<linalg>`
- `<simd>`
- `<hive>`
- Parallel range algorithms
- ...

First: a couple of small, but interesting changes...

std::osyncstream

```cpp
#include <iostream>
#include <omp.h>

auto main() -> int
{
    #pragma omp parallel for
    for (auto i = 0UL; i < 100UL; ++i) {
       std::cout << "counter = " << i << " on thread "
                 << omp_get_thread_num() << "\n";
    }
}
```


JÜLICH Forschungszentrum

First: a couple of small, but interesting changes...

std::osyncstream

JÜLICH Forschungszentrum

First: a couple of small, but interesting changes...

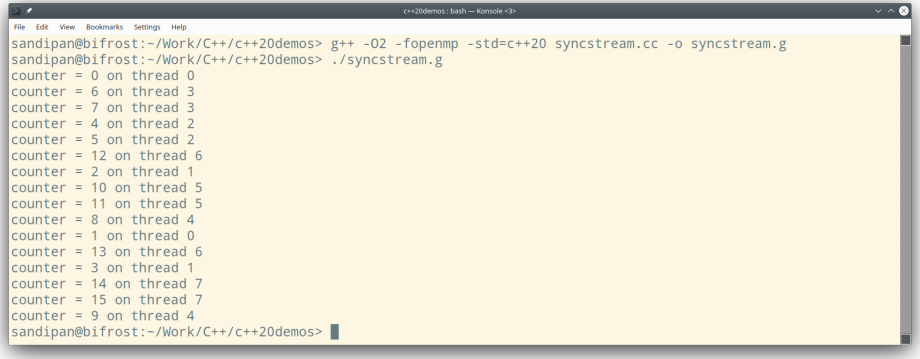
std::osyncstream

```
1  #include <iostream>
2  #include <syncstream>
3  #include <omp.h>
4
5  auto main() -> int
6  {
7      #pragma omp parallel for
8      for (auto i = 0UL; i < 100UL; ++i) {
9         std::osyncstream{std::cout} << "counter = " << i << " on thread "
10                                    << omp_get_thread_num() << "\n";
11     }
12 }
```


JÜLICH Forschungszentrum

First: a couple of small, but interesting changes...

std::osyncstream



Exercise 1.5:

Demo programs for `std::syncstream` are in the examples folder. Inspect, build, check effects...

```
$ G syncstream.cc -fopenmp -o syncstream.g
$ ./syncstream.g
```

`examples/garbled.cc` and `examples/syncstream.cc` demonstrate the use of `std::osyncstream` as shown above. `examples/syncstream_mpi.cc` demonstrates that the synchronisation of output stream also works with output from different MPI processes.

```
$ mpicxx -std=c++23 -O3 -fopenmp syncstream_mpi.cc -o syncstream.mpi
$ OMP_NUM_THREADS=4 batch_run --ntasks=32 --cpus-per-task=4 ./syncstream.mpi
```

Immediate functions

```
1  // examples/immediate.cc
2  constexpr auto cxpr_sqr(auto x) { return x * x; }
3  consteval auto cevl_sqr(auto x) { return x * x; }
4
5  auto main(int argc, char* argv[]) -> int
6  {
7      std::array<double, cxpr_sqr(14)> A;
8      std::array<double, cevl_sqr(14)> B;
9      std::cout << cxpr_sqr(argc) << "\n";
10     std::cout << cevl_sqr(argc) << "\n";
11 }
```

- **constexpr** functions with compile time constant arguments are evaluated at compile time, if the result is needed to initialise a **constexpr** variable
- **constexpr** functions remain available for use with non-constant objects at run-time. This is sometimes desirable, but it also makes certain accidental uses possible, when we intend compile time evaluation but get something else.
- The new **consteval** specifier creates "immediate" functions. It is possible to use them in the compile time context. But it is an error to use them with non-constant arguments.

Designated initialisers

```
1  // examples/desig2.cc
2  struct v3 { double x, y, z; };
3  struct pars { int offset; v3 velocity; };
4  auto operator<<(std::ostream & os, const v3 & v) -> std::ostream&
5  {
6      return os << v.x << ", " << v.y << ", " << v.z << " ";
7  }
8  void example_func(pars p)
9  {
10     std::cout << p.offset << " with velocity " << p.velocity << "\n";
11 }
12 auto main() -> int
13 {
14     example_func({.offset = 5, .velocity = {.x=1., .y = 2., .z=3.}});
15 }
```

- Simple struct type objects can be initialised by designated initialisers for each field.
- Can be used to implement a kind of "keyword arguments" for functions. But remember, at least upto C++23, the field order can not be shuffled.

Couple of small, but interesting changes... I

- You can now write `auto` in function parameter lists, e.g.,
  `auto add(auto x, auto y) { return x + y; }`, to create a function template
  `template <class T, class U> auto add(T x, U y) { return x + y; }`
- You can now use explicit template parameters in lambda functions

```
[]<class T>(T x, const std::vector<T>& v) {
   for (auto el : v) x += el;
   return x;
};
```

- `std::string` can now `reserve()` memory
- `S.starts_with("pre")` checks if a string `S` starts with the prefix "pre". Similarly for `ends_with()`
- `M.contains("key")` answers whether a certain key "key" is present in an associative container. Cleaner than `try { M.at("key"); } catch (auto& err) { }`, or

```
1  if (auto it = find_if(M.begin(), M.end(), [](auto&& el){ return el.first == "key"; });
2       it != M.end()) { ... }
```

Couple of small, but interesting changes... II

- `std::erase(C, element)` and `std::erase_if(C, predicate)` erase elements equal to a given element or elements satisfying a given predicate from a container C. *Same behaviour for different containers.*
- `std::lerp(min, max, t)` : linear interpolation, `std::midpoint(a, b)` : overflow aware mid-point calculation
- `std::assume_aligned<16>(dptr)` returns the input pointer, but the compiler then assumes that the pointer is aligned to a given number of bytes.

Couple of small, but interesting changes...

- `std::span<T>` is a new non-owning view type for contiguous ranges of arbitrary element types `T`. It is like the `string_view`, but for other array like entities such as `vector<T>`, `array<T,N>`, `valarray<T>` or even C-style arrays. Can be used to encapsulate the (pointer, size) pairs often used as function arguments. Benefit: it gives us an STL style interface for the (pointer, size) pair, so that they can be directly used with C++ algorithms.
- Signed size of containers: `std::ssize(C)`, where `C` is a container, returns a signed integer (number of elements in container). Containers like `std::vector`, `std::list`, `std::map` have member functions `ssize()` for the same purpose. Signed sizes are useful, for instance, when iterating backwards through the container.
- Safe integer comparisons: functions like `cmp_less(i1, i2)` will perform integer comparisons without conversions. `cmp_less(-1, 1U)` will return **`true`**, where as, `-1 < 1U` returns **`false`**. Similarly, we have, `cmp_less_equal`, `cmp_greater`, `cmp_not_equal` and `cmp_equal`.

Couple of small, but interesting changes...

Bit manipulation:

- `std::bit_cast<>`: `bit_cast<uint64_t>(3.141592653)` reinterprets the bits in the object representation of the input and returns an object of a required type so that the corresponding bits match
- `has_single_bit(UnsignedInteger)`: answers if only one of the bits in the input is **1**, while the rest are **0**
- `std::rotl(UnsignedInteger, amount)`, `std::rotr(UnsignedInteger, amount)`: Rotates the bits in an unsigned integer left or right by a given amount
- `std::bit_floor(UnsignedInteger)`: Largest power of two not greater than input
- `std::bit_ceil(UnsignedInteger)`: Smallest power of two not smaller than input
- Count consecutive **0** bits from the left `countl_zero` or right `countr_zero`, and similarly for **1** bits
- `popcount`, count the total number of **1** bits in the entire input

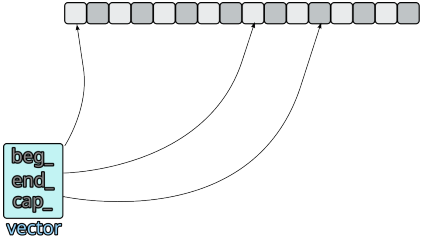
Exercise 1.6:

Some example programs about the minor new features of C++20 and C++23 are `desig.cc`, `desig2.cc`, `cxpr_algo0.cc`, `immediate.cc`, `intcmp.cc`, and `bit0.cc`, in the `examples/` directory. Check them, change them in small ways, ask related questions!
Alternatively, double click on the Jupyter Notebook in the examples directory called `MiscSmallFeatures.ipynb`, and work through the notebook.

std::span (C++20)

std::vector

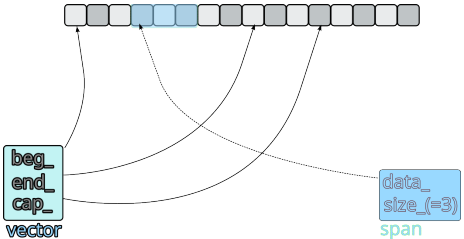
- `operator[]`
- `size()` and `ssize()`
- `begin()`
- `end()`



std::span (C++20)

std::vector

- `operator[]`
- `size()` and `ssize()`
- `begin()`
- `end()`



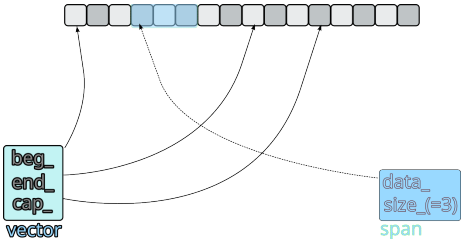
std::span

- `operator[]`
- `size()` and `ssize()`
- `begin()`
- `end()`

std::span (C++20)

std::vector

- `operator[]`
- `size()` and `ssize()`
- `begin()`
- `end()`



std::span

- `operator[]`
- `size()` and `ssize()`
- `begin()`
- `end()`

Contiguous containers

- RAII

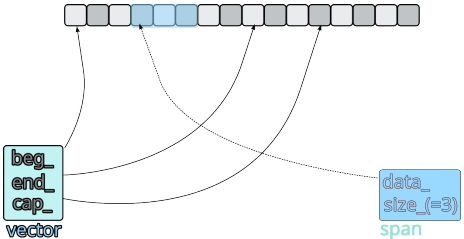
span: address and size of pre-existing data

- No resource ownership or management

std::span (C++20)

std::vector

- `operator[]`
- `size()` and `ssize()`
- `begin()`
- `end()`



std::span

- `operator[]`
- `size()` and `ssize()`
- `begin()`
- `end()`

Contiguous containers

- As long as container exists, elements can be accessed

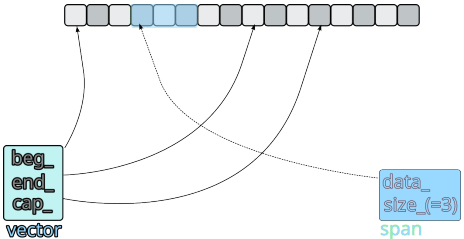
span: address and size of pre-existing data

- Even if span exists, accessibility of data is not guaranteed

std::span (C++20)

std::vector

- `operator[]`
- `size()` and `ssize()`
- `begin()`
- `end()`



std::span

- `operator[]`
- `size()` and `ssize()`
- `begin()`
- `end()`

Contiguous containers

- When container has expired, references / pointers / iterators to elements are invalidated

span: address and size of pre-existing data

- When span has expired, references / pointers / iterators to elements may remain valid

span

```
using std::transform_reduce;
using std::plus;
using std::multiplies;
using std::vector;


auto compute(const vector<double>& u,
    const vector<double>& v) -> double
{
    return transform_reduce(
        u.begin(), u.end(),
        v.begin(), 0., plus<double>{},
        multiplies<double>{});
}
void elsewhere()
{
    vector<double> A(100UL, 0.34);
    vector<double> B(100UL, 0.87);
    std::cout << compute(A, B) << "\n";
}
```

- We can avoid needlessly restrictive interfaces

JÜLICH Forschungszentrum

span

```
1  using std::transform_reduce;
2  using std::plus;
3  using std::multiplies;
4  using std::vector;
5  using std::valarray;
6
7  auto compute(const vector<double>& u,
8      const vector<double>& v) -> double
9  {
10     return transform_reduce(
11         u.begin(), u.end(),
12         v.begin(), 0., plus<double>{},
13         multiplies<double>{});
14 }
15 void elsewhere()
16 {
17     vector<double> A(100UL, 0.34);
18     valarray<double> B(100UL, 0.87);
19     std::cout << compute(A, B) << "\n";
20 }
```

- We can avoid needlessly restrictive interfaces
- As written here, `std::valarray` wouldn't be an acceptable input

JÜLICH Forschungszentrum

span

```
1  using std::transform_reduce;
2  using std::plus;
3  using std::multiplies;
4  using std::vector;
5  template <class T>
6  using VT = vector<T, tbb::scalable_allocator<T>>;
7
8  auto compute(const vector<double>& u,
9      const vector<double>& v) -> double
10 {
11     return transform_reduce(
12         u.begin(), u.end(),
13         v.begin(), 0., plus<double>{},
14         multiplies<double>{});
15 }
16 void elsewhere()
17 {
18     vector<double> A(100UL, 0.34);
19     VT<double> B(100UL, 0.87);
20     std::cout << compute(A, B) << "\n";
21 }
```

- We can avoid needlessly restrictive interfaces
- As written here, `std::valarray` wouldn't be an acceptable input
- As written here, even `std::vector` with a different allocator wouldn't be an acceptable input

span

```
1  using std::transform_reduce;
2  using std::plus;
3  using std::multiplies;
4  using std::vector;
5  template <class T>
6  using VT = vector<T, tbb::scalable_allocator<T>>;
7  using std::span;
8  auto compute(span<const double> u,
9       span<const double> v) -> double
10 {
11     return transform_reduce(
12         u.begin(), u.end(),
13         v.begin(), 0., plus<double>{},
14         multiplies<double>{});
15 }
16 void elsewhere()
17 {
18     vector<double> A(100UL, 0.34);
19     VT<double> B(100UL, 0.87);
20     std::cout << compute(A, B) << "\n";
21 }
```

- We can avoid needlessly restrictive interfaces
- As written here, `std::valarray` wouldn't be an acceptable input
- As written here, even `std::vector` with a different allocator wouldn't be an acceptable input
- With `std::span` we can write a concrete function, which can be used with any contiguous container!

span

```
1  using std::transform_reduce;
2  using std::plus;
3  using std::multiplies;
4  using std::vector;
5  template <class T>
6  using VT = vector<T, tbb::scalable_allocator<T>>;
7  using std::span;
8  auto compute(span<const double> u,
9       span<const double> v) -> double
10 {
11     return transform_reduce(
12         u.begin(), u.end(),
13         v.begin(), 0., plus<double>{},
14         multiplies<double>{});
15 }
16 void elsewhere(const double* A, size_t N)
17 {
18     VT<double> B(N, 0.87);
19     std::cout << compute(span(A, N), B) << "\n";
20 }
```

- We can avoid needlessly restrictive interfaces
- As written here, `std::valarray` wouldn't be an acceptable input
- As written here, even `std::vector` with a different allocator wouldn't be an acceptable input
- With `std::span` we can write a concrete function, which can be used with any contiguous container!
- Contiguous data stored anywhere, even C-style arrays, can be easily used for the same function

```
1  template <class NoBueno>
2  auto compute(std::span<NoBueno> s) {...}
3  void elsewhere(const VT& v) {
4      compute(v);
5      // Template argument deduction failed!
6      // compute(std::span(v));
7      // OK, if conversion possible
8  }
```


JÜLICH Forschungszentrum

span

```
1  using std::span;
2  using std::transform_reduce;
3  using std::plus;
4  using std::multiplies;
5  auto compute(span<const double> u,
6       span<const double> v) -> double
7  {
8      return transform_reduce(
9          u.begin(), u.end(),
10         v.begin(), 0., plus<double>{},
11         multiplies<double>{});
12 }
13
14 void elsewhere(double* x, double* y,
15                unsigned N)
16 {
17     return compute(span(x, N), span(y, N));
18 }
```

- Non-owning view type for a contiguous range
- No memory management
- Numeric operations can often be expressed in terms of existing arrays in memory, irrespective of how they got there and who cleans up after they expire
- `span` is designed to be that input for such functions
- Cheap to copy: essentially a pointer and a size
- STL container like interface

Exercise 1.7:

`examples/spans` is a directory containing the compute function as shown here. Notice how this function is used directly using C++ array types as arguments instead of spans, and indirectly when we only have pointers.

Recap of elementary features with an example

```cpp
// Trivial piece of code as a background for discussions
// examples/demo_saxpy/saxpy_0.cc
// includes ...
auto main() -> int
{
    const std::vector inp1 { 1., 2., 3., 4., 5. };
    const std::vector inp2 { 9., 8., 7., 6., 5. };
    std::vector outp(inp1.size(), 0.);

    auto saxpy = [](double a,
                    const std::vector<double>& x,
                    const std::vector<double>& y,
                    std::vector<double>&z) {
        std::transform(x.begin(), x.end(), y.begin(), z.begin(),
                       [a](double X, double Y){ return a * X + Y; });
    };

    std::ostream_iterator<double> cout { std::cout, "\n" };
    saxpy(10., inp1, inp2, outp);
    copy(outp.begin(), outp.end(), cout);
}
```


JÜLICH Forschungszentrum

Recap of elementary features with an example

```
1  // Trivial piece of code as a background for discussions
2  // examples/demo_saxpy/saxpy_0.cc
3  // includes ...
4  auto main() -> int
5  {
6      const std::vector inp1 { 1., 2., 3., 4., 5. };
7      const std::vector inp2 { 9., 8., 7., 6., 5. };
8      std::vector outp(inp1.size(), 0.);
9
10     auto saxpy = [](double a,
11                     const std::vector<double>& x,
12                     const std::vector<double>& y,
13                     std::vector<double>&z) {
14         std::transform(x.begin(), x.end(), y.begin(), z.begin(),
15                        [a](double X, double Y){ return a * X + Y; });
16     };
17
18     std::ostream_iterator<double> cout { std::cout, "\n" };
19     saxpy(10., inp1, inp2, outp);
20     copy(outp.begin(), outp.end(), cout);
21 }
```

How many syntax errors are there if we are using C++17 ?

A. 4

B. 3

C. 2

D. 0

JÜLICH Forschungszentrum

Generic lambdas...

```
1  // examples/demo_saxpy/saxpy_1.cc
2  // includes ...
3
4  auto main() -> int
5  {
6      const std::vector inp1 { 1., 2., 3., 4., 5. };
7      const std::vector inp2 { 9., 8., 7., 6., 5. };
8      std::vector outp(inp1.size(), 0.);
9
10     auto saxpy = [](double a, auto&& x, auto&& y, auto& z) {
11         std::transform(x.begin(), x.end(), y.begin(), z.begin(),
12                        [a](auto X, auto Y){ return a * X + Y; });
13     };
14
15     std::ostream_iterator<double> cout { std::cout, "\n" };
16     saxpy(10., inp1, inp2, outp);
17     copy(outp.begin(), outp.end(), cout);
18 }
```

We can make the lambda more compact by making it generic. But now the types of `x`, `y` and `z` are deduced independently. How can we keep it generic, and yet indicate that we want the same types for `x` and `y` ?

Explicit template syntax for lambdas

```
1  // examples/demo_saxpy/saxpy_2.cc
2  // includes ...
3  auto main() -> int
4  {
5      const std::vector inp1 { 1., 2., 3., 4., 5. };
6      const std::vector inp2 { 9., 8., 7., 6., 5. };
7      std::vector outp(inp1.size(), 0.);
8      auto saxpy = [] <class T, class T_in, class T_out>
9                      (T a, const T_in& x, const T_in& y, T_out& z) {
10         std::transform(x.begin(), x.end(), y.begin(), z.begin(),
11                        [a](T X, T Y){ return a * X + Y; });
12     };
13
14     std::ostream_iterator<double> cout { std::cout, "\n" };
15     saxpy(10., inp1, inp2, outp);
16     copy(outp.begin(), outp.end(), cout);
17 }
```

For normal function templates, we could easily express relationships among the types of different parameters. Now, we can do that for generic lambdas.

Constraining generic functions

```
1  // examples/demo_saxpy/saxpy_3.cc
2      template <class T> using value_type_of = typename std::remove_cvref_t<T>::value_type;
3  {
4      const std::vector inp1 { 1., 2., 3., 4., 5. };
5      const std::vector inp2 { 9., 8., 7., 6., 5. };
6      std::vector outp(inp1.size(), 0.);
7      auto saxpy = []<class T_in, class T_out>
8              (value_type_of<T_in> a, T_in&& x, T_in&& y, T_out& z) {
9          using in_element_type = value_type_of<T_in>;
10         using out_element_type = value_type_of<T_out>;
11         static_assert(std::is_same_v<in_element_type, out_element_type>,
12                       "Input and output element types must match!");
13         std::transform(x.begin(), x.end(), y.begin(), z.begin(),
14                        [a](in_element_type X, in_element_type Y){ return a * X + Y; });
15     };
16 //...
17     std::ostream_iterator<double> cout { std::cout, "\n" };
18     saxpy(10., inp1, inp2, outp);
```

At the least, we can use this to get helpful error messages when we use the function in a way that violates our assumptions.

Constraining generic functions

```
1  // examples/demo_saxpy/saxpy_3b.cc
2      const std::array inp1 { 1., 2., 3., 4., 5. };
3      const std::array inp2 { 9., 8., 7., 6., 5. };
4      std::vector outp(inp1.size(), 0);
5      auto saxpy = []<class T_in, class T_out>
6              (value_type_of<T_in> a, T_in&& x, T_in&& y, T_out& z) {
7          using in_element_type = value_type_of<T_in>;
8          using out_element_type = value_type_of<T_out>;
9          static_assert(std::is_same_v<in_element_type, out_element_type>,
10                       "Input and output element types must match!");
11         std::transform(x.begin(), x.end(), y.begin(), z.begin(),
12                        [a](in_element_type X, in_element_type Y){ return a * X + Y; });
13     };
14
15     std::ostream_iterator<double> cout { std::cout, "\n" };
16     saxpy(10., inp1, inp2, outp);
```

```
saxpy_3b.cc:16:9: error: static_assert failed due to requirement
'std::is_same_v<double, int>' "Input and output element types must match!"
```

Constraining generic functions

```
1       const std::array inp1 { 1., 2., 3., 4., 5. };
2       const std::array inp2 { 9., 8., 7., 6., 5. };
3       std::vector outp(inp1.size(), 0.);
4
5       auto saxpy = []<class T_in, class T_out>
6               (value_type_of<T_in> a, T_in&& x, T_in&& y, T_out& z) {
7           using in_element_type = value_type_of<T_in>;
8           using out_element_type = value_type_of<T_out>;
9           static_assert(std::is_same_v<in_element_type, out_element_type>,
10                        "Input and output element types must match!");
11          std::transform(x.begin(), x.end(), y.begin(), z.begin(),
12                         [a](in_element_type X, in_element_type Y){ return a * X + Y; });
13      };
14
15      std::ostream_iterator<double> cout { std::cout, "\n" };
16      saxpy(10., inp1, inp2, outp);
17      copy(outp.begin(), outp.end(), cout);
```

Different container types are acceptable as long as element types match! Controlled generic behaviour!

Constraining generic functions

```cpp
// examples/demo_saxpy/saxpy_4.cc
// includes ...
template <class T> using value_type_of = std::remove_cvref_t<T>::value_type;
template <class T_in, class T_out>
auto saxpy(value_type_of<T_in> a, T_in&& x, T_in&& y, T_out& z)
{
    using in_element_type = value_type_of<T_in>;
    using out_element_type = value_type_of<T_out>;
    static_assert(std::is_same_v<in_element_type, out_element_type>,
        "Input and output element types must match!");

    std::transform(x.begin(), x.end(), y.begin(), z.begin(),
        [a](in_element_type X, in_element_type Y) { return a * X + Y; });
}
auto main() -> int { ... }
```

Constraining normal function templates with template metaprogramming is an old technique. The syntax has become clearer with newer standards. Still, we are not expressing in code that the template parameters `T_in` and `T_out` should be array like objects, with `begin()`, `end()` etc.

std::span as function parameters

```cpp
// examples/demo_saxpy/saxpy_5.cc
// other includes
#include <span>
template <class T>
void saxpy(T a, std::span<const T> x, std::span<const T> y, std::span<T> z)
{
    std::transform(x.begin(), x.end(), y.begin(), z.begin(),
        [a](T X, T Y) { return a * X + Y; });
}

auto main() -> int { ... }
```

- `std::span<T>` is a non-owning adaptor ("view") for an existing array of objects in memory. It is like a pointer and a size.
- Provides an STL compatible interface
- Can be constructed from typical array like containers, e.g., `vector` `array`, C-style arrays ...
- Writing the `saxpy` function in terms of the `span` allows us to easily express that the element types in all three containers must be the same as the scalar.
- Still general enough to be used with different container types and different `T`

Exercise 1.8:

The examples used in these slides are all present in the `examples/demo_saxpy` folder of your course material. Check examples `saxpy_1.cc` through `saxpy_5.cc` containing the various version discussed so far. The important C++20 features we have revisited in this section so far, are explicit template syntax for lambdas and `std::span`.

std::span as function parameters

```
1  // examples/demo_saxpy/saxpy_5.cc
2  // other includes
3  #include <span>
4  template <class T>
5  void saxpy(T a, std::span<const T> x, std::span<const T> y, std::span<T> z)
6  {
7      std::transform(x.begin(), x.end(), y.begin(), z.begin(),
8          [a](T X, T Y) { return a * X + Y; });
9  }
10 auto main() -> int
11 {
12     const std::array inp1 { 1., 2., 3., 4., 5. };
13     const std::array inp2 { 9., 8., 7., 6., 5. };
14     std::vector outp(inp1.size(), 0.);
15     saxpy(10., {inp1}, {inp2}, {outp});
16 }
```

No inheritance relationships between `span` and any other containers!

std::span as function parameters

```
1  // examples/demo_saxpy/saxpy_5.cc
2  // other includes
3  #include <span>
4  template <class T>
5  void saxpy(T a, std::span<const T> x, std::span<const T> y, std::span<T> z)
6  {
7      std::transform(x.begin(), x.end(), y.begin(), z.begin(),
8          [a](T X, T Y) { return a * X + Y; });
9  }
10 auto main() -> int
11 {
12     const std::array inp1 { 1., 2., 3., 4., 5. };
13     const std::array inp2 { 9., 8., 7., 6., 5. };
14     std::vector outp(inp1.size(), 0.);
15     saxpy(10., {inp1}, {inp2}, {outp});
16 }
```

Can we restrict the scalar type to just floating point numbers, like `float` or `double` ?

Constraining templates using concepts

```
1  template <class T>
2  auto saxpy(T a, std::span<const T> x, std::span<const T> y, std::span<T> z)
3      -> std::enable_if_t<std::is_floating_point_v<T>, void>
4  {
5      std::transform(x.begin(), x.end(), y.begin(), z.begin(),
6          [a](T X, T Y) { return a * X + Y; });
7  }
```

SFINAE: "Substitution Failure is not an error" is widely used to achieve the effect in C++.

- If `T` is not a floating point number, `is_floating_point_v` becomes false.
- `enable_if_t<cond, R>` is defined as `R` if `cond` is true. If not it is simply undefined!
- False condition to `enable_if_t` makes the result type, which is used as the output here, vanish.
- The compiler interprets that as : "Stupid substitution! If I do that the function ends up with no return type! That can't be the right function template. Let's look elsewhere!"

Does the job. But, in C++20, we have a better alternative...

Constraining templates using concepts

```
1  template <class T> requires std::floating_point<T>
2  void saxpy(T a, std::span<const T> x, std::span<const T> y, std::span<T> z)
3  {
4      std::transform(x.begin(), x.end(), y.begin(), z.begin(),
5          [a](T X, T Y) { return a * X + Y; });
6  }
```

concepts: Named requirements on template parameters.

- Far easier to read than SFINAE (even the name!)
- If `MyAPI` is a **`concept`**, and `T` is a type, `MyAPI<T>` evaluates at compile time to either true or false.
- Concepts can be combined using conjunctions (`&&`) and disjunctions (`||`) to make other concepts.
- A **`requires`** clause introduces a constraint on a template type

A suitably designed set of concepts can greatly improve readability of template code

```cpp
// examples/demo_saxpy/saxpy_6.cc
template <class T> concept Number = std::floating_point<T> or std::integral<T>;
template <class T> requires Number<T>
auto saxpy(T a, std::span<const T> x, std::span<const T> y, std::span<T> z)
{
    std::transform(x.begin(), x.end(), y.begin(), z.begin(),
        [a](T X, T Y) { return a * X + Y; });
}
auto main() -> int
{
    {
        const std::array inp1 { 1., 2., 3., 4., 5. };
        const std::array inp2 { 9., 8., 7., 6., 5. };
        std::vector outp(inp1.size(), 0.);
        saxpy(10., {inp1}, {inp2}, {outp});
    }
    {
        const std::array inp1 { 1, 2, 3, 4, 5 };
        const std::array inp2 { 9, 8, 7, 6, 5 };
        std::vector outp(inp1.size(), 0);
        saxpy(10, {inp1}, {inp2}, {outp});
    }
}
```

Using concepts for our example

```
1  // examples/demo_saxpy/saxpy_6b.cc
2  template <class T> concept Number = std::floating_point<T> or std::integral<T>;
3
4  template <Number T>
5  auto saxpy(T a, std::span<const T> x, std::span<const T> y, std::span<T> z)
6  {
7      std::transform(x.begin(), x.end(), y.begin(), z.begin(),
8          [a](T X, T Y) { return a * X + Y; });
9  }
```

Our function is still a function template. But it does not accept "anything" as input. Acceptable inputs must have the following properties:

- The scalar type (first argument here) is a number by our definition
- The next two are contiguously stored constant arrays of the same scalar type
- The last is another span of non-const objects of the same scalar type

Using standard concepts and ranges in our example

```
1  // examples/demo_saxpy/saxpy_7.cc
2  namespace sr = std::ranges;
3  auto saxpy(std::floating_point auto a,
4             sr::input_range auto&& x, sr::input_range auto&& y,
5             std::weakly_incrementable auto&& z)
6  {
7      sr::transform(x, y, z, [a](auto X, auto Y) { return a * X + Y; });
8  }
9  auto main() -> int
10 {
11     std::vector inp1 { 1., 2., 3., 4., 5. };
12     std::vector inp2 { 9., 8., 7., 6., 5. };
13     std::array  inp3 { 9., 8., 7., 6., 5. };
14     double cstyle[]  { 1., 2., 3., 4., 5. };
15     std::vector outp( inp1.size(), 0.);
16     saxpy(10., inp1, inp2, outp.begin());
17     saxpy(10., inp1, inp3, outp.begin());
18     saxpy(10., inp1, std::to_array(cstyle), outp.begin());
19 }
```

```
1  namespace sr = std::ranges;
2  void saxpy(std::floating_point auto a,
3             sr::input_range auto&& x, sr::input_range auto&& y,
4             std::weakly_incrementable auto&& z) {
5      sr::transform(x, y, z, [a](auto X, auto Y) { return a * X + Y; });
6  }
7  void saxpy(std::weakly_incrementable auto&& z, std::floating_point auto a,
8             sr::input_range auto&& x, sr::input_range auto&& y) {
9      sr::transform(x, y, z, [a](auto X, auto Y) { return a * X + Y; });
10 }
11 auto main() -> int {
12     std::vector inp1 { 1., 2., 3., 4., 5. };
13     std::vector inp2 { 9., 8., 7., 6., 5. };
14     std::array  inp3 { 9., 8., 7., 6., 5. };
15     double cstyle[]  { 1., 2., 3., 4., 5. };
16     std::vector outp( inp1.size(), 0.);
17     saxpy(10., inp1, inp2, outp.begin());
18     saxpy(10., inp1, inp3, outp.begin());
19     saxpy(10., inp1, std::to_array(cstyle), outp.begin());
20     saxpy(outp.begin(), 10., inp1, inp3);
21 }
```


JÜLICH Forschungszentrum

We can now specify our requirements thoroughly...

```
1  namespace sr = std::ranges;
2  template <std::floating_point D, sr::input_range IR, std::weakly_incrementable OI>
3  requires std::is_same_v<D, std::iter_value_t<IR>> and std::indirectly_writable<OI, D>
4  void saxpy(D a, IR x, IR y, OI z)
5  {
6      sr::transform(x, y, z, [a](auto X, auto Y) { return a * X + Y; });
7  }
8
9  template <std::floating_point D, sr::input_range IR, std::weakly_incrementable OI>
10 requires std::same_as<D, std::iter_value_t<IR>> and std::indirectly_writable<OI, D>
11 void saxpy(OI z, D a, IR x, IR y)
12 {
13     sr::transform(x, y, z, [a](const auto& X, const auto& Y) { return a * X + Y; });
14 }
15
```

Look up cppreference.com and find out what pre-defined concepts and ranges are available in the standard library.

Exercise 1.9:

The program `examples/demo_saxpy/saxpy_9.cc` contains this last version with the requirements on template parameters as well as two overloads. Verify that even if the two functions are both function templates with 4 function parameters, they are indeed distinct for the compiler. Depending on the placement of our arguments, one or the other version is chosen. Try changing data types uniformly in all parameters. Try using different numeric types between source, destination arrays. Try changing container types for the 3 containers involved.

Formatted output

```cpp
for (auto i = 0UL; i < 100UL; ++i) {
    std::cout << "i = " << i
        << ", E_1 = " << cos(i * wn)
        << ", E_2 = " << sin(i * wn)
        << "\n";
}
```

```
i = 5, E_1 = 0.55557, E_2 = 0.83147
i = 6, E_1 = 0.382683, E_2 = 0.92388
i = 7, E_1 = 0.19509, E_2 = 0.980785
i = 8, E_1 = 6.12323e-17, E_2 = 1
i = 9, E_1 = -0.19509, E_2 = 0.980785
i = 10, E_1 = -0.382683, E_2 = 0.92388
i = 11, E_1 = -0.55557, E_2 = 0.83147
```

- While convenient and type safe and extensible, the interface of `ostream` objects like `std::cout` isn't by itself conducive to regular well-formatted output

Formatted output

```cpp
for (auto i = 0UL; i < 100UL; ++i) {
    std::cout << "i = " << i
        << ", E_1 = " << cos(i * wn)
        << ", E_2 = " << sin(i * wn)
        << "\n";
}
```

```
i = 5, E_1 = 0.55557, E_2 = 0.83147
i = 6, E_1 = 0.382683, E_2 = 0.92388
i = 7, E_1 = 0.19509, E_2 = 0.980785
i = 8, E_1 = 6.12323e-17, E_2 = 1
i = 9, E_1 = -0.19509, E_2 = 0.980785
i = 10, E_1 = -0.382683, E_2 = 0.92388
i = 11, E_1 = -0.55557, E_2 = 0.83147
```

- While convenient and type safe and extensible, the interface of `ostream` objects like `std::cout` isn't by itself conducive to regular well-formatted output
- C `printf` often has a simpler path towards visually uniform columnar output, although it is neither type safe nor extensible
- C++ `<iomanip>` header allows formatting with a great deal of control, but has a verbose and inconsistent syntax

Formatted output

```cpp
for (auto i = 0UL; i < 100UL; ++i) {
    std::cout << format(
        "i = {:>4d}, E_1 = {:< 12.8f}, "
        "E_2 = {:< 12.8f}\n",
         i, cos(i * wn), sin(i * wn));
}
```

```
i =    5, E_1 =  0.55557023 , E_2 =  0.83146961
i =    6, E_1 =  0.38268343 , E_2 =  0.92387953
i =    7, E_1 =  0.19509032 , E_2 =  0.98078528
i =    8, E_1 =  0.00000000 , E_2 =  1.00000000
i =    9, E_1 = -0.19509032 , E_2 =  0.98078528
i =   10, E_1 = -0.38268343 , E_2 =  0.92387953
i =   11, E_1 = -0.55557023 , E_2 =  0.83146961
```

- While convenient and type safe and extensible, the interface of `ostream` objects like `std::cout` isn't by itself conducive to regular well-formatted output
- C `printf` often has a simpler path towards visually uniform columnar output, although it is neither type safe nor extensible
- C++ `<iomanip>` header allows formatting with a great deal of control, but has a verbose and inconsistent syntax
- C++20 introduced the `<format>` header, which introduces Python like string formatting
- Based on the open source `fmt` library.

Formatted output

```
1  for (auto i = 0UL; i < 100UL; ++i) {
2      std::cout << format(
3          "i = {:>4d}, E_1 = {:< 12.8f}, "
4          "E_2 = {:< 12.8f}\n",
5           i, cos(i * wn), sin(i * wn));
6  }
```

```
i =    5, E_1 =  0.55557023 , E_2 =  0.83146961
i =    6, E_1 =  0.38268343 , E_2 =  0.92387953
i =    7, E_1 =  0.19509032 , E_2 =  0.98078528
i =    8, E_1 =  0.00000000 , E_2 =  1.00000000
i =    9, E_1 = -0.19509032 , E_2 =  0.98078528
i =   10, E_1 = -0.38268343 , E_2 =  0.92387953
i =   11, E_1 = -0.55557023 , E_2 =  0.83146961
```

Perfectly aligned, as all numeric output should be.

- While convenient and type safe and extensible, the interface of `ostream` objects like `std::cout` isn't by itself conducive to regular well-formatted output
- C `printf` often has a simpler path towards visually uniform columnar output, although it is neither type safe nor extensible
- C++ `<iomanip>` header allows formatting with a great deal of control, but has a verbose and inconsistent syntax
- C++20 introduced the `<format>` header, which introduces Python like string formatting
- Based on the open source `fmt` library.
- Elegant. Safe. Fast. Extensible.

Formatted output

- `std::format("format string {}, {} etc.", args...)` takes a compile time constant format string and a parameter pack to produce a formatted output string
- `std::vformat` can be used if the format string is not known at compilation time
- If instead of receiving output as a newly created string, output into a container or string is desired, `std::format_to` or `std::format_to_n` are available
- The format string contains python style placeholder braces to be filled with formatted values from the argument list
- The braces can optionally contain `id : spec` descriptors. `id` is a 0 based index to choose an argument from `args...` for that slot. `spec` controls how the value is to be written: width, precision, alignment, padding, base of numerals etc. Details of the format specifiers can be found here!

std::print

- Introduced in C++23
- Formats using the `std::format` syntax, but then directs the output to stdout, as if you had written `std::cout << std::format(...);`
- Formatting capabilities were extended to containers (ranges in general), date/time utilities

```
1  std::print("Hello world!\n");
2  std::print("answer = {: >12.8f}\n", d);
3  std::print("{}\n", v);
4  std::print("{}\n", t1 - t0);
```

Exercise 1.10:

A simple example demonstrating the text formatting library of C++20 is in `examples/format1.cc`. Replace cout+format by the equivalent use of `std::print`!

Templates: not a mere text substitution engine!

Template specialisation

```
template <class T>
class vector {
  // implementation of a general
  // vector for any type T
};
template <>
class vector<bool> {
  // Store the true false values
  // in a compressed way, i.e.,
  // 32 of them in a single int
};
void somewhere_else()
{
  vector<bool> A;
  // Uses the special implementation
}
```

- Templates are defined to work with generic template parameters
- But special values of those parameters, which should be treated differently
- If a matching specialisation is found, it is preferred over the general template

```
template <class A, class B>
constexpr auto are_same = false;
template <class A>
constexpr auto are_same<A, A> = true;
static_assert(are_same<int, long>); // Fails
using Integer = int;
static_assert(are_same<int, Integer>); // Passes
```

Not a text substitution engine!

Recursion and integer arithmetic

```
1  template <unsigned N> constexpr unsigned fact = N * fact<N-1>;
2  template <> constexpr unsigned fact<0> = 1U;
3  static_assert(fact<7> == 5040)
```

- Templates support recursive instantiation
- Combined with specialisation to terminate recursion
- Recursion and specialisation can be used to emulate “loop” like calculations via tail-recursion

Exercise 1.11:

The example source file `examples/no_textsub.cc` demonstrates recursion and specialisation in templates, and uses **`static_assert`** to verify that the arithmetic calculations in this case indeed happen during compilation.

Not a text substitution engine!

Because: SFINAE

```
template <bool Cond, class T> struct enable_if {};
template <class T> struct enable_if<true, T> { using type = T; }
template <class T>
auto func(T x) -> enable_if<sizeof(T) == 8UL, T>::type {
//impl1
}
template <class T>
auto func(T x) -> enable_if<sizeof(T) != 8UL, T>::type {
//impl2
}
```

- **S**ubstitution **F**ailure **I**s **N**ot **A**n **E**rror
- If substituting a template parameter results in incomplete or invalid function declarations, that overload is ignored.
- The compiler simply tries to find another match
- If it can't find any, then you have an error
- `func(1)` will resolve to the second version and `func(1.0)` will resolve to the first version during compilation!

Not a text substitution engine!

Because: concepts

```
template <class T>
auto func(T x) -> T requires (sizeof(T) == 8UL) {
//impl1
}
template <class T>
auto func(T x) -> T requires (sizeof(T) != 8UL) {
//impl2
}
```

- Different implementations can be provided requiring different properties of the input type
- Before C++20, this sort of selection was done using `std::enable_if`. Now, `concepts` provide a far cleaner alternative.
- Again, `func(1)` will resolve to the second version and `func(1.0)` will resolve to the first version during compilation!

Constrained templates

- We created overloaded functions so that different strategies can be employed for different input types

```
auto power(double x, double y) -> double ;
auto power(double x, int i) -> double ;
```

Constrained templates

- We created overloaded functions so that different strategies can be employed for different input types

```
auto power(double x, double y) -> double ;
auto power(double x, int i) -> double ;
```

- Same strategy for different types: function templates

```
template <class T> auto power(double x, T i) -> double ;
```

Constrained templates

- We created overloaded functions so that different strategies can be employed for different input types

```
auto power(double x, double y) -> double ;
auto power(double x, int i) -> double ;
```

- Same strategy for different types: function templates

```
template <class T> auto power(double x, T i) -> double ;
```

- Can we combine the two, so that we have two function templates, both looking like the above, but one is automatically selected whenever `T` is an integral type and the other whenever `T` is a floating point type ?

Constrained templates

- We created overloaded functions so that different strategies can be employed for different input types

```
auto power(double x, double y) -> double ;
auto power(double x, int i) -> double ;
```

- Same strategy for different types: function templates

```
template <class T> auto power(double x, T i) -> double ;
```

- Can we combine the two, so that we have two function templates, both looking like the above, but one is automatically selected whenever `T` is an integral type and the other whenever `T` is a floating point type ?
- Some way to impose requirements on permissible matches for the template parameters. Something like:

```
template <class T> auto power(double x, T i) -> double requires floating_point<T>;
template <class T> auto power(double x, T i) -> double requires integer<T>;
```

Constrained templates

- We created overloaded functions so that different strategies can be employed for different input types

```
auto power(double x, double y) -> double ;
auto power(double x, int i) -> double ;
```

- Same strategy for different types: function templates

```
template <class T> auto power(double x, T i) -> double ;
```

- Can we combine the two, so that we have two function templates, both looking like the above, but one is automatically selected whenever `T` is an integral type and the other whenever `T` is a floating point type ?
- Some way to impose requirements on permissible matches for the template parameters. Something like:

```
template <class T> auto power(double x, T i) -> double requires floating_point<T>;
template <class T> auto power(double x, T i) -> double requires integer<T>;
```

- If we could do that, we can combine the generality of templates with the selectiveness of function overloading

Constrained templates

- We created overloaded functions so that different strategies can be employed for different input types

```
auto power(double x, double y) -> double ;
auto power(double x, int i) -> double ;
```

- Same strategy for different types: function templates

```
template <class T> auto power(double x, T i) -> double ;
```

- Can we combine the two, so that we have two function templates, both looking like the above, but one is automatically selected whenever `T` is an integral type and the other whenever `T` is a floating point type ?
- Some way to impose requirements on permissible matches for the template parameters. Something like:

```
template <class T> auto power(double x, T i) -> double requires floating_point<T>;
template <class T> auto power(double x, T i) -> double requires integer<T>;
```

- If we could do that, we can combine the generality of templates with the selectiveness of function overloading
- We can. Or rather, we always could with C++ templates. But now the syntax is easier.

Concepts

Named requirements on template parameters

```
template <unsigned X> concept PowerOfTwo = (X != 0 && (X & (X-1)) == 0);
```


JÜLICH Forschungszentrum

Concepts

Named requirements on template parameters

```
template <unsigned X> concept PowerOfTwo = std::has_single_bit(X);
```


JÜLICH Forschungszentrum

Concepts

Named requirements on template parameters

```
template <unsigned X> concept PowerOfTwo = std::has_single_bit(X);
constexpr auto flag1 = PowerOfTwo<2048U>; // Compiler sets flag1 to True
constexpr auto flag2 = PowerOfTwo<2056U>; // Compiler sets flag2 to False
```

Concepts

Named requirements on template parameters

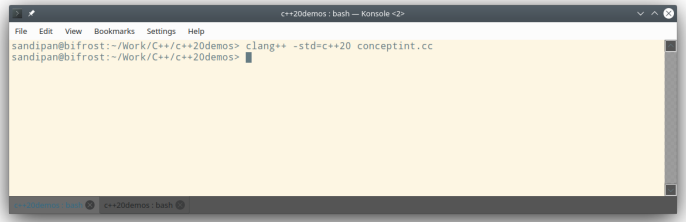
```
template <unsigned X> concept PowerOfTwo = std::has_single_bit(X);
template <class T, unsigned N> requires PowerOfTwo<N>
struct MyMatrix {
    // code which assumes that the square matrix size is a power of two
};
```


JÜLICH Forschungszentrum

Concepts

Named requirements on template parameters

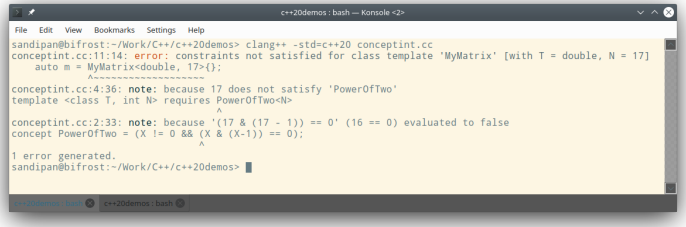
```
template <unsigned X> concept PowerOfTwo = std::has_single_bit(X);
template <class T, unsigned N> requires PowerOfTwo<N>
struct MyMatrix {
    // code which assumes that the square matrix size is a power of two
};
auto main() -> int
{
    auto m = MyMatrix<double, 16U>{};
}
```



Concepts

Named requirements on template parameters

```
template <unsigned X> concept PowerOfTwo = std::has_single_bit(X);
template <class T, unsigned N> requires PowerOfTwo<N>
struct MyMatrix {
    // code which assumes that the square matrix size is a power of two
};
auto main() -> int
{
    auto m = MyMatrix<double, 17U>{};
}
```



JÜLICH Forschungszentrum

Concepts

Named requirements on template parameters

```
template <unsigned X> concept PowerOfTwo = std::has_single_bit(X);
template <class T> concept Number = std::integral<T> or std::floating_point<T>;
```


JÜLICH Forschungszentrum

Concepts

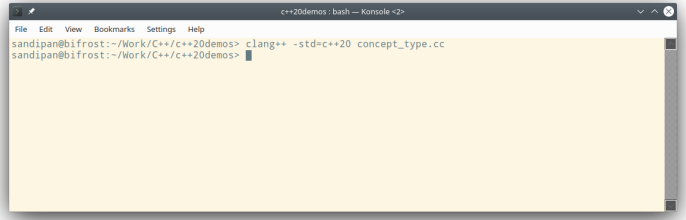
Named requirements on template parameters

```
template <unsigned X> concept PowerOfTwo = std::has_single_bit(X);
template <class T> concept Number = std::integral<T> or std::floating_point<T>;
template <class T, unsigned N> requires Number<T> && PowerOfTwo<N>
struct MyMatrix {
    // assume that the square matrix size is a power of two, and T is a numeric type
};
```

Concepts

Named requirements on template parameters

```cpp
template <unsigned X> concept PowerOfTwo = std::has_single_bit(X);
template <class T> concept Number = std::integral<T> or std::floating_point<T>;
template <class T, unsigned N> requires Number<T> && PowerOfTwo<N>
struct MyMatrix {
    // assume that the square matrix size is a power of two, and T is a numeric type
};
auto main() -> int
{
    auto m = MyMatrix<double, 16U>{};
}
```

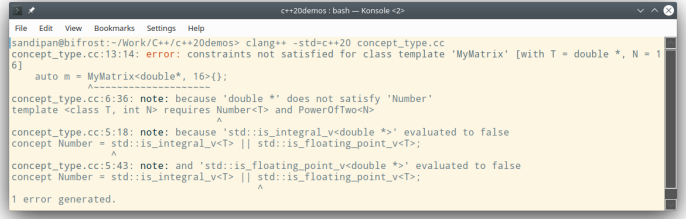


JÜLICH Forschungszentrum

Concepts

Named requirements on template parameters

```cpp
template <unsigned X> concept PowerOfTwo = std::has_single_bit(X);
template <class T> concept Number = std::integral<T> or std::floating_point<T>;
template <class T, unsigned N> requires Number<T> && PowerOfTwo<N>
struct MyMatrix {
    // assume that the square matrix size is a power of two, and T is a numeric type
};
auto main() -> int
{
    auto m = MyMatrix<double*, 16U>{};
}
```



Concepts

Named requirements on template parameters

- `concept`s are named requirements on template parameters, such as `floating_point`, `contiguous_range`
- If `MyAPI` is a `concept`, and `T` is a template parameter, `MyAPI<T>` evaluates at compile time to either true or false.
- Concepts can be combined using conjunctions (`&&`) and disjunctions (`||`) to make other concepts.
- A `requires` *clause* introduces a constraint on a template type

A suitably designed set of concepts can greatly improve readability of template code

Creating concepts

```
template <template-pars>
concept conceptname = constraint_expr;
```

```
template <class T>
concept Integer = std::is_integral_v<T>;
template <class D, class B>
concept Derived = std::is_base_of<B, D>;
```

```
class Counters;
template <class T>
concept Integer_ish = Integer<T> ||
                      Derived<T,Counters>;
```

```
template <class T>
concept Addable = requires (T a, T b) {
    { a + b };
};
template <class T>
concept Indexable = requires(T A) {
    { A[0UL] };
};
```

- Out of a simple `type_traits` style boolean expression
- Combine with logical operators to create more complex requirements
- The **`requires`** *expression* allows creation of syntactic requirements as shown in the last two examples.

Creating concepts

```
template <template-pars>
concept conceptname = constraint_expr;
```

```
template <class T>
concept Integer = std::is_integral_v<T>;
template <class D, class B>
concept Derived = std::is_base_of<B, D>;
```

```
class Counters;
template <class T>
concept Integer_ish = Integer<T> ||
                      Derived<T,Counters>;
```

```
template <class T>
concept Addable = requires (T a, T b) {
    { a + b };
};
template <class T>
concept Indexable = requires(T A) {
    { A[0UL] };
};
```

- Out of a simple `type_traits` style boolean expression
- Combine with logical operators to create more complex requirements
- The **`requires`** *expression* allows creation of syntactic requirements as shown in the last two examples.

Creating concepts

```
template <template-pars>
concept conceptname = constraint_expr;
```

```
template <class T>
concept Integer = std::is_integral_v<T>;
template <class D, class B>
concept Derived = std::is_base_of<B, D>;
```

```
class Counters;
template <class T>
concept Integer_ish = Integer<T> ||
                      Derived<T,Counters>;
```

```
template <class T>
concept Addable = requires (T a, T b) {
    { a + b };
};
template <class T>
concept Indexable = requires(T A) {
    { A[0UL] };
};
```

- Out of a simple `type_traits` style boolean expression
- Combine with logical operators to create more complex requirements
- The **`requires`** *expression* allows creation of syntactic requirements as shown in the last two examples.

JÜLICH Forschungszentrum

Creating concepts

```
template <template-pars>
concept conceptname = constraint_expr;

template <class T>
concept Integer = std::is_integral_v<T>;
template <class D, class B>
concept Derived = std::is_base_of<B, D>;

class Counters;
template <class T>
concept Integer_ish = Integer<T> ||
                      Derived<T,Counters>;

template <class T>
concept Addable = requires (T a, T b) {
    { a + b };
};
template <class T>
concept Indexable = requires(T A) {
    { A[0UL] };
};
```

- Out of a simple `type_traits` style boolean expression
- Combine with logical operators to create more complex requirements
- The **`requires`** *expression* allows creation of syntactic requirements as shown in the last two examples.
- The requires expression can contain a parameter list and a brace enclosed sequence of requirements, which can be:
  - type requirements, e.g., typename T::value_type;
  - simple requirements as shown on the left
  - compound requirements with optional return type constraints, e.g.,

```
{ A[0UL] } -> convertible_to<int>;
```

Using concepts

```
template <class T>
requires Integer_ish<T>
auto categ0(T&& i, double x) -> T;

template <class T>
auto categ1(T&& i, double x) -> T
    requires Integer_ish<T>;

template <Integer_ish T>
auto categ2(T&& i, double x) -> T;

void erase(Integer_ish auto&& i)
```

To constrain template parameters, one can

- place a `requires` clause immediately after the template parameter list
- place a `requires` clause after the function parameter parentheses
- Use the `concept` name in place of `class` or `typename` in the template parameter list
- Use `ConceptName auto` in the function parameter list

JÜLICH Forschungszentrum

Using concepts

```
template <class T>
requires Integer_ish<T>
auto categ0(T&& i, double x) -> T;

template <class T>
auto categ1(T&& i, double x) -> T
    requires Integer_ish<T>;

template <Integer_ish T>
auto categ2(T&& i, double x) -> T;

void erase(Integer_ish auto&& i)
```

To constrain template parameters, one can

- place a `requires` clause immediately after the template parameter list
- place a `requires` clause after the function parameter parentheses
- Use the `concept` name in place of `class` or `typename` in the template parameter list
- Use `ConceptName auto` in the function parameter list

JÜLICH Forschungszentrum

Using concepts

```
template <class T>
requires Integer_ish<T>
auto categ0(T&& i, double x) -> T;

template <class T>
auto categ1(T&& i, double x) -> T
    requires Integer_ish<T>;

template <Integer_ish T>
auto categ2(T&& i, double x) -> T;

void erase(Integer_ish auto&& i)
```

To constrain template parameters, one can

- place a `requires` clause immediately after the template parameter list
- place a `requires` clause after the function parameter parentheses
- Use the `concept` name in place of `class` or `typename` in the template parameter list
- Use `ConceptName auto` in the function parameter list

JÜLICH Forschungszentrum

Using concepts

```
template <class T>
requires Integer_ish<T>
auto categ0(T&& i, double x) -> T;

template <class T>
auto categ1(T&& i, double x) -> T
    requires Integer_ish<T>;

template <Integer_ish T>
auto categ2(T&& i, double x) -> T;

void erase(Integer_ish auto&& i)
```

To constrain template parameters, one can

- place a `requires` clause immediately after the template parameter list
- place a `requires` clause after the function parameter parentheses
- Use the `concept` name in place of `class` or `typename` in the template parameter list
- Use `ConceptName auto` in the function parameter list

JÜLICH Forschungszentrum

Declaring function input parameters with auto

```
1  template <class T>
2  auto sqr(const T& x) { return x * x; }
```

- Because of syntax introduced for functions with constrained templates in C++20, we have a new way to write fully unconstrained function templates...

Declaring function input parameters with auto

```
1  auto sqr(const auto& x) { return x * x; }
```

- Because of syntax introduced for functions with constrained templates in C++20, we have a new way to write fully unconstrained function templates...
- Functions with `auto` in their parameter list are implicitly function templates

Declaring function input parameters with auto

Exercise 1.12:

The program `examples/gcd_w_concepts.cc` shows a very small concept definition and its use in a function calculating the greatest common divisor of two integers.

Exercise 1.13:

The series of programs `examples/generic_func1.cc` through `generic_func4.cc` shows some trivial functions implemented with templates with and without constraints. The files contain plenty of comments explaining the rationale and use of concepts.

Overloading based on concepts

```
template <class N>
concept Number = std::floating_point<N>
                 or std::integral<N>;


void proc(Number auto&& x) {
    std::cout << "Called proc for numbers";
}



auto main() -> int {
    proc(-1);
    proc(88UL);
    proc("0118 999 88199 9119725   3");
    proc(3.141);
    proc("eighty"s);
}
```

- Constraints on template parameters are not just "documentation" or decoration.

JÜLICH Forschungszentrum

Overloading based on concepts

```
template <class N>
concept Number = std::floating_point<N>
                 or std::integral<N>;
template <class N>
concept NotNumber = not Number<N>;
void proc(Number auto&& x) {
    std::cout << "Called proc for numbers";
}
void proc(NotNumber auto&& x) {
    std::cout << "Called proc for non-numbers";
}
auto main() -> int {
    proc(-1);
    proc(88UL);
    proc("0118 999 88199 9119725   3");
    proc(3.141);
    proc("eighty"s);
}
```

- Constraints on template parameters are not just "documentation" or decoration.
- The compiler can choose between different versions of a function based on concepts

Overloading based on concepts

```
template <class N>
concept Number = std::floating_point<N>
                 or std::integral<N>;
template <class N>
concept NotNumber = not Number<N>;
void proc(Number auto&& x) {
    std::cout << "Called proc for numbers";
}
void proc(NotNumber auto&& x) {
    std::cout << "Called proc for non-numbers";
}
auto main() -> int {
    proc(-1);
    proc(88UL);
    proc("0118 999 88199 9119725   3");
    proc(3.141);
    proc("eighty"s);
}
```

- Constraints on template parameters are not just "documentation" or decoration.
- The compiler can choose between different versions of a function based on concepts
- The version of a function chosen depends on *properties* of the input types, rather than their identities. "It's not who you are underneath, it's what you (can) do that defines you."

Overloading based on concepts

```
template <class N>
concept Number = std::floating_point<N>
                 or std::integral<N>;
template <class N>
concept NotNumber = not Number<N>;
void proc(Number auto&& x) {
    std::cout << "Called proc for numbers";
}
void proc(NotNumber auto&& x) {
    std::cout << "Called proc for non-numbers";
}
auto main() -> int {
    proc(-1);
    proc(88UL);
    proc("0118 999 88199 9119725   3");
    proc(3.141);
    proc("eighty"s);
}
```

- During overload resolution, in case multiple matches are found, the most constrained overload is chosen.

JÜLICH Forschungszentrum

Overloading based on concepts

```
template <class N>
concept Number = std::floating_point<N>
                 or std::integral<N>;
template <class N>
concept NotNumber = not Number<N>;
void proc(Number auto&& x) {
    std::cout << "Called proc for numbers";
}
void proc(NotNumber auto&& x) {
    std::cout << "Called proc for non-numbers";
}
auto main() -> int {
    proc(-1);
    proc(88UL);
    proc("0118 999 88199 9119725   3");
    proc(3.141);
    proc("eighty"s);
}
```

- During overload resolution, in case multiple matches are found, the most constrained overload is chosen.
- Not based on any inheritance relationships

Overloading based on concepts

```
template <class N>
concept Number = std::floating_point<N>
                 or std::integral<N>;
template <class N>
concept NotNumber = not Number<N>;
void proc(Number auto&& x) {
    std::cout << "Called proc for numbers";
}
void proc(NotNumber auto&& x) {
    std::cout << "Called proc for non-numbers";
}
auto main() -> int {
    proc(-1);
    proc(88UL);
    proc("0118 999 88199 9119725   3");
    proc(3.141);
    proc("eighty"s);
}
```

- During overload resolution, in case multiple matches are found, the most constrained overload is chosen.
- Not based on any inheritance relationships
- Not a “quack like a duck, or bust” approach either.

Overloading based on concepts

```
template <class N>
concept Number = std::floating_point<N>
                 or std::integral<N>;
template <class N>
concept NotNumber = not Number<N>;
void proc(Number auto&& x) {
    std::cout << "Called proc for numbers";
}
void proc(NotNumber auto&& x) {
    std::cout << "Called proc for non-numbers";
}
auto main() -> int {
    proc(-1);
    proc(88UL);
    proc("0118 999 88199 9119725   3");
    proc(3.141);
    proc("eighty"s);
}
```

- During overload resolution, in case multiple matches are found, the most constrained overload is chosen.
- Not based on any inheritance relationships
- Not a “quack like a duck, or bust” approach either.
- Entirely compile time mechanism

Exercise 1.14:

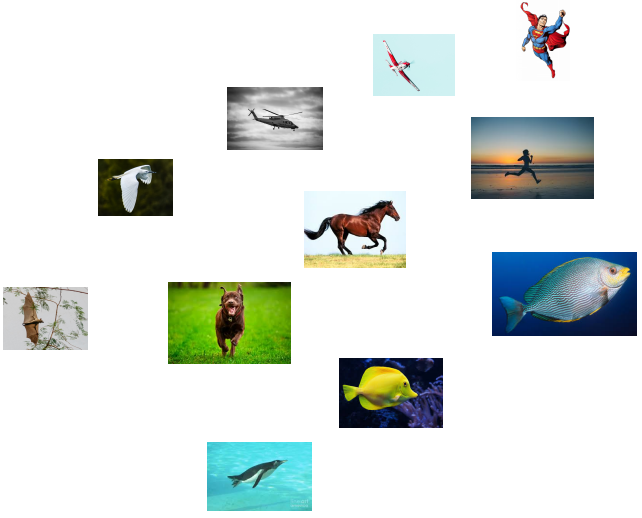
Check how you can use concepts to implement alternative versions of a function based on properties of the input parameters! The program `examples/overload_w_concepts.cc` contains the code just shown. Can you add another overload that is picked if the input type is an array? This means, if `X` is the input parameter, `X[i]` is syntactically valid for unsigned integer `i`. The array version should be picked up if the input is a `vector`, `array`, etc. , but also `string`. How would you prevent the `string` and C-style strings picking the array version?

Predefined useful concepts

Many concepts useful in building our own concepts are available in the standard library header `<concepts>`.

- `same_as`
- `convertible_to`
- `signed_ingegral`, `unsigned_integral`
- `floating_point`
- `assignable_from`
- `swappable`, `swappable_with`
- `derived_from`
- `move_constructible`, `copy_constructible`
- `invocable`
- `predicate`
- `relation`

Concepts: summary



f(those who can fly)

f(runners)

f(swimmers)

JÜLICH Forschungszentrum

Ranges

The range concept

```
1  def python_sum(Container, start=0):
2      res = start
3      for x in Container:
4          res += x
5      return res
```


JÜLICH Forschungszentrum

The range concept

```
1  auto sum(auto&& Container, auto start = 0) {
2      for (auto&& el : Container) start += el;
3      return start;
4  }
```

A C++ version can be as compact as the python version, but then it will also have the same problems:

- We did not ensure that the first parameter is a container. Just calling it `Container` isn't good enough
- We did not ensure that the type of the second parameter was the data type of the first

The range concept

```
1  template <class T> requires has_referenceable_begin_end<T>
2  auto sum(T&& Container, element_type_of<T> start = 0) {
3      for (auto&& el : Container) start += el;
4      return start;
5  }
```

As compact as the python version, but with the same problems:

- Use `concepts` to put constraints on the function template!
- What matters for the code inside `sum` to work is the presence of `begin` and `end` functions, which return iterator types
- The type of the second parameter should somehow be obtained from the first using some kind of meta-function

The range concept

```
1  template <class T> using cleanup = std::remove_cvref_t<T>;
2  template <class T> using element = std::iter_value_t<cleanup<T>>;
3  template <class T> requires std::ranges::forward_range<T>
4  auto sum(T&& a, element<T> start) {
5      for (auto&& el : a) start += el;
6      return start;
7  }
```

Using definitions in the `ranges` header, we have a few more lines, but:

- Only available when `T` really is a sequence where forward iteration is possible
- The second parameter must be the element type of the first one

The range concept

```
1  template <class T> using cleanup = std::remove_cvref_t<T>;
2  template <class T> using element = std::iter_value_t<cleanup<T>>;
3  template <class T> requires std::ranges::forward_range<T>
4  auto sum(T&& a, element<T> start) {
5      for (auto&& el : a) start += el;
6      return start;
7  }
8  template <class ... T, class U> requires ((std::same_as<T, U>) && ...)
9  auto sum(U&& start, T&& ... a) { return (start + ... + a); }
```


JÜLICH Forschungszentrum

The range concept

```
1  template <class T> using cleanup = std::remove_cvref_t<T>;
2  template <class T> using element = std::iter_value_t<cleanup<T>>;
3  template <class T> requires std::ranges::forward_range<T>
4  auto sum(T&& a, element<T> start) {
5      for (auto&& el : a) start += el;
6      return start;
7  }
8  template <class ... T, class U> requires ((std::same_as<T, U>) && ...)
9  auto sum(U&& start, T&& ... a) { return (start + ... + a); }
```

- We can overload with a different function template taking the same number of generic parameters, but different constraints
- We can overload with a variadic function template of the same name, so long as the constraints are different

The range concept

```cpp
template <class T> using cleanup = std::remove_cvref_t<T>;
template <class T> using element = std::iter_value_t<cleanup<T>>;
template <class T> requires std::ranges::forward_range<T>
auto sum(T&& a, element<T> start) {
    for (auto&& el : a) start += el;
    return start;
}
template <class ... T, class U> requires ((std::same_as<T, U>) && ...)
auto sum(U&& start, T&& ... a) { return (start + ... + a); }
```

```cpp
auto main() -> int {
    std::vector v{ 1, 2, 3, 4, 5 };
    std::list l{9.1, 9.2, 9.3, 9.4, 9.5, 9.6};
    std::cout << sum(v, 0) << "\n";
    std::cout << sum(l, 0.) << "\n";
    std::cout << sum(4.5, 9.) << "\n";
    std::cout << sum(4.5, 3.4, 5., 9.) << "\n";
}
```

Tremendous flexibility, but still resolved at compilation time!

The range concept

```cpp
std::vector v{ 1, 2, 3, 4, 5, 6, 7, 8, 9 };
// before std::ranges we did this...
std::reverse(v.begin(), v.end());
std::rotate(v.begin(), v.begin() + 3, v.end());
std::sort(v.begin(), v.end());
```

```cpp
std::vector v{ 1, 2, 3, 4, 5, 6, 7, 8, 9 };
namespace sr = std::ranges;
sr::reverse(v);
sr::rotate(v, v.begin() + 3);
sr::sort(v);
```

- The `<ranges>` header defines a special kind of concept describing entities with a start and an end.
- The range concept is defined in terms of
  - the existence of an iterator type and a "sentinel" type.
  - the iterator should behave like an iterator, e.g., allow `++it` `*it` etc.
  - it should be possible to compare the iterators with other iterators or with a sentinel for equality.
  - A `begin()` function returning an iterator and an `end()` function returning a sentinel
- Other useful `concepts` defined in the `ranges` header:
  - `view` is a range with constant time copy/move/assignment
  - `sr::sized_range`, `input_range`, `output_range`
  - `borrowed_range` : a type such that its iterators can be returned without the danger of dangling.
- The `<algorithm>` header has many algorithms taking ranges as inputs instead of pairs of iterators

The range concept

```
1  // examples/ranges/ranges0.cc
2  #include <ranges>
3  #include <span>
4  auto sum(std::ranges::input_range auto&& seq) {
5      std::iter_value_t<decltype(seq)> ans{};
6      for (auto x : seq) ans += x;
7      return ans;
8  }
9  auto main() -> int
10 {
11     //using various namespaces;
12     cout << "vector   : " << sum(vector(  { 9,8,7,2 } )) << "\n";
13     cout << "list     : " << sum(list(    { 9,8,7,2 } )) << "\n";
14     cout << "valarray : " << sum(valarray({ 9,8,7,2 } )) << "\n";
15     cout << "array    : "
16          << sum(array<int,4>({ 9,8,7,2 } )) << "\n";
17     cout << "array    : "
18          << sum(array<string, 4>({ "9"s,"8"s,"7"s,"2"s } )) << "\n";
19     int A[]{1,2,3};
20     cout << "span(built-in array) : " << sum(span(A)) << "\n";
21 }
```


JÜLICH Forschungszentrum

Exercise 1.15:

The function template `sum` in `examples/ranges/ranges0.cc` accepts any input range, i.e., some entity whose iterators satisfy the requirements of an `input_iterator`. Notice how we obtain the value type of the range

Fun with ranges and views

```
1  // examples/ranges/iota.cc
2  #include <ranges>
3  #include <iostream>
4  auto main() -> int {
5      namespace sv = std::views;
6      for (auto i : sv::iota(1UL)) {
7          if ((i+1) % 10000UL == 0UL) {
8              std::cout << i << ' ';
9              if ((i+1) % 100000UL == 0UL)
10                 std::cout << '\n';
11             if (i >= 1000000000UL) break;
12         }
13     }
14 }
```

- All containers are ranges, but not all ranges are containers
- `std::string_view` is a perfectly fine range. Has iterators with the right properties. Has `begin()` and `end()` functions. It does not own the contents, but "ownership" is not part of the idea of a range.
- We could take this further by creating `views` which need not actually contain any objects of the sequence, but simply fake it when we dereference the iterators.
- Example: the standard view `std::views::iota(integer)` gives us an infinite sequence of integers starting at a given value.

JÜLICH Forschungszentrum

Fun with ranges and views

```cpp
// examples/ranges/iota.cc
#include <ranges>
#include <iostream>
auto main() -> int {
    namespace sv = std::views;
    for (auto i : sv::iota(1UL)) {
        if ((i+1) % 10000UL == 0UL) {
            std::cout << i << ' ';
            if ((i+1) % 100000UL == 0UL)
                std::cout << '\n';
            if (i >= 1000000000UL) break;
        }
    }
}
```

- All containers are ranges, but not all ranges are containers
- `std::string_view` is a perfectly fine range. Has iterators with the right properties. Has `begin()` and `end()` functions. It does not own the contents, but "ownership" is not part of the idea of a range.
- We could take this further by creating `views` which need not actually contain any objects of the sequence, but simply fake it when we dereference the iterators.
- Example: the standard view `std::views::iota(integer)` gives us an infinite sequence of integers starting at a given value.

JÜLICH Forschungszentrum

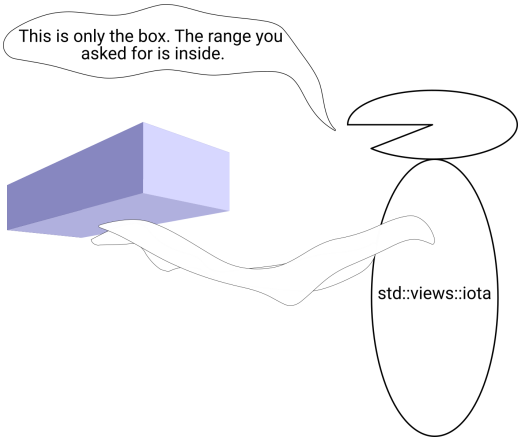
Fun with ranges and views

```
1  // examples/ranges/iota.cc
2  #include <ranges>
3  #include <iostream>
4  auto main() -> int {
5      namespace sv = std::views;
6      for (auto i : sv::iota(1UL)) {
7          if ((i+1) % 10000UL == 0UL) {
8              std::cout << i << ' ';
9              if ((i+1) % 100000UL == 0UL)
10                 std::cout << '\n';
11             if (i >= 1000000000UL) break;
12         }
13     }
14 }
```

- All containers are ranges, but not all ranges are containers
- `std::string_view` is a perfectly fine range. Has iterators with the right properties. Has `begin()` and `end()` functions. It does not own the contents, but "ownership" is not part of the idea of a range.
- We could take this further by creating `views` which need not actually contain any objects of the sequence, but simply fake it when we dereference the iterators.
- Example: the standard view `std::views::iota(integer)` gives us an infinite sequence of integers starting at a given value.

JÜLICH Forschungszentrum

Fun with ranges and views



- All containers are ranges, but not all ranges are containers
- `std::string_view` is a perfectly fine range. Has iterators with the right properties. Has `begin()` and `end()` functions. It does not own the contents, but "ownership" is not part of the idea of a range.
- We could take this further by creating `views` which need not actually contain any objects of the sequence, but simply fake it when we dereference the iterators.
- Example: the standard view `std::views::iota(integer)` gives us an infinite sequence of integers starting at a given value.

Fun with ranges and views

```
1  #include <ranges>
2  #include <iostream>
3  auto main() -> int {
4      namespace sv = std::views;
5      for (auto i : sv::iota(1UL)) {
6          if ((i+1) % 10000UL == 0UL) {
7              std::cout << i << ' ';
8              if ((i+1) % 100000UL == 0UL)
9                  std::cout << '\n';
10             if (i >= 100000000UL) break;
11         }
12     }
13 }
```

- All containers are ranges, but not all ranges are containers
- `std::string_view` is a perfectly fine range. Has iterators with the right properties. Has `begin()` and `end()` functions. It does not own the contents, but "ownership" is not part of the idea of a range.
- We could take this further by creating `views` which need not actually contain any objects of the sequence, but simply fake it when we dereference the iterators.
- Example: the standard view `std::views::iota(integer)` gives us an infinite sequence of integers starting at a given value.

JÜLICH Forschungszentrum

View adaptors

```
1  namespace sv = std::views;
2  std::vector v{1,2,3,4,5};
3  auto v3 = sv::take(v, 3);
4  // v3 is some sort of object so
5  // that it represents the first
6  // 3 elements of v. It does not
7  // own anything, and has constant
8  // time copy/move etc. It's a view.
9
10 // sv::take() is a view adaptor
```

- A `view` is a range with constant time copy, move etc. Think `string_view`
- A view adaptor is a function object, which takes a "viewable" range as an input and constructs a view out of it. `viewable` is defined as "either a `borrowed_range` or already a view.
- A `borrowed_range` is a range so that its iterators can be returned from a function without the danger of dangling: e.g., L-value references to ranges, `std::span`, `std::string_view`...
- View adaptors in the `<ranges>` library have very interesting properties, and make some new ways of coding possible.

View adaptors

```
Adaptor(Viewable) -> View
Viewable | Adaptor -> View
V | A1 | A2 | A3 ... -> View
```

```
Adaptor(Viewable, Args...) ->  View
Adaptor(Args...)(Viewable) ->  View
Viewable | Adaptor(Args...) -> View
```

- A `view` itself is trivially viewable.
- Since a view adaptor produces a view, successive applications of such adaptors makes sense.
- If an adaptor takes only one argument, it can be called using the pipe operator as shown. These adaptors can then be chained to produce more complex adaptors.
- For adaptors taking multiple arguments, there exists an equivalent adaptor taking only the viewable range.

View adaptors

```
Adaptor(Viewable) -> View
Viewable | Adaptor -> View
V | A1 | A2 | A3 ... -> View
```

```
Adaptor(Viewable, Args...) ->  View
Adaptor(Args...)(Viewable) ->  View
Viewable | Adaptor(Args...) -> View
```

- A `view` itself is trivially viewable.
- Since a view adaptor produces a view, successive applications of such adaptors makes sense.
- If an adaptor takes only one argument, it can be called using the pipe operator as shown. These adaptors can then be chained to produce more complex adaptors.
- For adaptors taking multiple arguments, there exists an equivalent adaptor taking only the viewable range.

View adaptors

```
Adaptor(Viewable) -> View
Viewable | Adaptor -> View
V | A1 | A2 | A3 ... -> View
```

```
Adaptor(Viewable, Args...) ->  View
Adaptor(Args...)(Viewable) ->  View
Viewable | Adaptor(Args...) -> View
```

- A `view` itself is trivially viewable.
- Since a view adaptor produces a view, successive applications of such adaptors makes sense.
- If an adaptor takes only one argument, it can be called using the pipe operator as shown. These adaptors can then be chained to produce more complex adaptors.
- For adaptors taking multiple arguments, there exists an equivalent adaptor taking only the viewable range.

View adaptors

```
Adaptor(Viewable) -> View
Viewable | Adaptor -> View
V | A1 | A2 | A3 ... -> View
```

```
Adaptor(Viewable, Args...) ->  View
Adaptor(Args...)(Viewable) ->  View
Viewable | Adaptor(Args...) -> View
```

- A `view` itself is trivially viewable.
- Since a view adaptor produces a view, successive applications of such adaptors makes sense.
- If an adaptor takes only one argument, it can be called using the pipe operator as shown. These adaptors can then be chained to produce more complex adaptors.
- For adaptors taking multiple arguments, there exists an equivalent adaptor taking only the viewable range.

View adaptors

```
Adaptor(Viewable) -> View
Viewable | Adaptor -> View
V | A1 | A2 | A3 ... -> View
```

```
Adaptor(Viewable, Args...) ->  View
Adaptor(Args...)(Viewable) ->  View
Viewable | Adaptor(Args...) -> View
```

- A `view` itself is trivially viewable.
- Since a view adaptor produces a view, successive applications of such adaptors makes sense.
- If an adaptor takes only one argument, it can be called using the pipe operator as shown. These adaptors can then be chained to produce more complex adaptors.
- For adaptors taking multiple arguments, there exists an equivalent adaptor taking only the viewable range.

So what are we going to do with this ?

View adaptors

Pretend that you want to verify that $sin^2(x) + cos^2(x) = 1$

JÜLICH Forschungszentrum

View adaptors

Pretend that you want to verify that $sin^2(x) + cos^2(x) = 1$

- Start with a range of integers from 0 to $N = 10000$.
- $R_0 = \{0, 1, 2, 3...\}$

View adaptors

Pretend that you want to verify that $sin^2(x) + cos^2(x) = 1$

- Start with a range of integers from 0 to $N = 10000$.
- Map the integer range to real numbers in the range $[0, 2\pi)$

- $R_0 = \{0, 1, 2, 3...\}$
- $R_1 = T_{10} R_0 = T(n \mapsto \frac{2\pi n}{N}) R_0$

JÜLICH Forschungszentrum

View adaptors

Pretend that you want to verify that $sin^2(x) + cos^2(x) = 1$

- Start with a range of integers from 0 to $N = 10000$.
- Map the integer range to real numbers in the range $[0, 2\pi)$
- Evaluate $sin^2(x) + cos^2(x) - 1$ over the resulting range

- $R_0 = \{0, 1, 2, 3...\}$
- $R_1 = T_{10} R_0 = T(n \mapsto \frac{2\pi n}{N}) R_0$
- $R_2 = T_{21} R_1 = T(x \mapsto (sin^2(x) + cos^2(x) - 1)) R_1$

$$\begin{aligned} R_2 &= T_{21} R_1 = T_{21} T_{10} R_0 \\ &= R_0 | T_{10} | T_{21} \\ &= R_0 | (T_{10} | T_{21}) \end{aligned}$$

JÜLICH Forschungszentrum

View adaptors

Pretend that you want to verify that $sin^2(x) + cos^2(x) = 1$

- Start with a range of integers from 0 to $N = 10000$.
- Map the integer range to real numbers in the range $[0, 2\pi)$
- Evaluate $sin^2(x) + cos^2(x) - 1$ over the resulting range
- If absolute value of any of the values in the result exceeds ϵ, we have found a counter example

- $R_0 = \{0, 1, 2, 3...\}$
- $R_1 = T_{10} R_0 = T(n \mapsto \frac{2\pi n}{N}) R_0$
- $R_2 = T_{21} R_1 = T(x \mapsto (sin^2(x) + cos^2(x) - 1)) R_1$

$$\begin{aligned} R_2 &= T_{21} R_1 = T_{21} T_{10} R_0 \\ &= R_0 | T_{10} | T_{21} \\ &= R_0 | (T_{10} | T_{21}) \end{aligned}$$

JÜLICH Forschungszentrum

View adaptors

Pretend that you want to verify that $sin^2(x) + cos^2(x) = 1$

- Start with a range of integers from 0 to $N = 10000$.
- Map the integer range to real numbers in the range $[0, 2\pi)$
- Evaluate $sin^2(x) + cos^2(x) - 1$ over the resulting range
- If absolute value of any of the values in the result exceeds ϵ, we have found a counter example
- Intuitive left-to-right readability

- $R_0 = \{0, 1, 2, 3...\}$
- $R_1 = T_{10} R_0 = T(n \mapsto \frac{2\pi n}{N}) R_0$
- $R_2 = T_{21} R_1 = T(x \mapsto (sin^2(x) + cos^2(x) - 1)) R_1$

$$\begin{aligned} R_2 &= T_{21} R_1 = T_{21} T_{10} R_0 \\ &= R_0 | T_{10} | T_{21} \\ &= R_0 | (T_{10} | T_{21}) \end{aligned}$$

JÜLICH Forschungszentrum

View adaptors

Pretend that you want to verify that $sin^2(x) + cos^2(x) = 1$

- Start with a range of integers from 0 to $N = 10000$.
- Map the integer range to real numbers in the range $[0, 2\pi)$
- Evaluate $sin^2(x) + cos^2(x) - 1$ over the resulting range
- If absolute value of any of the values in the result exceeds ϵ, we have found a counter example
- Intuitive left-to-right readability

- $R_0 = \{0, 1, 2, 3...\}$
- $R_1 = T_{10}R_0 = T(n \mapsto \frac{2\pi n}{N})R_0$
- $R_2 = T_{21}R_1 = T(x \mapsto (sin^2(x) + cos^2(x) - 1))R_1$

$$\begin{aligned} R_2 &= T_{21}R_1 = T_{21}T_{10}R_0 \\ &= R_0|T_{10}|T_{21} \\ &= R_0|(T_{10}|T_{21}) \end{aligned}$$

```
find . -name "*.cc" | xargs grep "if" | grep -v "constexpr" | less
```

View adaptors

```
find . -name "*.cc" | xargs grep "if" | grep -v "constexpr" | less
```

- Command line of Linux, Mac OS ...

View adaptors

```
find . -name "*.cc" | xargs grep "if" | grep -v "constexpr" | less
```

- Command line of Linux, Mac OS ...
- Small utilities. Each program does one thing, and does it well.

View adaptors

```
find . -name "*.cc" | xargs grep "if" | grep -v "constexpr" | less
```

- Command line of Linux, Mac OS ...
- Small utilities. Each program does one thing, and does it well.
- There is a way to chain them together with the pipe

View adaptors

```
find . -name "*.cc" | xargs grep "if" | grep -v "constexpr" | less
```

- Command line of Linux, Mac OS ...
- Small utilities. Each program does one thing, and does it well.
- There is a way to chain them together with the pipe
- Overall usefulness of the tool set is amplified exponentially!

View adaptors

```
find . -name "*.cc" | xargs grep "if" | grep -v "constexpr" | less
```

- Command line of Linux, Mac OS ...
- Small utilities. Each program does one thing, and does it well.
- There is a way to chain them together with the pipe
- Overall usefulness of the tool set is amplified exponentially!
- What about writing something similar in C++ ?

View adaptors

Pretend that you want to verify that $sin^2(x) + cos^2(x) = 1$

JÜLICH Forschungszentrum

View adaptors

Pretend that you want to verify that $sin^2(x) + cos^2(x) = 1$

- Start with a range of integers from 0 to $N = 10000$. `R0 = iota(0, N)`

JÜLICH Forschungszentrum

View adaptors

Pretend that you want to verify that $sin^2(x) + cos^2(x) = 1$

- Start with a range of integers from 0 to $N = 10000$. `R0 = iota(0, N)`
- Map the integer range to real numbers in the range $[0, 2\pi)$, i.e., perform the transformation $n \mapsto \frac{2\pi n}{N}$ over the range: `R1 = R0 | transform([](int n) -> double { return 2*pi*n/N; })`

View adaptors

Pretend that you want to verify that $sin^2(x) + cos^2(x) = 1$

- Start with a range of integers from 0 to $N = 10000$. `R0 = iota(0, N)`
- Map the integer range to real numbers in the range $[0, 2\pi)$, i.e., perform the transformation $n \mapsto \frac{2\pi n}{N}$ over the range: `R1 = R0 | transform([](int n) -> double { return 2*pi*n/N; })`
- Perform the transformation $x \mapsto sin^2(x) + cos^2(x) - 1$ over the resulting range

```
R2 = R1 | transform([](double x) -> double { return sin(x)*sin(x)+cos(x)*cos(x); } );
```

View adaptors

Pretend that you want to verify that $sin^2(x) + cos^2(x) = 1$

- Start with a range of integers from 0 to $N = 10000$. `R0 = iota(0, N)`
- Map the integer range to real numbers in the range $[0, 2\pi)$, i.e., perform the transformation $n \mapsto \frac{2\pi n}{N}$ over the range: `R1 = R0 | transform([](int n) -> double { return 2*pi*n/N; })`
- Perform the transformation $x \mapsto sin^2(x) + cos^2(x) - 1$ over the resulting range

```
R2 = R1 | transform([](double x) -> double { return sin(x)*sin(x)+cos(x)*cos(x); } );
```

- If absolute value of any of the values in the result exceeds ϵ, we have found a counter example

```
if (any_of(R2, [](auto x){return fabs(x) > eps;})) ...
```

View adaptors

```cpp
auto main() -> int {
    namespace sr = std::ranges;
    namespace sv = std::views;
    using std::numbers::pi;
    constexpr auto npoints = 10'000'00UL;
    constexpr auto eps = 100 * std::numeric_limits<double>::epsilon();
    auto to_0_2pi = [=](size_t idx) -> double {
        return std::lerp(0., 2*pi, idx * 1.0 / npoints);
    };
    auto x_to_fx = [ ](double x) -> double {
        return sin(x) * sin(x) + cos(x) * cos(x) - 1.0;
    };
    auto is_bad = [=](double x){ return std::fabs(x) > eps; };

    auto res = sv::iota(0UL, npoints) | sv::transform(to_0_2pi) | sv::transform(x_to_fx);

    if (sr::any_of(res, is_bad) ) {
        std::cerr << "The relation does not hold.\n";
    } else {
        std::cout << "The relation holds for all inputs\n";
    }
}
```

View adaptors

- The job of each small transform in the previous example was small, simple, easily verified for correctness.
- The view adaptors allow us to chain them to produce a resulting range
- Algorithms like `std::range::any_of` work on ranges, so they can work on the resulting views from chained view adaptors.
- No operation is done on any range when we create the variable `res` above.
- When we try to access an element of the range in the `any_of` algorithm, one element is taken on the fly out of the starting range, fed through the pipeline and catered to `any_of`
- `any_of` does not process the range beyond what is necessary to establish its truth value. The remaining elements in the result array are never calculated.

Exercise 1.16:

The code used for the demonstration of view adaptors is `examples/ranges/trig_views.cc`. Build this code with GCC and Clang.

```
g++ -std=c++20  trig_views.cc
./a.out

clang++ -std=c++20 -stdlib=libc++ trig_views.cc
./a.out
```

Exercise 1.17:

The trigonometric relation we used is true, so not all possibilities are explored. In `examples/ranges/trig_views2.cc` there is another program trying to verify the bogus claim $sin^2(x) < 0.99$. It's mostly true, but sometimes it isn't, so that our **`if`** and **`else`** branches both have work to do. The lambdas in this program have been rigged to print messages before returning. Convince yourself of the following:

- The output from the lambdas come out staggered, which means that the program does not process the entire range for the first transform and then again for the second ...
- Processing stops at the first instance where `any_of` gets a **`true`** answer.

View adaptors

```
1  // examples/ranges/gerund.cc
2      using itertype = std::istream_iterator<std::string>;
3      std::ifstream fin { argv[1] };
4      auto gerund = [](std::string_view w) { return w.ends_with("ing"); };
5      auto in = sr::istream_view<std::string>(fin);
6      std::print("{}\n", in | sv::filter(gerund));
7
```

- `sr::istream_view<T>` creates an (input) iterable range from an input stream. Each element of this range is of the type `T`.
- `sv::filter` is a view adaptor, which when applied to a range, produces another containing only the elements satisfying a given condition
- In the above, `std::print` is shown writing out a range. This works with the Clang standard library `libc++`. Since GCC doesn't have an implementation yet, please use `clang++` for this.

View adaptors

A program to print the alphabetically -first and -last word entered on the command line, excluding the program name.

```cpp
// examples/ranges/views_and_span.cc
auto main(int argc, char* argv[]) -> int
{
    if (argc < 2) return 1;
    namespace sr = std::ranges;
    namespace sv = std::views;

    std::span args(argv, argc);
    auto str = [](auto cstr) -> std::string_view { return cstr; };
    auto [mn, mx] = sr::minmax(args | sv::drop(1) | sv::transform(str));

    std::cout << "Alphabetically first = " << mn << " last = " << mx << "\n";
}
```

Ranges improvements in C++23

```cpp
std::vector v { "apples"s, "oranges"s,
                "mangos"s, "bananas"s };

for (auto [i, fruit] : sv::enumerate(v)) {
    std::print("{}: {}\n", i, fruit);
}
```

```
$ G -std=c++23 enumerate.cc
$ ./a.out
0: apples
1: oranges
2: mangos
3: bananas
```

With the definitions
`namespace sr = std::ranges`,
and `namespace sv = sr::views`

- `sv::enumerate`
- `sv::zip`
- `sv::zip_transform`
- `sv::adjacent`
- `sr::to`
- Formatting ranges

Ranges improvements in C++23

```
1  std::vector v { "apples"s, "oranges"s,
2                  "mangos"s, "bananas"s };
3
4  for (auto [fruit1, fruit2] :
5           sv::zip(v, sv::reverse(v))) {
6      std::print("{}: {}\n", fruit1, fruit2);
7  }
```

```
$ G -std=c++23 zip.cc
$ ./a.out
apples: bananas
oranges: mangos
mangos: oranges
bananas: apples
```

With the definitions `namespace sr = std::ranges`, and `namespace sv = sr::views`

- `sv::enumerate`
- `sv::zip`
- `sv::zip_transform`
- `sv::adjacent`
- `sr::to`
- Formatting ranges

Ranges improvements in C++23

```
for (auto s : sv::zip_transform(
          [](auto&& s1, auto&& s2) {
              return format("{} <--> {}", s1, s2);
          },
          v, sv::reverse(v))) {
              std::cout << s << "\n";
}
```

```
$ G -std=c++23 zip_transform.cc
$ ./a.out
apples <--> bananas
oranges <--> mangos
mangos <--> oranges
bananas <--> apples
```

With the definitions `namespace sr = std::ranges`, and `namespace sv = sr::views`

- `sv::enumerate`
- `sv::zip`
- `sv::zip_transform`
- `sv::adjacent`
- `sr::to`
- Formatting ranges

Ranges improvements in C++23

```
for (auto [i0, i1, i2]:
        sv::iota(0UL, 15UL) | sv::adjacent<3UL>) {
    std::print("{}, {}, {}\n", i0, i1, i2);
}
```

```
$ G -std=c++23 adjacent.cc
$ ./a.out
0, 1, 2
1, 2, 3
2, 3, 4
3, 4, 5
4, 5, 6
5, 6, 7
...
```

With the definitions
`namespace sr = std::ranges`,
and `namespace sv = sr::views`

- `sv::enumerate`
- `sv::zip`
- `sv::zip_transform`
- `sv::adjacent`
- `sr::to`
- Formatting ranges

Ranges improvements in C++23

```
1  auto R = sv::iota(0UL, 50UL)
2       | sv::transform([](auto i) { return 2. * pi * i; })
3       | sr::to<std::vector>();
```

With the definitions `namespace sr = std::ranges`, and `namespace sv = sr::views`

- `sv::enumerate`
- `sv::zip`
- `sv::zip_transform`
- `sv::adjacent`
- `sr::to`
- Formatting ranges

Ranges improvements in C++23

```cpp
auto main() -> int
{
    using namespace std::literals;
    namespace sr = std::ranges;
    namespace sv = sr::views;
    std::vector v{"One"s, "Two"s, "Three"s, "Four"s};
    std::print("{}\n", v);
    std::print("{}\n", v | sv::reverse);
    std::print("{}\n", sv::zip(v | sv::reverse, v));
    std::print("{}\n", sv::zip(sv::iota(1UL), v));
    std::print("{}\n", sv::enumerate(v));
} // rngout.cc
```

With the definitions `namespace sr = std::ranges`, and `namespace sv = sr::views`

- `sv::enumerate`
- `sv::zip`
- `sv::zip_transform`
- `sv::adjacent`
- `sr::to`
- Formatting ranges

```
["One", "Two", "Three", "Four"]
["Four", "Three", "Two", "One"]
[("Four", "One"), ("Three", "Two"), ("Two", "Three"), ("One", "Four")]
[(1, "One"), (2, "Two"), (3, "Three"), (4, "Four")]
[(0, "One"), (1, "Two"), (2, "Three"), (3, "Four")]
```

Ranges improvements in C++23

```cpp
auto main() -> int
{
    using namespace std::literals;
    std::vector v{"One"s, "Two"s, "Three"s, "Four"s};
    std::print("v = {}\n", v);
    std::print("{:n}\n", v);
    std::print("{:_>50n}\n", v);
    std::print("{:n:.^8}\n", v);
    std::vector name_ind{ std::pair("One"s, 1),
       std::pair("Two"s, 2), std::pair("Three"s, 3) };
    std::print("{:m}\n", name_ind);
    std::print("{}\n", name_ind);
} // rngout2.cc
```

With the definitions
`namespace sr = std::ranges`,
and `namespace sv = sr::views`

- `sv::enumerate`
- `sv::zip`
- `sv::zip_transform`
- `sv::adjacent`
- `sr::to`
- Formatting ranges

```
v = ["One", "Two", "Three", "Four"]
"One", "Two", "Three", "Four"
__________________"One", "Two", "Three", "Four"
..One..., ..Two..., .Three.., ..Four..
{"One": 1, "Two": 2, "Three": 3}
[("One", 1), ("Two", 2), ("Three", 3)]
```

Modules

C++20 modules

Traditionally, C++ projects are organised into header and source files. As an example, consider a simple `saxpy` program ...

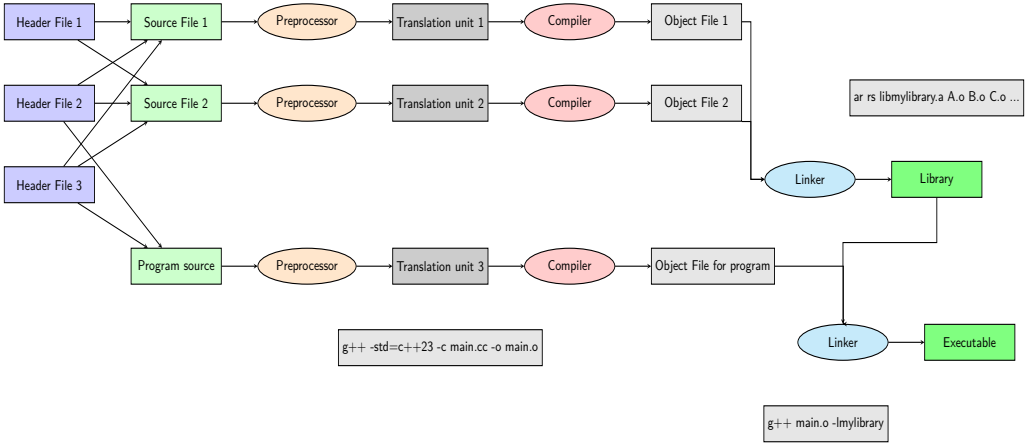
```
#ifndef SAXPY_HH
#define SAXPY_HH
#include <algorithm>
#include <span>
template <class T> concept Number = std::floating_point<T> or std::integral<T>;
template <class T> requires Number<T>
auto saxpy(T a, std::span<const T> x, std::span<const T> y, std::span<T> z){
    std::transform(x.begin(), x.end(), y.begin(), z.begin(),
        [a](T X, T Y) { return a * X + Y; });
}
#endif
```

```
#include "saxpy.hh"
auto main() -> int {
    //declarations
    saxpy(10., {inp1}, {inp2}, {outp});
}
```

Problems with header files

- Headers contain declarations of functions, classes etc., and definitions of inline functions.
- Source files contain implementations of other functions, such as `main`.
- Since function templates and class templates have to be visible to the compiler at the point of instantiation, these have traditionally lived in headers.
- Standard library, TBB, Thrust, Eigen ... a lot of important C++ libraries consist of a lot of template code, and therefore in header files.
- The `#include <abc>` mechanism is essentially a copy-and-paste solution. The preprocessor inserts the entire source of the headers in each source file that includes it, creating large translation units.
- The same template code gets re-parsed over and over for every new translation unit.
- If the headers contain expression templates, CRTP, metaprogramming repeated processing of the templates is a waste of resources.

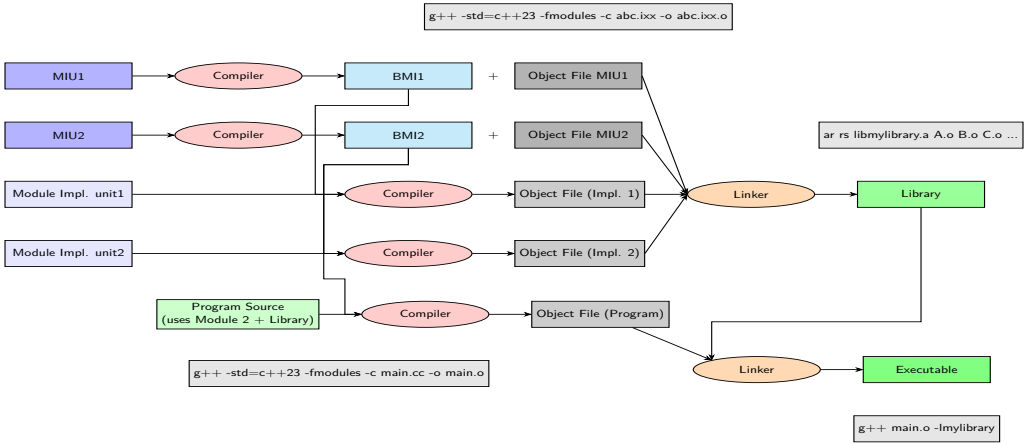
The compilation process



Modules

- The C++20 `module`s offer an alternative (better) organisation, in which all code, including template code, can now reside in source files
- Module sources are processed once to generate the so called compiled module interface (binary module interface, BMI) in addition to an object file
- The BMI caches syntactic information from all entities in the module
- Any source `import`ing the module immediately has access to the precompiled syntax tree in the BMI, leading to faster compilation
- Enforces ordered compilation: since a source file may export a module to be imported by another source file
- C++23 added the standard library as a module

Compilation flow using modules



JÜLICH Forschungszentrum

Using modules

```cpp
// examples/hello_m23.cc
import std;
auto main() -> int
{
  std::print("Hello, world!\n");
}
```

- In C++23, the standard library is also available as a module.
- **Note:** We have set up an alias `Gm='g++ -std=c++23 -fmodules'` in the course

```
$ Gm -c --compile-std-module
$ Gm hello_m23.cc
$ ./a.out
$
```

- The first step (since GCC 16) generates the BMI for the standard library and the second compiles the actual program
- For GCC 15, replace the first step with `Gm -c -fsearch-include-path bits/std.cc`

Using modules

```cpp
// examples/hello_m23.cc
import std;
auto main() -> int
{
  std::print("Hello, world!\n");
}
```

- Clang standard library is also avilable as a module, but usually it requires building using build system generators like CMake

```
$ export libcxx_module_src_dir="$(dirname $(find $(dirname $(which clang) )/..\
    -name std.cppm) 2>/dev/null)"
$ clang++ -std=c++23 -stdlib=libc++ -O3 -Wno-reserved-module-identifier \
    -fprebuilt-module-path=. --precompile $libcxx_module_src_dir/std.cppm\
    $libcxx_module_src_dir/std.compat.cppm
$ clang++ -std=c++23 -stdlib=libc++ -fmodules -fprebuilt-module-path=. \
    hello_m23.cc
$ ./a.out
```

- Not as elegant as the GCC oneliner, but it's possible to precompile the standard library module with clang
- It is necessary to explicitly specify the location of the module file when using it, either using `-fmodule-file=std=std.pcm -fmodule-file=std.compat=std.compat.pcm` or `-fprebuilt-module-path=.`

Using modules

Exercise 1.18:

Convert a few of the example programs you have seen during the course to use modules syntax instead. At the moment it means no more than replacing the `#include` lines with the `import` line for the standard library. The point is to get used to the extra compilation options you need with modules at the moment. Use, for instance, the date time library demo programs like `feb.cc` and `advent.cc`. In the next step, replace all the `import` lines using individual header units for standard library features by a single `import std;` line. Refer to the slides and compile using `g++`.

Creating a module (example)

```
class NullSimulator {
  bool config_ok{false}, run_ok{false};
public:
  void configure(std::string_view pars)
  {
    if (pars.empty())
      throw std::runtime_error{"..."};
    std::println("config {}", pars);
    config_ok = true;
  }
  void run()
  {
    if (not config_ok)
      throw std::runtime_error{"..."};
    std::println("running NullSimulator!");
    run_ok = true;
  }
  auto summary() const -> std::string
  {
    if (not run_ok)
      throw std::runtime_error{"..."};
    return { "And here are the results:" };
```

```
auto main() -> int {
  using namespace cxx_course;
  try {
    NullSimulator sim;
    sim.configure("Modules demo");
    sim.run();
    std::print("{}\n", sim.summary());
    return 0;
  } catch (std::exception& err) {
    std::print("{}\n", err.what());
    return 1;
  };
}
```

- A simple “do nothing” simulator class, mimicking the top level control flow in many applications
- We want to put our `NullSimulator` in a module and **`import`** it in `main` and use it as shown

JÜLICH Forschungszentrum

Creating a module (example)

- When using our module, like when using the standard library module, there are no changes in syntax
- We just have to import our module
- Notice that the module name, namespace name and the class name are all independent in C++
- `#include "myheader.hh"` searches for the header file and inserts it in place
- In contrast, the file name containing module code is not tied to the module name (which is why we had to explicitly state `-fmodule-file=std=std.pcm` when compiling with clang earlier)

```
// main.cc
import NullSim;
import std;
auto main() -> int {
  using namespace cxx_course;
  try {
    NullSimulator sim;
    sim.configure("Modules demo");
    sim.run();
    std::print("{}\n", sim.summary());
    return 0;
  } catch (std::exception& err) {
    std::print("{}\n", err.what());
    return 1;
  };
}
```

- In the module world, there are **no transitive imports**. Module `std` has to be imported independently wherever it is used

Creating a module (example)

```
// NullSimulator.cc
export module NullSim;
import std;
namespace cxx_course {
export class NullSimulator {
  bool config_ok{false}, run_ok{false};
public:
  void configure(std::string_view pars) {
    // code...
  }
  void run() {
    // code...
  }
  auto summary() const -> std::string {
    // code...
  }
};
void i_am_invisible() {
//...
}
}
```

- The class/function declarations and definitions can all be put in module source files
- The first non-comment line has to declare the module.
  - `export module XYZ;` : Module **interface** unit
  - `module XYZ;` : Module **implementation** unit
  - `module;` : Start of the "global module fragment"

Creating a module (example)

```
// NullSimulator.cc
export module NullSim;
import std;
namespace cxx_course {
export class NullSimulator {
  bool config_ok{false}, run_ok{false};
public:
  void configure(std::string_view pars) {
    // code...
  }
  void run() {
    // code...
  }
  auto summary() const -> std::string {
    // code...
  }
};
void i_am_invisible() {
//...
}
}
```

- Unlike declarations in a header file, those in a module file are not automatically visible if you import the module
- Only `exported` symbols are visible to code which imports a module
- `Unexported` declarations remain private

JÜLICH Forschungszentrum

Creating a module (example)

```
// NullSimulator.cc
export module NullSim;
import std;
namespace cxx_course {
export class NullSimulator {
  bool config_ok{false}, run_ok{false};
public:
  void configure(std::string_view pars) {
    // code...
  }
  void run() {
    // code...
  }
  auto summary() const -> std::string {
    // code...
  }
};
void i_am_invisible() {
//...
}
}
```

```
// main.cc
import NullSim;
import std;
auto main() -> int {
  using namespace cxx_course;
  try {
    NullSimulator sim;
    sim.configure("Modules demo");
    sim.run();
    std::print("{}\n", sim.summary());
  } catch (std::exception& err) {
    std::print("{}\n", err.what());
    return 1;
  };
}
```

```
Gm -c -fsearch-include-path bits/std.cc
Gm -c NullSimulator.cc
Gm -c main.cc
g++ main.o NullSimulator.o std.o -o nullsim
```


JÜLICH Forschungszentrum

Building the project using CMake

```
cmake_minimum_required(VERSION 3.30 FATAL_ERROR)
set (CMAKE_CXX_EXTENSIONS OFF)
set (CMAKE_CXX_STANDARD 23)
set (CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_EXPORT_COMPILE_COMMANDS ON)
if(CMAKE_VERSION VERSION_GREATER_EQUAL "4.4")
  set(CMAKE_EXPERIMENTAL_CXX_IMPORT_STD
    "f35a9ac6-8463-4d38-8eec-5d6008153e7d")
elseif(CMAKE_VERSION VERSION_GREATER_EQUAL "4.3")
  set(CMAKE_EXPERIMENTAL_CXX_IMPORT_STD
    "451f2fe2-a8a2-47c3-bc32-94786d8fc91b")
elseif(CMAKE_VERSION VERSION_GREATER_EQUAL "4.0.3")
  set(CMAKE_EXPERIMENTAL_CXX_IMPORT_STD
    "d0edc3af-4c50-42ea-a356-e2862fe7a444")
elseif(CMAKE_VERSION VERSION_GREATER_EQUAL "4.0.0")
  set(CMAKE_EXPERIMENTAL_CXX_IMPORT_STD
    "a9e1cf81-9932-4810-974b-6eccaf14e457")
elseif(CMAKE_VERSION VERSION_GREATER_EQUAL "3.30.0")
  set(CMAKE_EXPERIMENTAL_CXX_IMPORT_STD
    "0e5b6991-d74f-4b3d-a41c-cf096e0b2508")
endif()
project(nullsim LANGUAGES CXX)
set(CMAKE_CXX_MODULE_STD 1)
add_executable(nullsim main.cc)
target_sources(nullsim PUBLIC FILE_SET CXX_MODULES
    FILES NullSimulator.cc)
```

- Let CMake handle the necessary compiler options

```
export CMAKE_IMPORT_STD_MODULE_DIR=\
 /dir/of/EnableCxxImportStd.cmake

CXX=g++ cmake -B build -S . -GNinja
cmake --build build
```

- For clang, you would configure like this:

```
CXX=clang++ CXXFLAGS="-stdlib=libc++"\
   cmake -GNinja --build cuild
```

- The odd experimental import std guids are temporary as long as the feature is considered “experimental”

Building the project using CMake

```
1  cmake_minimum_required(VERSION 3.30 FATAL_ERROR)
2  set (CMAKE_CXX_EXTENSIONS OFF)
3  set (CMAKE_CXX_STANDARD 23)
4  set (CMAKE_CXX_STANDARD_REQUIRED ON)
5  set(CMAKE_EXPORT_COMPILE_COMMANDS ON)
6  if (NOT CMAKE_IMPORT_STD_MODULE_DIR)
7    set (CMAKE_IMPORT_STD_MODULE_DIR $ENV{CMAKE_IMPORT_STD_MODULE_DIR})
8  endif()
9  include($ENV{CMAKE_IMPORT_STD_MODULE_DIR}/EnableCxxImportStd.cmake)
10 project(nullsim LANGUAGES CXX)
11 set(CMAKE_CXX_MODULE_STD 1)
12 add_executable(nullsim main.cc)
13 target_sources(nullsim PUBLIC FILE_SET CXX_MODULES
14     FILES NullSimulator.cc)
```

- Let CMake handle the necessary compiler options

```
export CMAKE_IMPORT_STD_MODULE_DIR=\
 /dir/of/EnableCxxImportStd.cmake

CXX=g++ cmake -B build -S . -GNinja
cmake --build build
```

- For clang, you would configure like this:

```
CXX=clang++ CXXFLAGS="-stdlib=libc++"\
   cmake -GNinja --build cuild
```

- The odd experimental import std guids are temporary as long as the feature is considered “experimental”

Exercise 1.19:

The simple example with `NullSimulator` in the previous slides is in the folder `examples/modules/create0`. Build it using CMake with both Clang and GCC as compilers. Try building it directly using GCC as shown. The idea is to familiarise yourself with the modules based workflow. Insert a simple `I_am_invisible` function as shown in the slides. Try to use it in `main()`. What error do you get, even if we are importing the module containing it? Now, add the keyword **`export`** before that function, and test again!

Creating a module: implementation units

```
// NullSimulator.ixx
export module NullSim;
export class NullSimulator {
public:
    void configure(std::string_view pars);
    void run();
    auto summary() const -> std::string;
};
// NullSimulator.cc
module NullSim;
import std;
void NullSimulator::configure(std::string_...) {
    // actual implementation
}
void NullSimulator::run() {
    // code...
}
auto NullSimulator::summary() const
    -> std::string {
    // code...
}
```

- Separating implementation is not syntactically necessary, but may sometimes be desirable to offer a clearer overview of the interface without the implementation code
- Only the interface unit shall export the symbols and the module itself, not the implementation unit
- The implementation units are bound to the module by the module lines at the start

Creating a module: implementation units

```
// NullSimulator.ixx
export module NullSim;
export class NullSimulator {
public:
    void configure(std::string_view pars);
    void run();
    auto summary() const -> std::string;
};
// NullSimulator.cc
module NullSim;
import std;
void NullSimulator::configure(std::string_...) {
    // actual implementation
}
void NullSimulator::run() {
    // code...
}
auto NullSimulator::summary() const
    -> std::string {
    // code...
}
```

```
Gm -c -fsearch-include-path bits/std.cc
Gm -c NullSimulator.ixx
Gm -c NullSimulator.cc
Gm -c main.cc
g++ main.o NullSimulator.o std.o -o nullsim
```

Creating a module: implementation units

```
// NullSimulator.ixx
export module NullSim;
export class NullSimulator {
public:
    void configure(std::string_view pars);
    void run();
    auto summary() const -> std::string;
};
// NullSimulator.cc
module NullSim;
import std;
void NullSimulator::configure(std::string_...) {
    // actual implementation
}
void NullSimulator::run() {
    // code...
}
auto NullSimulator::summary() const
    -> std::string {
    // code...
}
```

```
Gm -c -fsearch-include-path bits/std.cc
Gm -c NullSimulator.ixx
Gm -c NullSimulator.cc
Gm -c main.cc
g++ main.o NullSimulator.o std.o -o nullsim
```

Works, but masks a big problem! Can you see it?

Creating a module: implementation units

```
Gm -c -fsearch-include-path bits/std.cc
Gm -c NullSimulator.ixx
Gm -c NullSimulator.cc
Gm -c main.cc
g++ main.o NullSimulator.o std.o -o nullsim
```

- The third line overwrites one of the outputs `NullSimulator.o` from the second line
- When we compile the interface unit, the by-products are the BMI and an object file.
  - The BMI contains information about the interface and the pre-compiled inline functions, class and function templates.
  - Since implementation units are not mandatory, the interface units can contain ordinary function definitions, which will result in binary code stored in the object file
- Compiling the implementation unit is like compiling any other C++ file, and results in an object file for the functions defined there.
- Our example works because we separated all implementation code into the implementation unit `NullSimulator.cc`. What if we left one function body in `NullSimulator.ixx`?

Exercise 1.20:

The example directory `modules/create1` contains the code where we have split the `NullSimulator` code between the module interface and implementation units. Compile it with GCC as shown in the previous slides. What happens if you delete the `NullSimulator.o` output after the second step and go through with the rest? Then in `modules/create1b`, we have the same code, but, we have inserted a new free function `check_results`. It will lead to a compiler error in step 2, which you should be able to solve. Does the rest of the build process go through without issues?

Non-inline functions in interface units

- ... end up in the object file when the interface unit is compiled. If we are using those functions, we need that object file when linking
- $\Longrightarrow$ Possible conflict in output filename when compiling the implementation unit
- Common solutions:
  - When compiling with option `-c`, use an explicit output filename, e.g.,

```
Gm -c NullSimulator.ixx -o NullSimulator.ixx.o
Gm -c NullSimulator.cc -o NullSimulator.cc.o
Gm -c main.cc -o main.cc.o
g++ main.cc.o NullSimulator.cc.o NullSimulator.ixx.o std.o
```

  - Abandon suffix based differentiation, and name interface and implementation units differently, e.g., `NullSimulator.cc` and `NullSimulator_impl.cc`.

```
Gm -c NullSimulator.cc
Gm -c NullSimulator_impl.cc
Gm -c main.cc
g++ main.o NullSimulator.o NullSimulator_impl.o std.o
```

Non-inline functions in interface units

- How do the functions defined inside the class declarations behave?
- Answer: as of C++23, like regular non-inline functions!

```
// NullSimulator.ixx
export module NullSim;
import std;
namespace cxx_course {
export class NullSimulator {
    bool config_ok{false}, run_ok{false};
public:
    void configure(std::string_view pars);
    void run();
    auto summary() const -> std::string
    {
        return {"Summary of results:"};
    }
};
}
```

- The function `summary` is compiled and placed in the object code when compiling `NullSimulator.ixx`

```
// NullSimulator.cc
module NullSim;
void NullSimulator::configure(...)
{
    ...
}
void NullSimulator::run()
{
    ...
}
```

- The functions `configure()` and `run()` end up in the object file when `NullSimulator.cc` is compiled.
- Both generated object files are required for linking!

Building process including modules

- Object files from the interface and implementation units can be combined into static/shared libraries.
- The interface units also produce the BMI
- To compile a module user, we need the BMI of all modules it uses
- To link the application, we need the object files or libraries made of them.

Splitting the module interface

- Large module interface units may be split into multiple files
  - Example: different large classes in their own files
  - Reason: maintenance, collaborative development
  - How: module partitions
- Each of the 3 module interface units shown here belong to the same module `Measurements`. Observe how they are named and exported
- Implementation units belong to the module as a whole, not to any partitions

```
// Measurements.ixx
export module Measurements;
export import :RMSD;
export import :Rg;
export import :ContactOrder;
// RMSD.ixx
export module Measurements:RMSD;
export class RMSD {/*...*/};
// Rg.ixx
export module Measurements:Rg;
export class Rg {/*...*/};
// ContactOrder.ixx
export module Measurements:ContactOrder;
export class ContactOrder {/*...*/};
```

- Partitions can be imported by other partitions
- Partitions can not be directly imported outside the module

Splitting the module interface

- Large module interface units may be split into multiple files
  - Example: different large classes in their own files
  - Reason: maintenance, collaborative development
  - How: module partitions
- Each of the 3 module interface units shown here belong to the same module `Measurements`. Observe how they are named and exported
- Implementation units belong to the module as a whole, not to any partitions

```
// Measurements.ixx
export module Measurements;
export import :RMSD;
export import :Rg;
export import :ContactOrder;
// RMSD.ixx
export module Measurements:RMSD;
export class RMSD {/*...*/};
// Rg.ixx
export module Measurements:Rg;
export class Rg {/*...*/};
// ContactOrder.ixx
export module Measurements:ContactOrder;
export class ContactOrder {/*...*/};
```

- The interface exported by the module partitions can be exported by the primary module interface

Organisation of interface partitions vs headers

```cpp
// M.hh
constexpr auto R = 42;
```

```cpp
// A.hh
struct cA {
    auto func() const -> int;
};
```

```cpp
// B.hh
struct cB {
    auto func() const -> int;
};
```

- Quite common to have classes in their own headers
- Must include the header for a class when implementing member functions in separate source files

```cpp
// A.cc
#include "A.hh"
auto cA::func() const -> int {
        return 42;
}
```

```cpp
// B.cc
#include "B.hh"
auto cB::func() const -> int {
        return 43;
}
```

```cpp
// main.cc
#include "A.hh"
#include "B.hh"
#include "M.hh"
auto main() -> int {
    cA a;
    cB b;
    return R + a.func() + b.func();
}
```

Organisation of interface partitions vs headers

```
// M.ixx
export module M;
export constexpr auto R = 42;
export struct cA {
    auto func() const -> int;
};
export struct cB {
    auto func() const -> int;
};
```

- Large module interface units are not considered bad
- Names exported in the primary module interface don't need to be imported inside the module's implementation units

```
// A.cc
module M;
auto cA::func() const -> int {
        return 42;
}
```

```
// B.cc
module M;
auto cB::func() const -> int {
        return 43;
}
```

```
// main.cc
import M;
auto main() -> int {
    cA a;
    cB b;
    return a.func() + b.func();
}
```


JÜLICH Forschungszentrum

Organisation of interface partitions vs headers

```
// M.ixx
export module M;
export constexpr auto R = 42;
export import :A;
export import :B;
```

```
// A.ixx
export module M:A;
export struct cA {
    auto func() const -> int;
};
```

```
// B.ixx
export module M:B;
export struct cB {
    auto func() const -> int;
};
```

- In case the primary interface is split into partitions, the partitions need to be re-exported in it, in order that the declarations in them be visible in implementation units

```
// A.cc
module M;
auto cA::func() const -> int {
        return 42;
}
```

```
// B.cc
module M;
auto cB::func() const -> int {
        return 43;
}
```

```
// main.cc
import M;
auto main() -> int {
    cA a;
    cB b;
    return R + a.func() + b.func();
}
```


JÜLICH Forschungszentrum

Organisation of interface partitions vs headers

```cpp
// M.ixx
export module M;
export constexpr auto R = 42;
export import :A;
export import :B;
```

```cpp
// A.ixx
export module M:A;
export struct cA {
    auto func() const -> int;
};
```

```cpp
// B.ixx
export module M:B;
export struct cB {
    auto func() const -> int;
};
```

- Module exporters and importers must be compiled in a specific order, not always trivially inferred

```cpp
// A.cc
module M;
auto cA::func() const -> int {
        return 42;
}
```

```cpp
// B.cc
module M;
auto cB::func() const -> int {
        return 43;
}
```

```cpp
// main.cc
import M;
auto main() -> int {
    cA a;
    cB b;
    return R + a.func() + b.func();
}
```

Letting the build system generators handle modules

- CMake and other build system generators simplify the management of modules based compilation
- A set of module interface units can be attached to an executable or library target in CMake with the `target_sources` function
- CMake determines the order in which they must be compiled, and sets where the BMI and object files are stored
- The library or executable is then linked using the relevant set of object files
- CMake places the BMI files at the right places so that it can find them while compiling the rest of the project
- The BMI must be generated from the module interface units fresh using the build flags.

Using header files inside module units

- Even if you want to use modules for your project, you might need external dependencies which don't yet support a modules based build
- Including headers is possible, but restricted to a specific segment in a module unit, called the **global module fragment**
- The global module fragment, if present, has to be the first section in a module unit.

```
module;
#include <Eigen/Dense>
#include <boost/type_index.hpp>
module Measurements;

import ...;
```

Using header files inside module units

- Even if you want to use modules for your project, you might need external dependencies which don't yet support a modules based build
- Including headers is possible, but restricted to a specific segment in a module unit, called the **global module fragment**
- The global module fragment, if present, has to be the first section in a module unit.

```
module;
#include <Eigen/Dense>
#include <boost/type_index.hpp>
module Measurements;

import ...;
```

- It starts like an empty or nameless module declaration (`module;`). It ends at the actual module declaration (with `export` for interface and without for implementation)

As of October 2025, mixed mode projects when using the standard library as a module, but also including it directly or indirectly through dependencies, lead to errors with GCC 15! GCC 16 patches a lot of the errors for mixed mode compilation.

Converting older header based projects

```cpp
// saxpy.hh
#ifndef SAXPY_HH
#define SAXPY_HH
#include <algorithm>
#include <span>

template <class T>
concept Number = std::floating_point<T>
              or std::integral<T>;
template <Number T>
auto saxpy(T a, std::span<const T> x,
           std::span<const T> y,
           std::span<T> z)
{
    std::transform(x.begin(), x.end(),
                   y.begin(), z.begin(),
        [a](T X, T Y) {
            return a * X + Y;
        });
}
#endif
```

- A header file contains a function template `saxpy`, and a `concept` necessary to define that function
- A source file, `main.cc` which includes the header and uses the function

JÜLICH Forschungszentrum

Converting older header based projects

```cpp
// usesaxpy.cc
#include <iostream>
#include <array>
#include <vector>
#include <span>
#include "saxpy.hh"

auto main() -> int
{
    using namespace std;
    const array inp1 { 1., 2., 3., 4., 5. };
    const array inp2 { 9., 8., 7., 6., 5. };
    vector outp(inp1.size(), 0.);

    saxpy(10., {inp1}, {inp2}, {outp});
    for (auto x : outp) cout << x << "\n";
    cout << ":::::::::::::::::::::\n";
}
```

- A header file contains a function template `saxpy`, and a `concept` necessary to define that function
- A source file, `main.cc` which includes the header and uses the function

Converting older header based projects

Make a module interface unit

```
// saxpy.hh -> saxpy.ixx
#ifndef SAXPY_HH
#define SAXPY_HH
#include <algorithm>
#include <span>

template <class T>
concept Number = std::floating_point<T>
              or std::integral<T>;
template <Number T>
auto saxpy(T a, std::span<const T> x,
           std::span<const T> y,
           std::span<T> z)
{
    std::transform(x.begin(), x.end(),
                   y.begin(), z.begin(),
        [a](T X, T Y) {
            return a * X + Y;
        });
}
#endif
```


JÜLICH Forschungszentrum

Converting older header based projects

```
// saxpy.hh -> saxpy.ixx
#ifndef SAXPY_HH
#define SAXPY_HH
#include <algorithm>
#include <span>

template <class T>
concept Number = std::floating_point<T>
              or std::integral<T>;
template <Number T>
auto saxpy(T a, std::span<const T> x,
           std::span<const T> y,
           std::span<T> z)
{
    std::transform(x.begin(), x.end(),
                   y.begin(), z.begin(),
        [a](T X, T Y) {
            return a * X + Y;
        });
}
#endif
```

Make a module interface unit

- Include guards are no longer required, since importing a module does not transitively import things used inside the module

JÜLICH Forschungszentrum

Converting older header based projects

Make a module interface unit

```
// saxpy.hh -> saxpy.ixx


#include <algorithm>
#include <span>

template <class T>
concept Number = std::floating_point<T>
              or std::integral<T>;
template <Number T>
auto saxpy(T a, std::span<const T> x,
           std::span<const T> y,
           std::span<T> z)
{
    std::transform(x.begin(), x.end(),
                   y.begin(), z.begin(),
        [a](T X, T Y) {
            return a * X + Y;
        });
}
```


JÜLICH Forschungszentrum

Converting older header based projects

```
// saxpy.hh -> saxpy.ixx
module;
#include <algorithm>
#include <span>
export module saxpy;

template <class T>
concept Number = std::floating_point<T>
              or std::integral<T>;
template <Number T>
auto saxpy(T a, std::span<const T> x,
           std::span<const T> y,
           std::span<T> z)
{
    std::transform(x.begin(), x.end(),
                   y.begin(), z.begin(),
        [a](T X, T Y) {
            return a * X + Y;
        });
}
```

Make a module interface unit

- Start a global module fragment to enclose the headers you have to use.
- Export the module.

Converting older header based projects

```
// saxpy.hh -> saxpy.ixx

export module saxpy;
import std;

template <class T>
concept Number = std::floating_point<T>
              or std::integral<T>;
template <Number T>
auto saxpy(T a, std::span<const T> x,
           std::span<const T> y,
           std::span<T> z)
{
    std::transform(x.begin(), x.end(),
                   y.begin(), z.begin(),
        [a](T X, T Y) {
            return a * X + Y;
        });
}
```

Make a module interface unit

- Start a global module fragment to enclose the headers you have to use.
- Export the module.
- If you can get by with only **import**s, replace *#include* lines with corresponding **import** lines. Omit the global module fragment in this case

Converting older header based projects

```
// saxpy.hh -> saxpy.ixx

export module saxpy;
import std;

template <class T>
concept Number = std::floating_point<T>
              or std::integral<T>;
export template <Number T>
auto saxpy(T a, std::span<const T> x,
           std::span<const T> y,
           std::span<T> z)
{
    std::transform(x.begin(), x.end(),
                   y.begin(), z.begin(),
        [a](T X, T Y) {
            return a * X + Y;
        });
}
```

Make a module interface unit

- Start a global module fragment to enclose the headers you have to use.
- Export the module.
- If you can get by with only **import**s, replace *#include* lines with corresponding **import** lines. Omit the global module fragment in this case
- Explicitly export any definitions (classes, functions...) you want for users of the module. Anything not exported by a module is automatically private to the module

Using the module

Use your module

```
// usesaxpy.cc
#include <iostream>
#include <array>
#include <vector>
#include <span>
#include "saxpy.hh"

auto main() -> int
{
    using namespace std;
    const array inp1 { 1., 2., 3., 4., 5. };
    const array inp2 { 9., 8., 7., 6., 5. };
    vector outp(inp1.size(), 0.);

    saxpy(10., {inp1}, {inp2}, {outp});
    for (auto x : outp) cout << x << "\n";
    cout << ":::::::::::::::::::::\n";
}
```

Using the module

```
// usesaxpy.cc
import std;
#include "saxpy.hh"

auto main() -> int
{
    using namespace std;
    const array inp1 { 1., 2., 3., 4., 5. };
    const array inp2 { 9., 8., 7., 6., 5. };
    vector outp(inp1.size(), 0.);

    saxpy(10., {inp1}, {inp2}, {outp});
    for (auto x : outp) cout << x << "\n";
    cout << ":::::::::::::::::::::\n";
}
```

Use your module

- Optionally replace `#include` lines with corresponding `import` line(s).
- If the source is not a module unit, include directives may be used alongside imports

Using the module

```
// usesaxpy.cc
import std;
import saxpy;

auto main() -> int
{
    using namespace std;
    const array inp1 { 1., 2., 3., 4., 5. };
    const array inp2 { 9., 8., 7., 6., 5. };
    vector outp(inp1.size(), 0.);

    saxpy(10., {inp1}, {inp2}, {outp});
    for (auto x : outp) cout << x << "\n";
    cout << ":::::::::::::::::::::\n";
}
```

Use your module

- Optionally replace `#include` lines with corresponding `import` line(s).
- If the source is not a module unit, include directives may be used alongside imports
- Import your module by name

Using the module

```
// usesaxpy.cc
import std;
import saxpy;

auto main() -> int
{
    using namespace std;
    const array inp1 { 1., 2., 3., 4., 5. };
    const array inp2 { 9., 8., 7., 6., 5. };
    vector outp(inp1.size(), 0.);

    saxpy(10., {inp1}, {inp2}, {outp});
    for (auto x : outp) cout << x << "\n";
    cout << ":::::::::::::::::::::\n";
}
```

Use your module

- Optionally replace `#include` lines with corresponding `import` line(s).
- If the source is not a module unit, include directives may be used alongside imports
- Import your module by name
- Importing `saxpy` here, only grants us access to the explicitly exported function `saxpy`. Not other functions, classes, concepts etc. defined in the module `saxpy`, not any other module imported in the module interface unit.

Setup building with CMake

```
cmake_minimum_required(VERSION 3.30)
set (CMAKE_CXX_EXTENSIONS OFF)
set (CMAKE_CXX_STANDARD 23)
set (CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_EXPORT_COMPILE_COMMANDS ON)
include(\$ENV{CMAKE_IMPORT_STD_MODULE_DIR}/
EnableCxxImportStd.cmake)

project(use_saxpy-example LANGUAGES CXX)
set(CMAKE_CXX_MODULE_STD 1)
add_executable(use_saxpy usesaxpy.cc)
target_sources(use_saxpy
  PUBLIC
    FILE_SET CXX_MODULES
    FILES saxpy.ixx
)
```

- `CMake` supports C++ modules since the version 3.28
- Since version 3.30 it supports creation and use of standard library as a module if the compiler + standard library combination supports it
- This means Clang >= 18.1 or GCC >= 15.1. (prefer GCC >= 16.1)
- The `Ninja` generator is required
- Massive simplification of the build process!

```
CMAKE_IMPORT_STD_MODULE_DIR=\
/path/EnableCxxImportStd.cmake\
cmake -GNinja -B build0 \
    -S source_path
cmake --build0 -- -j 4
```

Exercise 1.21:

Versions of the `saxpy` program written using header files and then modules can be found in the `examples/saxpy/`. Familiarise yourself with the process of building applications with modules. Experiment by writing a new inline function in the module interface file without exporting it. Try to call that function from `main`. Check again after exporting it in the module.

Exercise 1.22:

As a more complicated example, we have in `examples/2_any` the second version of our container with polymorphic geometrical objects. The header and source files for each class `Point`, `Circle` etc have been rewritten for modules. Compare the two versions, build them, run them.

Modules: Summary

- Status, since around September 2024: Usable!
- Build with CMake with GCC and Clang.
- A different organisation of multi-file projects than the one with header and source files
- Promise:
  - easier control over symbol visibility
  - no “copy and paste” solution of headers
  - smaller translation units and hence faster compilation
  - no transitive imports, no import of MACROs defined in imported modules
- Does not change anything about functions, classes, templates or concepts, just how we place them in files
- Module interface units play a similar role to headers, but without their problems

C++ linkage and how modules affect them

```
namespace app {
    int variable{};
}
void A()
{
    std::printf("%d\n",
      app::variable++);
}
```

C++ linkage and how modules affect them

```
namespace app {
    int variable{};
}
void A()
{
    std::printf("%d\n",
      app::variable++);
}
```

```
g++ -S -O3 A.cc
```


JÜLICH Forschungszentrum

C++ linkage and how modules affect them

```cpp
namespace app {
    int variable{};
}
void A()
{
    std::printf("%d\n",
      app::variable++);
}
```

```
g++ -S -O3 A.cc
```

```
A():
    movl    app::variable(%rip), %esi
    movl    $.LC0, %edi
    leal    1(%rsi), %eax
    movl    %eax, app::variable(%rip)
    xorl    %eax, %eax
    jmp     printf
app::variable:
        .zero   4
```


JÜLICH Forschungszentrum

C++ linkage and how modules affect them

```cpp
namespace app {
    int variable{};
}
void A()
{
    std::printf("%d\n",
      app::variable++);
}
```

```
g++ -S -O3 A.cc
```

```cpp
namespace app {
    int variable{};
}
void B()
{
    std::printf("%d\n",
      app::variable++);
}
```

```
A():
    movl    app::variable(%rip), %esi
    movl    $.LC0, %edi
    leal    1(%rsi), %eax
    movl    %eax, app::variable(%rip)
    xorl    %eax, %eax
    jmp     printf
app::variable:
        .zero   4
```


JÜLICH Forschungszentrum

C++ linkage and how modules affect them

```cpp
namespace app {
    int variable{};
}
void A()
{
    std::printf("%d\n",
      app::variable++);
}
```

```
g++ -S -O3 A.cc
```

```cpp
namespace app {
    int variable{};
}
void B()
{
    std::printf("%d\n",
      app::variable++);
}
```

```
g++ -S -O3 B.cc
```

```
A():
    movl    app::variable(%rip), %esi
    movl    $.LC0, %edi
    leal    1(%rsi), %eax
    movl    %eax, app::variable(%rip)
    xorl    %eax, %eax
    jmp     printf
app::variable:
        .zero   4
```


JÜLICH Forschungszentrum

C++ linkage and how modules affect them

```cpp
namespace app {
    int variable{};
}
void A()
{
    std::printf("%d\n",
      app::variable++);
}
```

```
g++ -S -O3 A.cc
```

```cpp
namespace app {
    int variable{};
}
void B()
{
    std::printf("%d\n",
      app::variable++);
}
```

```
g++ -S -O3 B.cc
```

```
A():
    movl    app::variable(%rip), %esi
    movl    $.LC0, %edi
    leal    1(%rsi), %eax
    movl    %eax, app::variable(%rip)
    xorl    %eax, %eax
    jmp     printf
app::variable:
        .zero   4
```

```
B():
    movl    app::variable(%rip), %esi
    movl    $.LC0, %edi
    leal    1(%rsi), %eax
    movl    %eax, app::variable(%rip)
    xorl    %eax, %eax
    jmp     printf
app::variable:
        .zero   4
```

C++ linkage and how modules affect them

```cpp
namespace app {
    int variable{};
}
void A()
{
    std::printf("%d\n",
      app::variable++);
}
```

```
g++ -S -O3 A.cc
```

```cpp
namespace app {
    int variable{};
}
void B()
{
    std::printf("%d\n",
      app::variable++);
}
```

```
g++ -S -O3 B.cc
```

```cpp
auto main() -> int
{
    A();
    B();
}
```

```
A():
    movl    app::variable(%rip), %esi
    movl    $.LC0, %edi
    leal    1(%rsi), %eax
    movl    %eax, app::variable(%rip)
    xorl    %eax, %eax
    jmp     printf
app::variable:
        .zero   4
```

```
B():
    movl    app::variable(%rip), %esi
    movl    $.LC0, %edi
    leal    1(%rsi), %eax
    movl    %eax, app::variable(%rip)
    xorl    %eax, %eax
    jmp     printf
app::variable:
        .zero   4
```


JÜLICH Forschungszentrum

C++ linkage and how modules affect them

```cpp
namespace app {
    int variable{};
}
void A()
{
    std::printf("%d\n",
      app::variable++);
}
```

```
g++ -S -O3 A.cc
```

```cpp
namespace app {
    int variable{};
}
void B()
{
    std::printf("%d\n",
      app::variable++);
}
```

```
g++ -S -O3 B.cc
```

```cpp
auto main() -> int
{
    A();
    B();
}
```

```
g++ -S -O3 main.cc
```

```
A():
    movl    app::variable(%rip), %esi
    movl    $.LC0, %edi
    leal    1(%rsi), %eax
    movl    %eax, app::variable(%rip)
    xorl    %eax, %eax
    jmp     printf
app::variable:
        .zero   4
```

```
B():
    movl    app::variable(%rip), %esi
    movl    $.LC0, %edi
    leal    1(%rsi), %eax
    movl    %eax, app::variable(%rip)
    xorl    %eax, %eax
    jmp     printf
app::variable:
        .zero   4
```


JÜLICH Forschungszentrum

C++ linkage and how modules affect them

```cpp
namespace app {
    int variable{};
}
void A()
{
    std::printf("%d\n",
      app::variable++);
}
```

```
g++ -S -O3 A.cc
```

```cpp
namespace app {
    int variable{};
}
void B()
{
    std::printf("%d\n",
      app::variable++);
}
```

```
g++ -S -O3 B.cc
```

```cpp
auto main() -> int
{
    A();
    B();
}
```

```
g++ -S -O3 main.cc
```

```
A():
    movl    app::variable(%rip), %esi
    movl    $.LC0, %edi
    leal    1(%rsi), %eax
    movl    %eax, app::variable(%rip)
    xorl    %eax, %eax
    jmp     printf
app::variable:
        .zero   4
```

```
B():
    movl    app::variable(%rip), %esi
    movl    $.LC0, %edi
    leal    1(%rsi), %eax
    movl    %eax, app::variable(%rip)
    xorl    %eax, %eax
    jmp     printf
app::variable:
        .zero   4
```

```
main:
    call    A()
    call    B()
    xorl    %eax, %eax
    addq    $8, %rsp
    ret$
```


JÜLICH Forschungszentrum

C++ linkage and how modules affect them

```cpp
namespace app {
    int variable{};
}
void A()
{
    std::printf("%d\n",
      app::variable++);
}
```

```
g++ -S -O3 A.cc
```

```cpp
namespace app {
    int variable{};
}
void B()
{
    std::printf("%d\n",
      app::variable++);
}
```

```
g++ -S -O3 B.cc
```

```cpp
auto main() -> int
{
    A();
    B();
}
```

```
g++ -S -O3 main.cc
```

```
A():
    movl    app::variable(%rip), %esi
    movl    $.LC0, %edi
    leal    1(%rsi), %eax
    movl    %eax, app::variable(%rip)
    xorl    %eax, %eax
    jmp     printf
app::variable:
        .zero   4
```

```
B():
    movl    app::variable(%rip), %esi
    movl    $.LC0, %edi
    leal    1(%rsi), %eax
    movl    %eax, app::variable(%rip)
    xorl    %eax, %eax
    jmp     printf
app::variable:
        .zero   4
```

```
main:
    call    A()
    call    B()
    xorl    %eax, %eax
    addq    $8, %rsp
    ret$
```

Link

JÜLICH Forschungszentrum

C++ linkage and how modules affect them

```cpp
namespace app {
    int variable{};
}
void A()
{
    std::printf("%d\n",
      app::variable++);
}
```

```
g++ -S -O3 A.cc
```

```cpp
namespace app {
    int variable{};
}
void B()
{
    std::printf("%d\n",
      app::variable++);
}
```

```
g++ -S -O3 B.cc
```

```cpp
auto main() -> int
{
    A();
    B();
}
```

```
g++ -S -O3 main.cc
```

```
A():
    movl    app::variable(%rip), %esi
    movl    $.LC0, %edi
    leal    1(%rsi), %eax
    movl    %eax, app::variable(%rip)
    xorl    %eax, %eax
    jmp     printf
app::variable:
        .zero   4
```

```
B():
    movl    app::variable(%rip), %esi
    movl    $.LC0, %edi
    leal    1(%rsi), %eax
    movl    %eax, app::variable(%rip)
    xorl    %eax, %eax
    jmp     printf
app::variable:
        .zero   4
```

```
main:
    call    A()
    call    B()
    xorl    %eax, %eax
    addq    $8, %rsp
    ret$
```

Link

- Have to produce one sequence of instructions.

JÜLICH Forschungszentrum

C++ linkage and how modules affect them

```cpp
namespace app {
    int variable{};
}
void A()
{
    std::printf("%d\n",
      app::variable++);
}
```

```
g++ -S -O3 A.cc
```

```cpp
namespace app {
    int variable{};
}
void B()
{
    std::printf("%d\n",
      app::variable++);
}
```

```
g++ -S -O3 B.cc
```

```cpp
auto main() -> int
{
    A();
    B();
}
```

```
g++ -S -O3 main.cc
```

```
A():
    movl    app::variable(%rip), %esi
    movl    $.LC0, %edi
    leal    1(%rsi), %eax
    movl    %eax, app::variable(%rip)
    xorl    %eax, %eax
    jmp     printf
app::variable:
        .zero   4
```

```
B():
    movl    app::variable(%rip), %esi
    movl    $.LC0, %edi
    leal    1(%rsi), %eax
    movl    %eax, app::variable(%rip)
    xorl    %eax, %eax
    jmp     printf
app::variable:
        .zero   4
```

```
main:
    call    A()
    call    B()
    xorl    %eax, %eax
    addq    $8, %rsp
    ret$
```

Link

- Have to produce one sequence of instructions.
- But there are two versions of `app::variable`, one from the compilation of `A.cc`, one from that of `B.cc`

JÜLICH Forschungszentrum

C++ linkage and how modules affect them

```cpp
namespace app {
    int variable{};
}
void A()
{
    std::printf("%d\n",
      app::variable++);
}
```

```
g++ -S -O3 A.cc
```

```cpp
namespace app {
    int variable{};
}
void B()
{
    std::printf("%d\n",
      app::variable++);
}
```

```
g++ -S -O3 B.cc
```

```cpp
auto main() -> int
{
    A();
    B();
}
```

```
g++ -S -O3 main.cc
```

```
A():
    movl    app::variable(%rip), %esi
    movl    $.LC0, %edi
    leal    1(%rsi), %eax
    movl    %eax, app::variable(%rip)
    xorl    %eax, %eax
    jmp     printf
app::variable:
        .zero   4
```

```
B():
    movl    app::variable(%rip), %esi
    movl    $.LC0, %edi
    leal    1(%rsi), %eax
    movl    %eax, app::variable(%rip)
    xorl    %eax, %eax
    jmp     printf
app::variable:
        .zero   4
```

```
main:
    call    A()
    call    B()
    xorl    %eax, %eax
    addq    $8, %rsp
    ret$
```

Link

- Have to produce one sequence of instructions.
- But there are two versions of `app::variable`, one from the compilation of `A.cc`, one from that of `B.cc`
- Are they to be treated as the same entity or different ones?

JÜLICH Forschungszentrum

Linkage: determining who is who in multi-file projects

```
// var.hh
namespace app {
    int variable{};
}
```


JÜLICH Forschungszentrum

Linkage: determining who is who in multi-file projects

```
// var.hh
namespace app {
    int variable{};
}
```

```
#include "var.hh"
void A()
{
    std::printf("%d\n",
      app::variable++);
}
```

```
#include "var.hh"
void B()
{
    std::printf("%d\n",
      app::variable++);
}
```


JÜLICH Forschungszentrum

Linkage: determining who is who in multi-file projects

```
// var.hh
namespace app {
    int variable{};
}
```

```
#include "var.hh"
void A()
{
    std::printf("%d\n",
      app::variable++);
}
```

```
#include "var.hh"
void B()
{
    std::printf("%d\n",
      app::variable++);
}
```

```
A():
    movl    app::variable(%rip), %esi
    movl    $.LC0, %edi
    leal    1(%rsi), %eax
    movl    %eax, app::variable(%rip)
    xorl    %eax, %eax
    jmp     printf
app::variable:
        .zero   4
```

```
B():
    movl    app::variable(%rip), %esi
    movl    $.LC0, %edi
    leal    1(%rsi), %eax
    movl    %eax, app::variable(%rip)
    xorl    %eax, %eax
    jmp     printf
app::variable:
        .zero   4
```


JÜLICH Forschungszentrum

Linkage: determining who is who in multi-file projects

```
// var.hh
namespace app {
    int variable{};
}
```

```
#include "var.hh"
void A()
{
    std::printf("%d\n",
      app::variable++);
}
```

```
#include "var.hh"
void B()
{
    std::printf("%d\n",
      app::variable++);
}
```

```
A():
    movl    app::variable(%rip), %esi
    movl    $.LC0, %edi
    leal    1(%rsi), %eax
    movl    %eax, app::variable(%rip)
    xorl    %eax, %eax
    jmp     printf
app::variable:
        .zero   4
```

```
B():
    movl    app::variable(%rip), %esi
    movl    $.LC0, %edi
    leal    1(%rsi), %eax
    movl    %eax, app::variable(%rip)
    xorl    %eax, %eax
    jmp     printf
app::variable:
        .zero   4
```

- Putting it in a header does not fix anything!

Linkage: determining who is who in multi-file projects

```
// var.hh
namespace app {
    int variable{};
}
```

```
#include "var.hh"
void A()
{
    std::printf("%d\n",
      app::variable++);
}
```

```
#include "var.hh"
void B()
{
    std::printf("%d\n",
      app::variable++);
}
```

```
A():
    movl    app::variable(%rip), %esi
    movl    $.LC0, %edi
    leal    1(%rsi), %eax
    movl    %eax, app::variable(%rip)
    xorl    %eax, %eax
    jmp     printf
app::variable:
        .zero   4
```

```
B():
    movl    app::variable(%rip), %esi
    movl    $.LC0, %edi
    leal    1(%rsi), %eax
    movl    %eax, app::variable(%rip)
    xorl    %eax, %eax
    jmp     printf
app::variable:
        .zero   4
```

- Putting it in a header does not fix anything!
- There are 4 types of "linkage" defined in C++, to address the issue of sameness of symbols across multiple translation units.

Linkage: determining who is who in multi-file projects

```
// var.hh
namespace app {
    int variable{};
}
```

```
#include "var.hh"
void A()
{
    std::printf("%d\n",
      app::variable++);
}
```

```
#include "var.hh"
void B()
{
    std::printf("%d\n",
      app::variable++);
}
```

```
A():
    movl    app::variable(%rip), %esi
    movl    $.LC0, %edi
    leal    1(%rsi), %eax
    movl    %eax, app::variable(%rip)
    xorl    %eax, %eax
    jmp     printf
app::variable:
        .zero   4
```

```
B():
    movl    app::variable(%rip), %esi
    movl    $.LC0, %edi
    leal    1(%rsi), %eax
    movl    %eax, app::variable(%rip)
    xorl    %eax, %eax
    jmp     printf
app::variable:
        .zero   4
```

- Putting it in a header does not fix anything!
- There are 4 types of "linkage" defined in C++, to address the issue of sameness of symbols across multiple translation units.
- Connected with a name, not with a file or project

Linkage type

- No-linkage: variables like block-scope entities can be safely excluded when trying to solve the problem of whether or not the same name appearing in different translation units refer to the same entity
- Internal linkage: the name is treated as private to each translation unit it appears in. Different occurences in different translation units are considered independent entities
- External linkage: Anywhere the name appears in all the translation units, it refers to the same entity. There needs to be a single definition of the object (ODR).
- Module linkage: The symbol is to be treated as the same entity everywhere it appears inside a module, but if it appears outside the module, it is another entity

External linkage

```cpp
// A.cc : A.o and B.o can't be linked together
namespace app {
    int variable{};
}
void A()
{
    std::printf("%d\n",
      app::variable++);
}
```

```cpp
// B.cc : A.o and B.o can't be linked together
namespace app {
    int variable{};
}
void A()
{
    std::printf("%d\n",
      app::variable++);
}
```

- "Top-level"/namespace scope (non-constant) variables and functions have external linkage by default

JÜLICH Forschungszentrum

External linkage

```cpp
// A.cc : A.o and B.o can't be linked together
namespace app {
    int variable{};
}
void A()
{
    std::printf("%d\n",
      app::variable++);
}
```

```cpp
// B.cc : A.o and B.o can't be linked together
namespace app {
    int variable{};
}
void A()
{
    std::printf("%d\n",
      app::variable++);
}
```

- "Top-level"/namespace scope (non-constant) variables and functions have external linkage by default
- One and only one definition is permitted for such a name(ODR: **O**ne **D**efinition **R**ule)

JÜLICH Forschungszentrum

External linkage

```cpp
// A.cc
namespace app {
    extern int variable{};
}
void A()
{
    std::printf("%d\n",
      app::variable++);
}
```

```cpp
// B.cc
namespace app {
    int variable{};
}
void A()
{
    std::printf("%d\n",
      app::variable++);
}
```

- "Top-level"/namespace scope (non-constant) variables and functions have external linkage by default
- One and only one definition is permitted for such a name(ODR: **O**ne **D**efinition **R**ule)
  - Variables: Make sure all translation units but one declare the symbol as `extern`, and the last one defines it without the `extern`. (`extern`: Somewhere out there there is an instance of this variable in memory. The linker will find it)

External linkage

```cpp
// A.cc
namespace app {
    extern int variable{};
}
void A()
{
    std::printf("%d\n",
      app::variable++);
}
```

```cpp
// B.cc
namespace app {
    int variable{};
}
void A()
{
    std::printf("%d\n",
      app::variable++);
}
```

- "Top-level"/namespace scope (non-constant) variables and functions have external linkage by default
- One and only one definition is permitted for such a name(ODR: **O**ne **D**efinition **R**ule)
  - Variables: Make sure all translation units but one declare the symbol as `extern`, and the last one defines it without the `extern`. (`extern`: Somewhere out there there is an instance of this variable in memory. The linker will find it)
  - Functions: Make sure all translation units except one have only access to only the function declaration, and only one translation unit contains the actual definition.

External linkage

```
// var.hh
namespace app {
    inline int variable{};
}
```

```
// A.cc
#include "var.hh"
void A()
{
    std::printf("%d\n",
      app::variable++);
}
```

```
// B.cc
#include "var.hh"
void B()
{
    std::printf("%d\n",
      app::variable++);
}
```

- "Top-level"/namespace scope (non-constant) variables and functions have external linkage by default
- One and only one definition is permitted for such a name(ODR: **O**ne **D**efinition **R**ule)
  - Variables: Make sure all translation units but one declare the symbol as `extern`, and the last one defines it without the `extern`. (`extern`: Somewhere out there there is an instance of this variable in memory. The linker will find it)
  - Functions: Make sure all translation units except one have only access to only the function declaration, and only one translation unit contains the actual definition.
- `inline`: Making a variable or function `inline` frees it from the one definition rule. Even if it has external linkage, the linker treats all instances of as the same entity

Internal linkage

```cpp
// var.hh
namespace app {
    const unsigned long max_dim = 1024UL;
    static auto f(int i) {
        // ...
    }
    static int variable{};
}
```

```cpp
// A.cc
#include "var.hh"
void A()
{
    std::printf("%d\n",
      app::variable++);
}
```

```cpp
// B.cc
#include "var.hh"
void B()
{
    std::printf("%d\n",
      app::variable++);
}
```

- "Top-level"/namespace scope constants have internal linkage by default in C++ (but not in C!)

Internal linkage

```
// var.hh
namespace app {
    const unsigned long max_dim = 1024UL;
    static auto f(int i) {
        // ...
    }
    static int variable{};
}
```

```
// A.cc
#include "var.hh"
void A()
{
    std::printf("%d\n",
      app::variable++);
}
```

```
// B.cc
#include "var.hh"
void B()
{
    std::printf("%d\n",
      app::variable++);
}
```

- "Top-level"/namespace scope constants have internal linkage by default in C++ (but not in C!)
- A free function can be declared `static` give it internal linkage

JÜLICH Forschungszentrum

Internal linkage

```cpp
// var.hh
namespace app {
    const unsigned long max_dim = 1024UL;
    static auto f(int i) {
        // ...
    }
    static int variable{};
}
```

```cpp
// A.cc
#include "var.hh"
void A()
{
    std::printf("%d\n",
      app::variable++);
}
```

```cpp
// B.cc
#include "var.hh"
void B()
{
    std::printf("%d\n",
      app::variable++);
}
```

- "Top-level"/namespace scope constants have internal linkage by default in C++ (but not in C!)
- A free function can be declared `static` give it internal linkage
- A non-constant namespace scope variable can be declared `static` to give it both a static storage duration and internal linkage

Internal linkage

```cpp
// var.hh
namespace app {
    const unsigned long max_dim = 1024UL;
    static auto f(int i) {
        // ...
    }
    static int variable{};
}
```

```cpp
// A.cc
#include "var.hh"
void A()
{
    std::printf("%d\n",
      app::variable++);
}
```

```cpp
// B.cc
#include "var.hh"
void B()
{
    std::printf("%d\n",
      app::variable++);
}
```

- "Top-level"/namespace scope constants have internal linkage by default in C++ (but not in C!)
- A free function can be declared `static` give it internal linkage
- A non-constant namespace scope variable can be declared `static` to give it both a static storage duration and internal linkage
- Since a variable or function with internal linkage can not be "required elsewhere", it is possible for the compiler to perform agressive optimizations, sometimes eliminating the symbol altogether from the translation unit

Module linkage

```
// A.cc
export module A;
import std;
namespace app {
    int variable{};
}
void other_func();
export void A()
{
    std::print("{}\n",
      app::variable++);
    other_func();
}
```

```
// A2.cc
module A;
void other_func()
{
    app::variable += 10;
}
```

```
export module B;
import std;
namespace app {
    int variable{};
}
export void B()
{
    std::print("{}\n",
      app::variable++);
}
```

- In between internal and external linkage: external as far as different module units inside the module are concerned, but visible only inside the module

Module linkage

```
// A.cc
export module A;
import std;
namespace app {
    int variable{};
}
void other_func();
export void A()
{
    std::print("{}\n",
      app::variable++);
    other_func();
}
```

```
// A2.cc
module A;
void other_func()
{
    app::variable += 10;
}
```

```
export module B;
import std;
namespace app {
    int variable{};
}
export void B()
{
    std::print("{}\n",
      app::variable++);
}
```

- In between internal and external linkage: external as far as different module units inside the module are concerned, but visible only inside the module
- Unexported names in the primary module interface unit are regarded as the same entity in all implementation units of the same module

Module linkage

```
// A.cc
export module A;
import std;
namespace app {
    int variable{};
}
void other_func();
export void A()
{
    std::print("{}\n",
      app::variable++);
    other_func();
}
```

```
// A2.cc
module A;
void other_func()
{
    app::variable += 10;
}
```

```
export module B;
import std;
namespace app {
    int variable{};
}
export void B()
{
    std::print("{}\n",
      app::variable++);
}
```

- In between internal and external linkage: external as far as different module units inside the module are concerned, but visible only inside the module
- Unexported names in the primary module interface unit are regarded as the same entity in all implementation units of the same module
- The exact same symbol may be used in a different module

Exercise 1.23:

The folder `examples/linkage` contains tiny demos for internal/external linkage (`intext`) and module linkage (`module`).

- `intext`: Build and run the program as is. Observe the output. Replace `inline` with **`static`** in the `intext/var.hh`. Build and run again. Reason about any differences in the output.
- **`module`**: Build and run using the approriate compiler options. Observe how the symbol `app::variable` is regarded the same across the different files of module `A`, but the same symbol is regarded as an independent object in module `B`

Miscellaneous C++23 updates

Miscellaneous C++23 updates

- Multi-dimensional indexing operator, `operator[](int i, int j, int k)`
- Static `operator()` and `operator[]`
- Explicit object parameter in class member functions
- `[[assume()]]`
- `if consteval`
- `<print>`
- `<flat_map>`, `<flat_set>`: Similar to `map` and `set`, but without the B-tree, node based structure. (E.g., a class keeping a sorted vector as internal storage)
- `<mdspan>`
- `<expected>`
- `ranges::fold_left`...
- `std::views::cartesian_product`
- `std::byteswap`
- ...

Operators [] and ()

- C++23 allows the indexing operator, `operator[]` to take more than one parameter

```
1  class Matrix {
2    auto operator()(unsigned i, unsigned j) const
3      -> double;
4    auto operator[](unsigned i) const
5      -> std::span<const double>;
6  };
7  void f() {
8      Matrix M{10, 10};
9      std::cout << M(4, 4);
10     std::cout << M[4][4];
11 }
```


JÜLICH Forschungszentrum

Operators [] and ()

- C++23 allows the indexing operator, `operator[]` to take more than one parameter

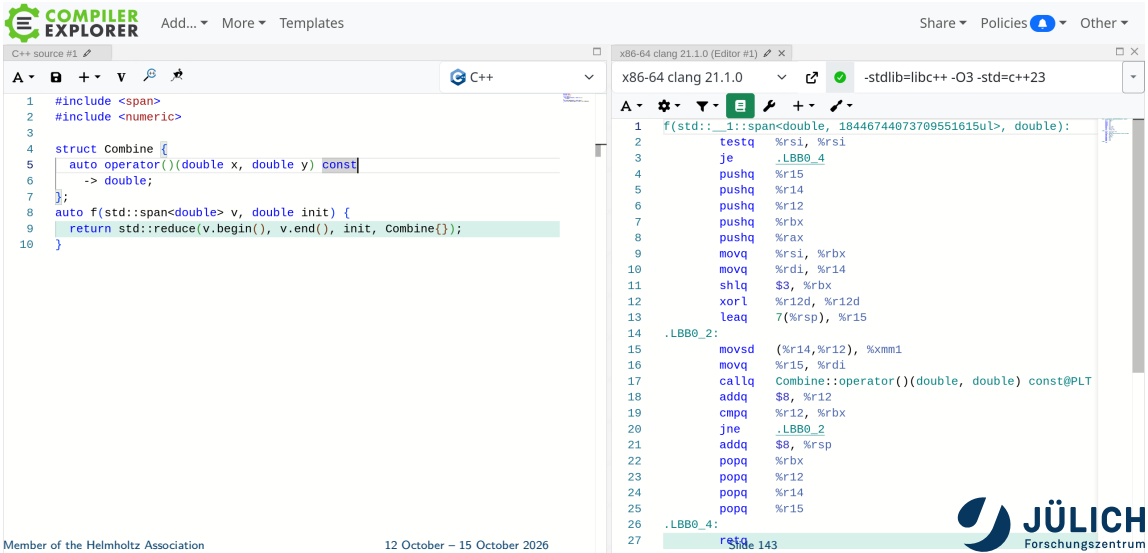
```
1  class Matrix {
2    auto operator()(unsigned i, unsigned j)
3      const -> double;
4    auto operator[](unsigned i, unsigned j)
5      const -> double;
6  };
7  void f() {
8      Matrix M{10, 10};
9      std::cout << M(4, 4);
10     std::cout << M[4, 4];
11 }
```

Operators [] and ()

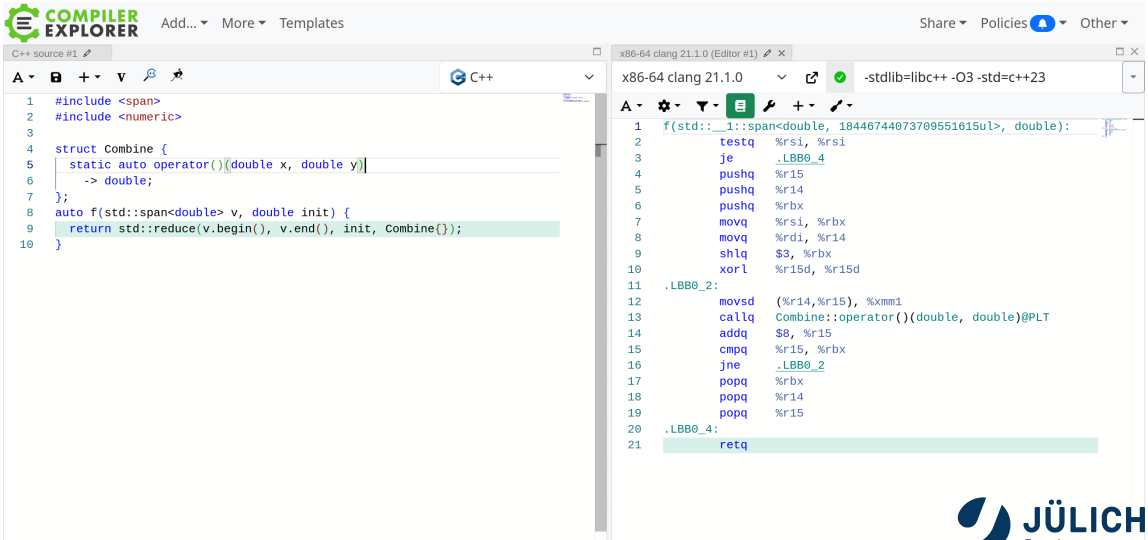
- C++23 allows the indexing operator, `operator[]` to take more than one parameter
- Both `operator()` and `operator[]` can be marked static. Especially useful for stateless function objects, as the function call will no longer require passing the calling instance

```
1  class Combine {
2    static auto operator()(double x, double y)
3      -> double;
4  };
5  auto f(std::span<double> v, double init) {
6    return std::reduce(v.begin(), v.end(),
7        init, Combine{});
8  }
```

Operators [] and ()



Operators [] and ()



Explicit object parameters for member functions

- Methods in a Python class are defined with the calling instance explicitly passed as the `self` parameter

```
1  class Example:
2      def fuzz(self, x):
3          pass
```


JÜLICH Forschungszentrum

Explicit object parameters for member functions

- Methods in a Python class are defined with the calling instance explicitly passed as the `self` parameter
- C++ class member functions, before C++23, couldn't do that. The calling instance in a non-static member function was passed implicitly as the **`this`** pointer

```
1  class Example {
2      double tot{};
3  public:
4      auto put(double x) {
5          tot += x; // 'this' : Example*
6      }
7      auto get() const {
8          return tot; // 'this' : const Example*
9      }
10 };
```

Explicit object parameters for member functions

- Methods in a Python class are defined with the calling instance explicitly passed as the `self` parameter
- C++ class member functions, before C++23, couldn't do that. The calling instance in a non-static member function was passed implicitly as the **`this`** pointer
- **`this`** is a **`const`** pointer if the member is declared **`const`**

```
1  class Example {
2      double tot{};
3  public:
4      auto put(double x) {
5          tot += x; // 'this' : Example*
6      }
7      auto get() const {
8          return tot; // 'this' : const Example*
9      }
10 };
```


JÜLICH Forschungszentrum

Explicit object parameters for member functions

- Methods in a Python class are defined with the calling instance explicitly passed as the `self` parameter
- C++ class member functions, before C++23, couldn't do that. The calling instance in a non-static member function was passed implicitly as the **`this`** pointer
- **`this`** is a **`const`** pointer if the member is declared **`const`**
- Since C++23, we can expose the calling instance explicitly in a C++ member function

```
1  class Example {
2      double tot{};
3  public:
4      auto put(this Example& self,
5          double x) {
6          self.tot += x;
7      }
8      auto get(this Example const& self) {
9          return self.tot;
10     }
11 };
```

Explicit object parameters for member functions

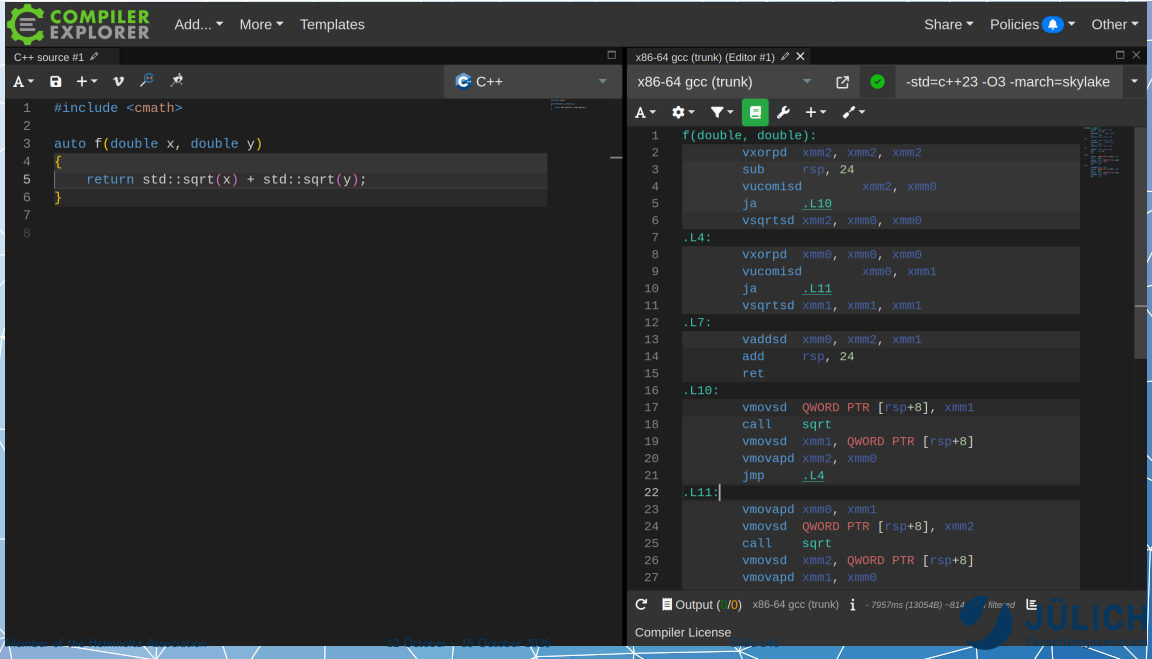
- Methods in a Python class are defined with the calling instance explicitly passed as the `self` parameter
- C++ class member functions, before C++23, couldn't do that. The calling instance in a non-static member function was passed implicitly as the **`this`** pointer
- **`this`** is a **`const`** pointer if the member is declared **`const`**
- Since C++23, we can expose the calling instance explicitly in a C++ member function
- Most useful when we combine this with template argument deduction, to combine **`const`**/non-**`const`**, L-value-ref / R-value-ref versions of the same member as one function template

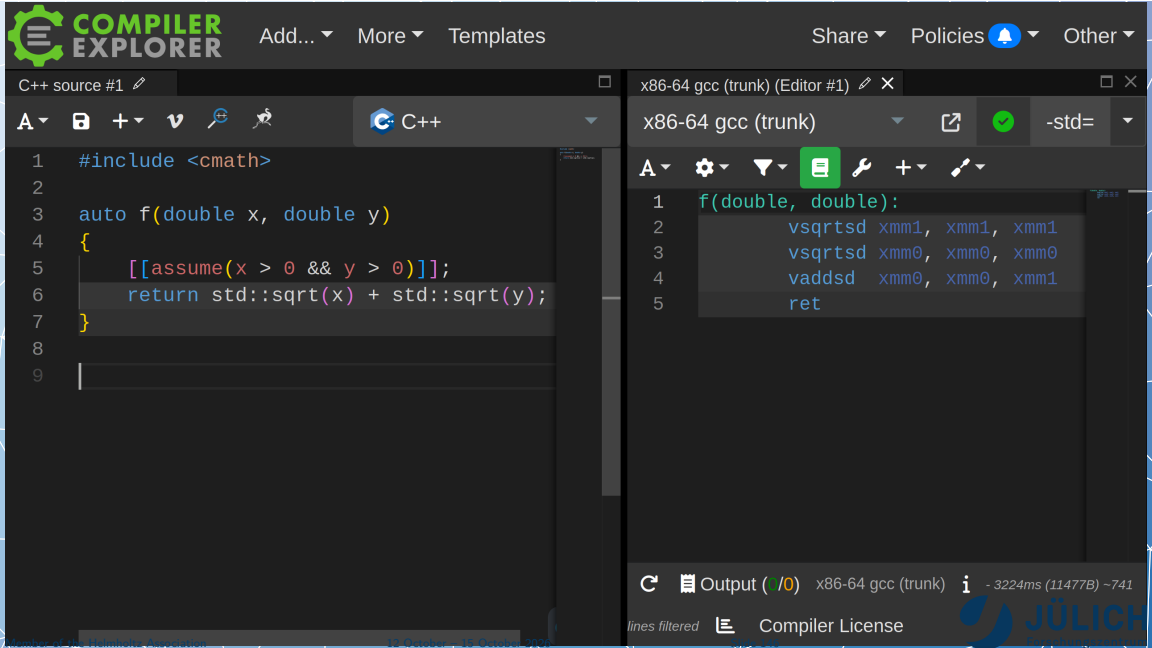
```
1  class Example {
2      double tot{};
3  public:
4      auto put(this Example& self,
5          double x) {
6          self.tot += x;
7      }
8      auto get(this Example const& self) {
9          return self.tot;
10     }
11 };
```


JÜLICH Forschungszentrum

Expressing assumptions

- Sometimes, relationships between function inputs can not be expressed through their types. The application developer might know that
  - the floating point input to a function is always between 0 and 1.
  - two integer inputs are always ordered smaller, greater
  - an array is never empty
  - ...
- When the compiler translates our code, such information is usually not available.
- To ensure correct results, code is generated to handle all kinds of corner cases, which we are certain can not ever happen
- C++23 introduced one such way to express such relations in code: `[[assume(expr)]]`
- `[[assume()]]` expressions may be placed anywhere in the function body and allow the compiler to make those assumptions at that point in code
- This gives a license to the compiler to make those assumptions and hence possibly generate some faster code. But faster code is not guaranteed.
- If the explicitly expressed assumptions are violated at the runtime, the result is undefined behaviour.
- It is usually better to use `[[assume(expr)]]` along with `assert` so that violations are detected during debugging


COMPILER EXPLORER
Add...
More
Templates
Share
Policies
Other
C++ source #1
C++
#include <cmath>
auto f(double x, double y)
{
return std::sqrt(x) + std::sqrt(y);
}
x86-64 gcc (trunk) (Editor #1)
x86-64 gcc (trunk)
-std=c++23 -O3 -march=skylake
f(double, double):
vxorpd xmm2, xmm2, xmm2
sub rsp, 24
vucomisd xmm2, xmm0
ja .L10
vsqrtsd xmm2, xmm0, xmm0
.L4:
vxorpd xmm0, xmm0, xmm0
vucomisd xmm0, xmm1
ja .L11
vsqrtsd xmm1, xmm1, xmm1
.L7:
vaddsd xmm0, xmm2, xmm1
add rsp, 24
ret
.L10:
vmovsd QWORD PTR [rsp+8], xmm1
call sqrt
vmovsd xmm1, QWORD PTR [rsp+8]
vmovapd xmm2, xmm0
jmp .L4
.L11:
vmovapd xmm0, xmm1
vmovsd QWORD PTR [rsp+8], xmm2
call sqrt
vmovsd xmm2, QWORD PTR [rsp+8]
vmovapd xmm1, xmm0
Output (/0)
x86-64 gcc (trunk)
Compiler License


COMPILER EXPLORER
Add...
More
Templates
Share
Policies
Other
C++ source #1
C++
#include <cmath>
auto f(double x, double y)
{
[[assume(x > 0 && y > 0)]];
return std::sqrt(x) + std::sqrt(y);
}
x86-64 gcc (trunk) (Editor #1)
x86-64 gcc (trunk)
-std=
f(double, double):
vsqrtsd xmm1, xmm1, xmm1
vsqrtsd xmm0, xmm0, xmm0
vaddsd xmm0, xmm0, xmm1
ret
Output (0/0)
x86-64 gcc (trunk)
- 3224ms (11477B) ~741
lines filtered
Compiler License

Byteswap

- `std::byteswap` can create an integer by swapping the bytes of an input integer

```cpp
#include <bit>
#include <print>
auto main() -> int
{
    std::print("{:0>8X} <-> {:0>8X}\n",
        0xC0FFEEu, std::byteswap(0xC0FFEEu));
    std::print("{:0>8X} <-> {:0>8X}\n",
        0xCAFEu, std::byteswap(0xCAFEu));
    std::print("{:0>8X} <-> {:0>8X}\n",
        0xFACEu, std::byteswap(0xFACEu));
    std::print("{:0>8X} <-> {:0>8X}\n",
        0xFACADEu, std::byteswap(0xFACADEu));
    std::print("{:0>8X} <-> {:0>8X}\n",
        0xFEEDu, std::byteswap(0xFEEDu));
}
```

```
00C0FFEE <-> EEFFC000
0000CAFE <-> FECA0000
0000FACE <-> CEFA0000
00FACADE <-> DECAFA00
0000FEED <-> EDFE0000
```

Optional values

```
#include <optional>
auto f(double x) -> std::optional<double> {
    std::optional<double> ans;
    const auto eps2 = 1.0e-24;
    if (x >= 0) {
        auto r0 = 0.5 * (1. + x);
        auto r1 = x / r0;
        while ((r0 - r1) * (r0 - r1) > eps2) {
            r0 = 0.5 * (r0 + r1);
            r1 = x / r0;
        }
        ans = r1;
    }
    return ans;
}
// Elsewhere...
std::cout << "Enter number : ";
std::cin >> x;
if (auto r = f(x); r.has_value()) {
    std::cout << "The result is "
          << r.value() << '\n';
}
```

- `std::optional<T>` is analogous to a box containing exactly one object of type `T` or nothing at all
- If created without any initialisers, the box is empty
- You store something in the box by assigning to the `optional`
- Evaluating the optional as a boolean gives a **`true`** outcome if there is an object inside, irrespective of the value of that object
- Empty box evaluates to **`false`**

Optional values

```
#include <optional>
auto f(double x) -> std::optional<double> {
    std::optional<double> ans;
    const auto eps2 = 1.0e-24;
    if (x >= 0) {
        auto r0 = 0.5 * (1. + x);
        auto r1 = x / r0;
        while ((r0 - r1) * (r0 - r1) > eps2) {
            r0 = 0.5 * (r0 + r1);
            r1 = x / r0;
        }
        ans = r1;
    }
    return ans;
}
// Elsewhere...
std::cout << "Enter number : ";
std::cin >> x;
if (auto r = f(x); r) {
    std::cout << "The result is "
          << *r << '\n';
}
```

- `std::optional<T>` is analogous to a box containing exactly one object of type `T` or nothing at all
- If created without any initialisers, the box is empty
- You store something in the box by assigning to the `optional`
- Evaluating the optional as a boolean gives a **`true`** outcome if there is an object inside, irrespective of the value of that object
- Empty box evaluates to **`false`**

C++23 std::expected

```cpp
#include <expected>
auto mysqrt(double x) -> std::expected<double, std::string> {
    const auto eps = 1.0e-12;
    const auto eps2 = eps * eps;
    if (x >= 0.) {
        auto r0 = 0.5 * (1. + x);
        auto r1 = x / r0;
        while ((r0 - r1) * (r0 - r1) > eps2) {
            r0 = 0.5 * (r0 + r1);
            r1 = x / r0;
        }
        return { r1 };
    } else {
        return std::unexpected { "Unexpected input!" };
    }
}
// Elsewhere...
if (auto rm = mysqrt(x); rm) std::cout << "Square root = " << rm.value() << "\n";
else std::cout << "Error: " << rm.error() << "\n";
```

- Similar to `std::optional`, but has more capacity to describe the error
- The *unexpected* value can be of a type of our choosing, making it very flexible

Reference qualified member functions

```
1  struct Box {
2      value_type r{};
3      auto value() const -> const value_type&
4      {
5          return r;
6      }
7      auto value() -> value_type& { return r; }
8  };
```

- If `b` is of the type `Box`, `b.value()` is a `const value_type&` or just `value_type&` depending on whether `b` is `const` or not

Reference qualified member functions

```cpp
// Since C++11
struct Box {
    value_type r{};
    auto value() const & -> const value_type&
    {
        return r;
    }
    auto value() & -> value_type& { return r; }
    auto value() const && -> const value_type&&
    {
        return r;
    }
    auto value() && -> value_type&& { return r; }
};
```

- If `b` is of the type `Box`, `b.value()` is a `const value_type&` or just `value_type&` depending on whether `b` is `const` or not
- Since C++11, more overloads are possible: one can have different versions of member functions depending on whether the calling instance is an L-value or an R-value reference.

Reference qualified member functions

```cpp
// Since C++11
struct Box {
    value_type r{};
    auto value() const & -> const value_type&
    {
        return r;
    }
    auto value() & -> value_type& { return r; }
    auto value() const && -> const value_type&&
    {
        return r;
    }
    auto value() && -> value_type&& { return r; }
};
```

- If `b` is of the type `Box`, `b.value()` is a `const value_type&` or just `value_type&` depending on whether `b` is `const` or not
- Since C++11, more overloads are possible: one can have different versions of member functions depending on whether the calling instance is an L-value or an R-value reference.
- Potentially quadruples the number of variations of a member function depending on the calling instance

Reference qualified member functions

```
1  struct FileData {
2      std::string header_text{};
3      std::vector<std::byte> bulk{};
4
5      auto header() const & -> const std::string&
6      {
7          return header_text;
8      }
9      auto header() & -> std::string&
10     {
11         return header_text;
12     }
13     auto header() const && -> const std::string&&
14     {
15         return std::move(header_text);
16     }
17     auto header() && -> std::string&&
18     {
19         return std::move(header_text);
20     }
21 };
22
23 auto readfile(std::filesystem::path fn) -> FileData;
```

- If `b` is of the type `Box`, `b.value()` is a `const value_type&` or just `value_type&` depending on whether `b` is `const` or not
- Since C++11, more overloads are possible: one can have different versions of member functions depending on whether the calling instance is an L-value or an R-value reference.
- Potentially quadruples the number of variations of a member function depending on the calling instance
- Sometimes, it is possible to provide some optimisations in situations where the calling instance is an R-value. An example demonstrating this can be explored in the notebook `ref_qual_members.ipynb`

Using templates for deduplication

```
struct Entity {
  Entity(const Vbose& x, const Vbose& y)
    : l{x}, r{y} {}
  Entity(const Vbose& x, Vbose&& y)
    : l{x}, r{std::move(y)} {}
  Entity(Vbose&& x, const Vbose& y)
    : l{std::move(x)}, r{y} {}
  Entity(Vbose&& x, Vbose&& y)
    : l{std::move(x)}, r{std::move(y)} {}

  Vbose l, r;
};
```

```
template <class T>
struct Entity {
  template <class U, class V>
  Entity(U&& x, V&& y)
      : l{std::forward<U>(x)},
      r{std::forward<V>(y)} {}

  T l, r;
};
```

- In the notebook `perfect_forwarding.ipynb` we explored a vaguely similar situation
- Instead of the 4 constructors in the first example, we could write a single function template, using forwarding references and `std::forward`
- The forwarding references, `U&&` and `V&&` capture the constantness L/R-value reference characteristics of the inputs
- The `std::forward` wrapping the uses of the respective variables casts them into their fully CVR qualified typenames.
- Can we do something like that to reduce the clutter in the previous examples?

Using templates for deduplication

```
1  struct FileData {
2      std::string header_text{};
3      std::vector<std::byte> bulk{};
4
5      auto header() const & -> const std::string&
6      {
7          return header_text;
8      }
9      auto header() & -> std::string&
10     {
11         return header_text;
12     }
13     auto header() const && -> const std::string&&
14     {
15         return std::move(header_text);
16     }
17     auto header() && -> std::string&&
18     {
19         return std::move(header_text);
20     }
21 };
```

- Imagine that, instead of these class member functions...

JÜLICH Forschungszentrum

Using templates for deduplication

```
1   struct FileData {
2       std::string header_text{};
3       std::vector<std::byte> bulk{};
4   };
5   auto header(const FileData& fd) -> const std::string&
6   {
7       return fd.header_text;
8   }
9   auto header(FileData& fd) -> std::string&
10  {
11      return fd.header_text;
12  }
13  auto header(const FileData&& fd) -> const std::string&&
14  {
15      return std::move(fd.header_text);
16  }
17  auto header(FileData && fd) -> std::string&&
18  {
19      return std::move(fd.header_text);
20  }
21  };
```

- Imagine that, instead of these class member functions...
- we had a set of free standing functions doing the same thing

JÜLICH Forschungszentrum

Using templates for deduplication

```
1   struct FileData {
2       std::string header_text{};
3       std::vector<std::byte> bulk{};
4   };
5   template <class C>
6   requires std::same_as<FileData,
7                std::remove_cvref_t<C>>
8   auto&& header(C&& fd)
9   {
10      return std::forward<C>(fd).header_text;
11  }
```

- Imagine that, instead of these class member functions...
- we had a set of free standing functions doing the same thing
- We could easily write a single template version for all this. This is doable, because the `FileData` object is explicitly available, and therefore can be made into a template parameter and so on.

Using templates for deduplication

```
1  struct FileData {
2      std::string header_text{};
3      std::vector<std::byte> bulk{};
4
5      auto header() const & -> const std::string&
6      {
7          return header_text;
8      }
9      auto header() & -> std::string&
10     {
11         return header_text;
12     }
13     auto header() const && -> const std::string&&
14     {
15         return std::move(header_text);
16     }
17     auto header() && -> std::string&&
18     {
19         return std::move(header_text);
20     }
21 };
```

- Imagine that, instead of these class member functions...
- we had a set of free standing functions doing the same thing
- We could easily write a single template version for all this. This is doable, because the `FileData` object is explicitly available, and therefore can be made into a template parameter and so on.
- Member functions don't expose the calling instance in the same way, since it is passed implicitly.

Using templates for deduplication

```
1  struct FileData {
2      std::string header_text{};
3      std::vector<std::byte> bulk{};
4      template <class Self>
5      auto&& header(this Self&& self)
6      {
7          // No 'this', only 'self'!
8          return std::forward<Self>(self)
9              .header_text;
10     }
11 };
12
```

- Imagine that, instead of these class member functions...
- we had a set of free standing functions doing the same thing
- We could easily write a single template version for all this. This is doable, because the `FileData` object is explicitly available, and therefore can be made into a template parameter and so on.
- Member functions don't expose the calling instance in the same way, since it is passed implicitly.
- Since in C++23, we can pass it explicitly, new possibilities...

Using templates for deduplication

```
1  struct FileData {
2      std::string header_text{};
3      std::vector<std::byte> bulk{};
4      template <class Self>
5      auto&& header(this Self&& self)
6      {
7          // No 'this', only 'self'!
8          return std::forward<Self>(self)
9              .header_text;
10     }
11 };
12
```

- Imagine that, instead of these class member functions...
- we had a set of free standing functions doing the same thing
- We could easily write a single template version for all this. This is doable, because the `FileData` object is explicitly available, and therefore can be made into a template parameter and so on.
- Member functions don't expose the calling instance in the same way, since it is passed implicitly.
- Since in C++23, we can pass it explicitly, new possibilities...
- The names `Self` etc are not special. You choose.

Using templates for deduplication

```
1   struct FileData {
2       std::string header_text{};
3       std::vector<std::byte> bulk{};
4       template <class Self>
5       auto&& header(this Self&& self)
6       {
7           // No 'this', only 'self'!
8           return std::forward<Self>(self)
9               .header_text;
10      }
11  };
12
```

- Imagine that, instead of these class member functions...
- we had a set of free standing functions doing the same thing
- We could easily write a single template version for all this. This is doable, because the `FileData` object is explicitly available, and therefore can be made into a template parameter and so on.
- Member functions don't expose the calling instance in the same way, since it is passed implicitly.
- Since in C++23, we can pass it explicitly, new possibilities...
- The names `Self` etc are not special. You choose.
- The special syntax to explicitly name the calling instance is shown here

Some smaller C++26 changes

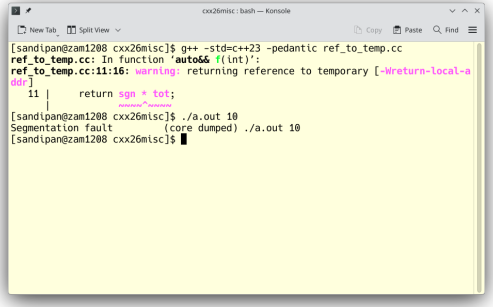
Safety features

- Through direct mistakes or inadequately carefull edits, we sometimes end up with memory errors: Here we are returning a reference to a temporary.

```
1  #include <print>
2  auto&& f(int x)
3  {
4      auto sgn = x < 0 ? -1 : 1;
5      auto tot = 0;
6      if (x < 0) x = -x;
7      while (x > 0) {
8          tot += x;
9          --x;
10     }
11     return sgn * tot;
12 }
13 auto main(int argc, char* argv[]) -> int
14 {
15     auto inp = argc > 1 ?
16         std::stoi(argv[1]) : 0;
17     std::print("Sum up to {} is {}\n",
18         inp, f(inp));
19 }
```

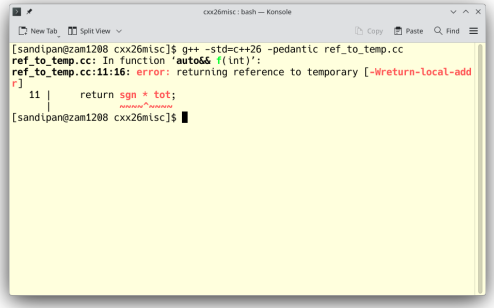
Safety features

- Through direct mistakes or inadequately carefull edits, we sometimes end up with memory errors: Here we are returning a reference to a temporary.
- This builds with a warning in C++23



Safety features

- Through direct mistakes or inadequately carefull edits, we sometimes end up with memory errors: Here we are returning a reference to a temporary.
- This builds with a warning in C++23
- C++26 finally makes it into an error



Pack indexing

```
1  // C++14 code
2  template <unsigned I, class... Args>
3  struct pick_helper;
4  template <unsigned I, class Arg0, class... Rest>
5  struct pick_helper<I, Arg0, Rest...> {
6    static constexpr auto get(Arg0 arg0, Rest... args) {
7      return pick_helper<I - 1, Rest...>::get(args...);
8    }
9  };
10 template <class Arg0, class... Rest>
T 11 struct pick_helper<0U, Arg0, Rest...> {
12   static constexpr auto get(Arg0 arg0, Rest... args) {
13     return arg0;
14   }
15 };
16 template <unsigned I, class... Args>
17 constexpr auto pick_Ith(Args... args)
18 {
19   static_assert(I < sizeof...(Args), "Invalid index");
20   return pick_helper<I, Args...>::get(args...);
21 }
22 static_assert(pick_Ith<2U>(0, 1, 2, 3) == 3, "Bad pick!");
23
```

- Template parameter packs were introduced in C++11, but without a convenient interface to address the elements of the pack

Pack indexing

```cpp
// C++14 code
template <unsigned I, class... Args>
struct pick_helper;
template <unsigned I, class Arg0, class... Rest>
struct pick_helper<I, Arg0, Rest...> {
  static constexpr auto get(Arg0 arg0, Rest... args) {
    return pick_helper<I - 1, Rest...>::get(args...);
  }
};
template <class Arg0, class... Rest>
struct pick_helper<0U, Arg0, Rest...> {
  static constexpr auto get(Arg0 arg0, Rest... args) {
    return arg0;
  }
};
template <unsigned I, class... Args>
constexpr auto pick_Ith(Args... args)
{
  static_assert(I < sizeof...(Args), "Invalid index");
  return pick_helper<I, Args...>::get(args...);
}
static_assert(pick_Ith<2U>(0, 1, 2, 3) == 3, "Bad pick!");

```

- Template parameter packs were introduced in C++11, but without a convenient interface to address the elements of the pack
- Recursion + specialization was used to achieve the effect, but it is cumbersome, and error prone

Pack indexing

```
1  // With C++26
2  template <unsigned I, class... Args>
3  auto pick_Ith(Args... args)
4  {
5      return args...[I];
6  }
```

- Template parameter packs were introduced in C++11, but without a convenient interface to address the elements of the pack
- Recursion + specialization was used to achieve the effect, but it is cumbersome, and error prone
- C++26 allows direct access to the elements of a pack, in a form reminiscent of array indexing, albeit with compile-time fixed indexes

Pack indexing

```
1  // With C++26
2  template <unsigned I, class... Args>
3  auto pick_Ith(Args... args)
4  {
5      return args...[I];
6  }
```

- Template parameter packs were introduced in C++11, but without a convenient interface to address the elements of the pack
- Recursion + specialization was used to achieve the effect, but it is cumbersome, and error prone
- C++26 allows direct access to the elements of a pack, in a form reminiscent of array indexing, albeit with compile-time fixed indexes
- Since elements of the pack can have different types, we are not calling `operator[]` on some object called `args...`. Think of `args...[I]` as a new kind of expression for the compiler to evaluate.

JÜLICH Forschungszentrum

Structured bindings, pack indexing, template for...

```
1  namespace sr = std::ranges;
2  namespace sv = sr::views;
3  template <class T>
4  concept addable = requires (T a, T b) {
5      { a + b } -> std::convertible_to<T>;
6  };
7  template <addable... Args>
8  auto add(const std::tuple<Args...>& t1,
9           const std::tuple<Args...>& t2) {
10     std::tuple<Args...> t3{};
11     auto&& [...c1] = t1;
12     auto&& [...c2] = t2;
13     auto&& [...c3] = t3;
14     template for (constexpr auto i :
15             sv::indices(sizeof...(Args))) {
16         c3...[i] = c1...[i] + c2...[i];
17     }
18     return t3;
19 }
20 auto main() -> int {
21     std::tuple<int, double, std::string> t1{1, 9., "A"};
22     std::tuple<int, double, std::string> t2{2, 3., "B"};
23     std::print("{} + {} = {}\n", t1, t2, add(t1, t2));
24 }
```

- It is now possible to introduce parameter packs in structured bindings, as long as it is done inside a template
- Packs can be indexed with strictly compile time constant indexes, with the obvious meaning of the i'th index
- `template for`, known as an "expansion statement", is an entirely compile time loop like construct.

JÜLICH Forschungszentrum

Contracts

Correctness checks

Pre-conditions, post-conditions, assertions

- A lot of error handling code written using assertions and exceptions are for verifying that the function inputs make sense (pre-conditions)
- Less commonly, we sometimes verify that the return values satisfy known constraints (post-conditions), so as to provide guarantees to the consumers of the function
- Regular logical assertions in the function body test of what should be true at that point

Correctness checks

Pre-conditions, post-conditions, assertions

- A lot of error handling code written using assertions and exceptions are for verifying that the function inputs make sense (pre-conditions)
- Less commonly, we sometimes verify that the return values satisfy known constraints (post-conditions), so as to provide guarantees to the consumers of the function
- Regular logical assertions in the function body test of what should be true at that point
- When one of these assumptions fails, the program is in an erroneous state, and we want to fix it, by addressing the logical errors on our part leading to that situation.
- This is why classical `assert` macro terminates the program when an assertion fails

Correctness checks

Pre-conditions, post-conditions, assertions

- A lot of error handling code written using assertions and exceptions are for verifying that the function inputs make sense (pre-conditions)
- Less commonly, we sometimes verify that the return values satisfy known constraints (post-conditions), so as to provide guarantees to the consumers of the function
- Regular logical assertions in the function body test of what should be true at that point
- When one of these assumptions fails, the program is in an erroneous state, and we want to fix it, by addressing the logical errors on our part leading to that situation.
- This is why classical `assert` macro terminates the program when an assertion fails
- By contrast, the `try {} catch (...) {}` mechanism of exceptions is designed with the idea of a program *recovering* from an error while running

Correctness checks

Pre-conditions, post-conditions, assertions

- A lot of error handling code written using assertions and exceptions are for verifying that the function inputs make sense (pre-conditions)
- Less commonly, we sometimes verify that the return values satisfy known constraints (post-conditions), so as to provide guarantees to the consumers of the function
- Regular logical assertions in the function body test of what should be true at that point
- When one of these assumptions fails, the program is in an erroneous state, and we want to fix it, by addressing the logical errors on our part leading to that situation.
- This is why classical `assert` macro terminates the program when an assertion fails
- By contrast, the `try {} catch (...) {}` mechanism of exceptions is designed with the idea of a program *recovering* from an error while running
- Sometimes, it would be nice to note where the assertion conditions fail, but letting the program continue, to observe, and hopefully, get clues about the true cause of the error

Contracts

- Dedicated syntax, `pre(cond)`/`post(cond)` to specify pre-conditions and post-conditions outside the function body

```
1  auto dot(span<const double> x,
2       span<const double> y, double cut)
3           -> double
4  pre(x.size() > 0UL)
5  pre(x.size() == y.size())
6  post(r: r >= 0.)
7  {
8       auto sum = 0.;
9       for (auto i = 0UL; i < x.size(); ++i) {
10          sum += x[i] * y[i];
11          contract_assert(x[i] > cut);
12          contract_assert(y[i] > cut);
13      }
14 }
```

- GCC's current experimental implementation makes the evaluation semantics into a per-translation-unit compile time choice: `-fcontract-evaluation-semantic=ignore|observe|enforce|...`

Contracts

- Dedicated syntax, `pre(cond)`/`post(cond)` to specify pre-conditions and post-conditions outside the function body
- Different evaluation semantics regarding what to do when a contract is violated:

```
1  auto dot(span<const double> x,
2       span<const double> y, double cut)
3           -> double
4  pre(x.size() > 0UL)
5  pre(x.size() == y.size())
6  post(r: r >= 0.)
7  {
8       auto sum = 0.;
9       for (auto i = 0UL; i < x.size(); ++i) {
10          sum += x[i] * y[i];
11          contract_assert(x[i] > cut);
12          contract_assert(y[i] > cut);
13      }
14 }
```

- GCC's current experimental implementation makes the evaluation semantics into a per-translation-unit compile time choice: `-fcontract-evaluation-semantic=ignore|observe|enforce|...`

Contracts

- Dedicated syntax, `pre(cond)`/`post(cond)` to specify pre-conditions and post-conditions outside the function body
- Different evaluation semantics regarding what to do when a contract is violated:
  1. ignore: do nothing. Like `assert` when built with `-DNDEBUG`

```
1  auto dot(span<const double> x,
2       span<const double> y, double cut)
3           -> double
4  pre(x.size() > 0UL)
5  pre(x.size() == y.size())
6  post(r: r >= 0.)
7  {
8       auto sum = 0.;
9       for (auto i = 0UL; i < x.size(); ++i) {
10          sum += x[i] * y[i];
11          contract_assert(x[i] > cut);
12          contract_assert(y[i] > cut);
13      }
14 }
```

- GCC's current experimental implementation makes the evaluation semantics into a per-translation-unit compile time choice: `-fcontract-evaluation-semantic=ignore|observe|enforce|...`

Contracts

- Dedicated syntax, `pre(cond)`/`post(cond)` to specify pre-conditions and post-conditions outside the function body
- Different evaluation semantics regarding what to do when a contract is violated:
  1. ignore: do nothing. Like `assert` when built with `-DNDEBUG`
  2. observe: record the violation, but continue

```
1  auto dot(span<const double> x,
2       span<const double> y, double cut)
3           -> double
4  pre(x.size() > 0UL)
5  pre(x.size() == y.size())
6  post(r: r >= 0.)
7  {
8       auto sum = 0.;
9       for (auto i = 0UL; i < x.size(); ++i) {
10          sum += x[i] * y[i];
11          contract_assert(x[i] > cut);
12          contract_assert(y[i] > cut);
13      }
14 }
```

- GCC's current experimental implementation makes the evaluation semantics into a per-translation-unit compile time choice: `-fcontract-evaluation-semantic=ignore|observe|enforce|...`

Contracts

- Dedicated syntax, `pre(cond)`/`post(cond)` to specify pre-conditions and post-conditions outside the function body
- Different evaluation semantics regarding what to do when a contract is violated:
  1. ignore: do nothing. Like `assert` when built with `-DNDEBUG`
  2. observe: record the violation, but continue
  3. enforce: terminate on violation, but after calling a contract-violation handler. Similar to a failed assertion when no `-DNDEBUG` flag is used for compilation, but with the possibility of a user provided contract-violation handler.

```
1  auto dot(span<const double> x,
2       span<const double> y, double cut)
3           -> double
4  pre(x.size() > 0UL)
5  pre(x.size() == y.size())
6  post(r: r >= 0.)
7  {
8       auto sum = 0.;
9       for (auto i = 0UL; i < x.size(); ++i) {
10          sum += x[i] * y[i];
11          contract_assert(x[i] > cut);
12          contract_assert(y[i] > cut);
13      }
14 }
```

- GCC's current experimental implementation makes the evaluation semantics into a per-translation-unit compile time choice: `-fcontract-evaluation-semantic=ignore|observe|enforce|...`

Contracts

- Dedicated syntax, `pre(cond)`/`post(cond)` to specify pre-conditions and post-conditions outside the function body
- Different evaluation semantics regarding what to do when a contract is violated:
  1. ignore: do nothing. Like `assert` when built with `-DNDEBUG`
  2. observe: record the violation, but continue
  3. enforce: terminate on violation, but after calling a contract-violation handler. Similar to a failed assertion when no `-DNDEBUG` flag is used for compilation, but with the possibility of a user provided contract-violation handler.
  4. quick-enforce: terminate on violation even more rudely than a classical assert macro!

```
1  auto dot(span<const double> x,
2       span<const double> y, double cut)
3           -> double
4  pre(x.size() > 0UL)
5  pre(x.size() == y.size())
6  post(r: r >= 0.)
7  {
8       auto sum = 0.;
9       for (auto i = 0UL; i < x.size(); ++i) {
10          sum += x[i] * y[i];
11          contract_assert(x[i] > cut);
12          contract_assert(y[i] > cut);
13      }
14 }
```

- GCC's current experimental implementation makes the evaluation semantics into a per-translation-unit compile time choice: `-fcontract-evaluation-semantic=ignore|observe|enforce|...`

Contracts

- Dedicated syntax, `pre(cond)`/`post(cond)` to specify pre-conditions and post-conditions outside the function body
- Different evaluation semantics regarding what to do when a contract is violated:
  1. ignore: do nothing. Like `assert` when built with `-DNDEBUG`
  2. observe: record the violation, but continue
  3. enforce: terminate on violation, but after calling a contract-violation handler. Similar to a failed assertion when no `-DNDEBUG` flag is used for compilation, but with the possibility of a user provided contract-violation handler.
  4. quick-enforce: terminate on violation even more rudely than a classical assert macro!
- An `assert` mechanism, contract asserts, following the same rules as `pre` and `post` above

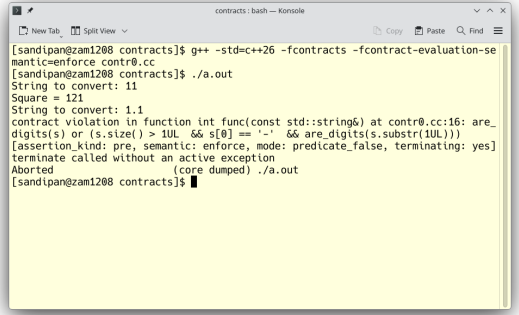
```
1  auto dot(span<const double> x,
2       span<const double> y, double cut)
3           -> double
4  pre(x.size() > 0UL)
5  pre(x.size() == y.size())
6  post(r: r >= 0.)
7  {
8       auto sum = 0.;
9       for (auto i = 0UL; i < x.size(); ++i) {
10          sum += x[i] * y[i];
11          contract_assert(x[i] > cut);
12          contract_assert(y[i] > cut);
13      }
14 }
```

- GCC's current experimental implementation makes the evaluation semantics into a per-translation-unit compile time choice: `-fcontract-evaluation-semantic=ignore|observe|enforce|...`

Contract evaluation semantic

```
1  import std;
2  namespace sr = std::ranges;
3  constexpr auto are_digits(std::string_view R)
4  {
5      return sr::all_of(R, [](char c) {
6          return std::isdigit(c);
7      });
8  }
9  auto func(const std::string& s) -> int
10     pre(not s.empty())
11     pre(are_digits(s) or
12             (s.size() > 1UL
13              && s[0] == '-'
14              && are_digits(s.substr(1UL))))
15 { return std::stoi(s); }
16 auto main(int argc, char *argv[]) -> int
17 {
18     while (1) {
19         std::string inp;
20         std::cout << "String to convert: ";
21         std::cin >> inp;
22         auto i = func(inp);
23         std::cout << "Square = "
24             << i * i << "\n";
25     }
26 }
```

- `-std=c++26 -fcontracts`
- In addition... `-fcontract-evaluation-semantic=`
  - `enforce`

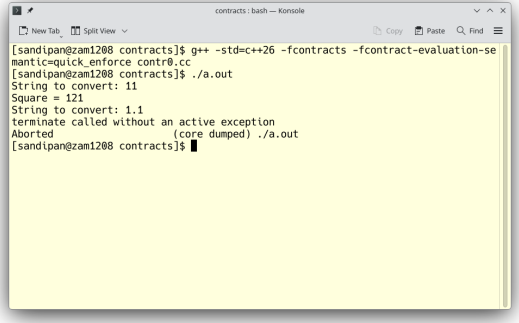


JÜLICH Forschungszentrum

Contract evaluation semantic

```
1  import std;
2  namespace sr = std::ranges;
3  constexpr auto are_digits(std::string_view R)
4  {
5      return sr::all_of(R, [](char c) {
6          return std::isdigit(c);
7      });
8  }
9  auto func(const std::string& s) -> int
10     pre(not s.empty())
11     pre(are_digits(s) or
12             (s.size() > 1UL
13              && s[0] == '-'
14              && are_digits(s.substr(1UL))))
15 { return std::stoi(s); }
16 auto main(int argc, char *argv[]) -> int
17 {
18     while (1) {
19         std::string inp;
20         std::cout << "String to convert: ";
21         std::cin >> inp;
22         auto i = func(inp);
23         std::cout << "Square = "
24             << i * i << "\n";
25     }
26 }
```

- `-std=c++26 -fcontracts`
- In addition... `-fcontract-evaluation-semantic=`
  - `enforce`
  - `quick_enforce`

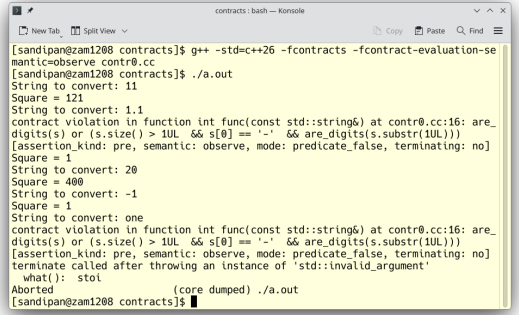


JÜLICH Forschungszentrum

Contract evaluation semantic

```
1  import std;
2  namespace sr = std::ranges;
3  constexpr auto are_digits(std::string_view R)
4  {
5      return sr::all_of(R, [](char c) {
6          return std::isdigit(c);
7      });
8  }
9  auto func(const std::string& s) -> int
10     pre(not s.empty())
11     pre(are_digits(s) or
12             (s.size() > 1UL
13              && s[0] == '-'
14              && are_digits(s.substr(1UL))))
15 { return std::stoi(s); }
16 auto main(int argc, char *argv[]) -> int
17 {
18     while (1) {
19         std::string inp;
20         std::cout << "String to convert: ";
21         std::cin >> inp;
22         auto i = func(inp);
23         std::cout << "Square = "
24             << i * i << "\n";
25     }
26 }
```

- `-std=c++26 -fcontracts`
- In addition... `-fcontract-evaluation-semantic=`
  - `enforce`
  - `quick_enforce`
  - `observe`

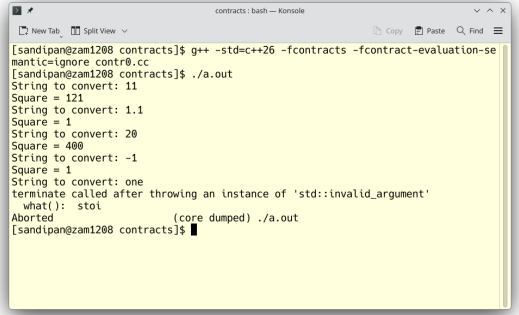


JÜLICH Forschungszentrum

Contract evaluation semantic

```
1  import std;
2  namespace sr = std::ranges;
3  constexpr auto are_digits(std::string_view R)
4  {
5      return sr::all_of(R, [](char c) {
6          return std::isdigit(c);
7      });
8  }
9  auto func(const std::string& s) -> int
10     pre(not s.empty())
11     pre(are_digits(s) or
12             (s.size() > 1UL
13              && s[0] == '-'
14              && are_digits(s.substr(1UL))))
15 { return std::stoi(s); }
16 auto main(int argc, char *argv[]) -> int
17 {
18     while (1) {
19         std::string inp;
20         std::cout << "String to convert: ";
21         std::cin >> inp;
22         auto i = func(inp);
23         std::cout << "Square = "
24             << i * i << "\n";
25     }
26 }
```

- `-std=c++26 -fcontracts`
- In addition... `-fcontract-evaluation-semantic=`
  - `enforce`
  - `quick_enforce`
  - `observe`
  - `ignore`



JÜLICH Forschungszentrum

Contracts

```
#include <contracts>
#include <print>
namespace con = std::contracts;
void handle_contract_violation(const con::contract_violation& v)
{
    auto&& loc = v.location();
    std::print(stderr, R"(Contract violation!
kind: {}
semantic: {}
comment: {}
location: {}:line={}
)",
               static_cast<int>(v.kind()),
               static_cast<int>(v.semantic()),
               v.comment(),
               loc.file_name(), loc.line());
    con::invoke_default_contract_violation_handler(v);
}
auto divide(int a, int b) pre (b != 0) { return a / b; }
auto main() -> int
{
    divide(10, 0);
    std::print("Won't be reached.\n");
}
```

Custom violation handler

- We can provide a custom handler to manage the exit when a contract violation happens
- Manage application specific urgent cleanup before the crash...

JÜLICH Forschungszentrum

Contracts: hardened standard library

- A significant part of memory bugs start with out-of-bounds access in containers, and operations on empty containers
- Changing all occurences of `v[i]` to `v.at(i)` would help finding more bugs, but it is tedius. Once the bugs are fixed, we would need a quick way back to an unchecked semantic for performance
- If standard library functions and function templates could be marked with contracts style pre-conditions, we could detect subtle bugs in many programs by simply changing compiler options, without changing a line of the user code!

Hardened implementation

- In a hardened implementation, many functions, like `operator[]`, `pop_back()`, `pop_front()` in many containers have hardened preconditions.
- When any such function is used, the predicates of its contract assertions are evaluated with a terminating semantic
- GCC's `libstdc++` and Clang's `libc++` both provide hardened implementations, requiring different compiler options
  - GCC: `-fhardened`
  - Clang: `-D_LIBCPP_HARDENING_MODE=`
    - `_LIBCPP_HARDENING_MODE_FAST`
    - `_LIBCPP_HARDENING_MODE_EXTENSIVE`
    - `_LIBCPP_HARDENING_MODE_DEBUG`
- Current implementation status: useful, but neither elegant nor comprehensive

Contracts: Summary

- Disentangle correctness checks from business logic

Contracts: Summary

- Disentangle correctness checks from business logic
- Non-macro assertions

Contracts: Summary

- Disentangle correctness checks from business logic
- Non-macro assertions
- Possible to turn off checks completely

Contracts: Summary

- Disentangle correctness checks from business logic
- Non-macro assertions
- Possible to turn off checks completely
- Tools or collaborators can more clearly see pre-conditions and guarantees

Contracts: Summary

- Disentangle correctness checks from business logic
- Non-macro assertions
- Possible to turn off checks completely
- Tools or collaborators can more clearly see pre-conditions and guarantees
- **Concerns:** Continued, well grounded, technical disagreement in the committee discussions:

Contracts: Summary

- Disentangle correctness checks from business logic
- Non-macro assertions
- Possible to turn off checks completely
- Tools or collaborators can more clearly see pre-conditions and guarantees
- **Concerns:** Continued, well grounded, technical disagreement in the committee discussions:
  - Too much left as implementation details: Compile time? Link time? Load time? Run time? What happens when different TUs with different evaluations semantics are linked?

Contracts: Summary

- Disentangle correctness checks from business logic
- Non-macro assertions
- Possible to turn off checks completely
- Tools or collaborators can more clearly see pre-conditions and guarantees
- **Concerns:** Continued, well grounded, technical disagreement in the committee discussions:
  - Too much left as implementation details: Compile time? Link time? Load time? Run time? What happens when different TUs with different evaluations semantics are linked?
  - No `assume` semantic allowing compilers to optimize based on assuming that the contracts are not violated

Contracts: Summary

- Disentangle correctness checks from business logic
- Non-macro assertions
- Possible to turn off checks completely
- Tools or collaborators can more clearly see pre-conditions and guarantees
- **Concerns:** Continued, well grounded, technical disagreement in the committee discussions:
  - Too much left as implementation details: Compile time? Link time? Load time? Run time? What happens when different TUs with different evaluations semantics are linked?
  - No `assume` semantic allowing compilers to optimize based on assuming that the contracts are not violated
  - What happens if the user specified violation handler throws?

Contracts: Summary

- Disentangle correctness checks from business logic
- Non-macro assertions
- Possible to turn off checks completely
- Tools or collaborators can more clearly see pre-conditions and guarantees
- **Concerns:** Continued, well grounded, technical disagreement in the committee discussions:
  - Too much left as implementation details: Compile time? Link time? Load time? Run time? What happens when different TUs with different evaluations semantics are linked?
  - No `assume` semantic allowing compilers to optimize based on assuming that the contracts are not violated
  - What happens if the user specified violation handler throws?
  - Should pre-condition predicate evaluations be attached to the caller or callee?

Contracts: Summary

- Disentangle correctness checks from business logic
- Non-macro assertions
- Possible to turn off checks completely
- Tools or collaborators can more clearly see pre-conditions and guarantees
- **Concerns:** Continued, well grounded, technical disagreement in the committee discussions:
  - Too much left as implementation details: Compile time? Link time? Load time? Run time? What happens when different TUs with different evaluations semantics are linked?
  - No `assume` semantic allowing compilers to optimize based on assuming that the contracts are not violated
  - What happens if the user specified violation handler throws?
  - Should pre-condition predicate evaluations be attached to the caller or callee?
  - Neither minimal nor mature

Contracts: Summary

- Disentangle correctness checks from business logic
- Non-macro assertions
- Possible to turn off checks completely
- Tools or collaborators can more clearly see pre-conditions and guarantees
- **Concerns:** Continued, well grounded, technical disagreement in the committee discussions:
  - Too much left as implementation details: Compile time? Link time? Load time? Run time? What happens when different TUs with different evaluations semantics are linked?
  - No `assume` semantic allowing compilers to optimize based on assuming that the contracts are not violated
  - What happens if the user specified violation handler throws?
  - Should pre-condition predicate evaluations be attached to the caller or callee?
  - Neither minimal nor mature
- Considered useful enough overall, so that it was voted in, despite the disagreement

Contracts: Summary

- Disentangle correctness checks from business logic
- Non-macro assertions
- Possible to turn off checks completely
- Tools or collaborators can more clearly see pre-conditions and guarantees
- **Concerns:** Continued, well grounded, technical disagreement in the committee discussions:
  - Too much left as implementation details: Compile time? Link time? Load time? Run time? What happens when different TUs with different evaluations semantics are linked?
  - No `assume` semantic allowing compilers to optimize based on assuming that the contracts are not violated
  - What happens if the user specified violation handler throws?
  - Should pre-condition predicate evaluations be attached to the caller or callee?
  - Neither minimal nor mature
- Considered useful enough overall, so that it was voted in, despite the disagreement
- Not for the type of input validation which can be thought of as normal program behaviour. A contract violation should immediately point to a bug in the program.

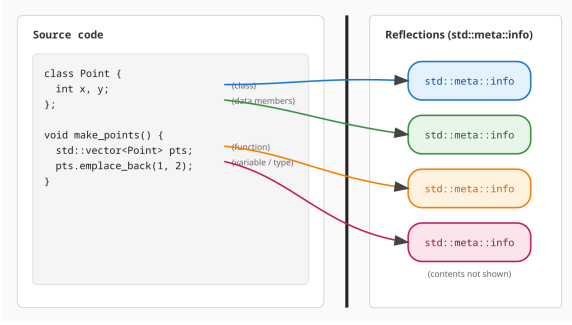
Reflection

Reflective programming

- Language internal mechanism to access information about the structure of program constructs
  - What data members does a class have?
  - What are all the possible values for an enumerated type?
- The compiler knows, but we didn't have a way to ask it
- Transform meta information about entities, and use the information: e.g., a function template asks about the data members of the element type of a container, and creates a new data type containing containers of those individual data members (AOS–SOA conversion)
- Create language bindings for other languages for arbitrary C++ classes
- Write serializer functions, writing out data members of an input type, for an unknown C++ type
- Convert arbitrary enum types to strings representing the enumerators

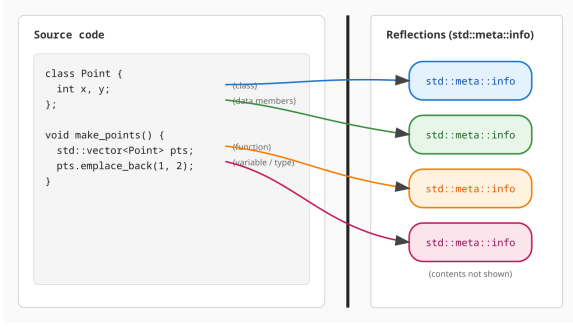
Reflection

- New fundamental type for reflection values. There is a unique null reflection. A non-null reflection can represent, e.g., a scalar value, an object, a variable, a function, a class, an enumerator, a template, a concept...



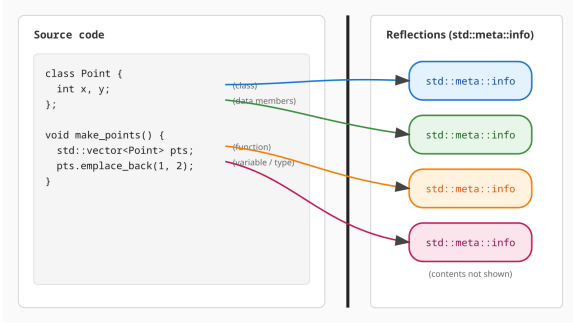
Reflection

- New fundamental type for reflection values. There is a unique null reflection. A non-null reflection can represent, e.g., a scalar value, an object, a variable, a function, a class, an enumerator, a template, a concept...
- The namespace `std::meta` is associated with reflections. Reflection values are represented by `std::meta::info`.



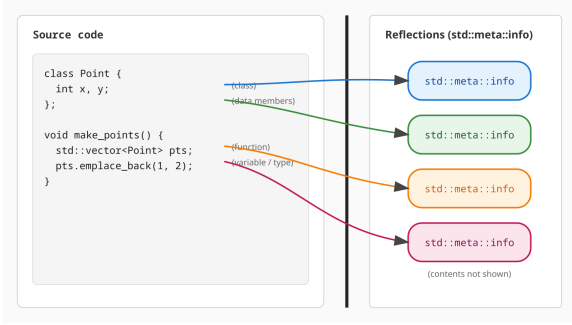
Reflection

- New fundamental type for reflection values. There is a unique null reflection. A non-null reflection can represent, e.g., a scalar value, an object, a variable, a function, a class, an enumerator, a template, a concept...
- The namespace `std::meta` is associated with reflections. Reflection values are represented by `std::meta::info`.
- Standard functions working with these reflection types are in the same namespace for ADL purposes



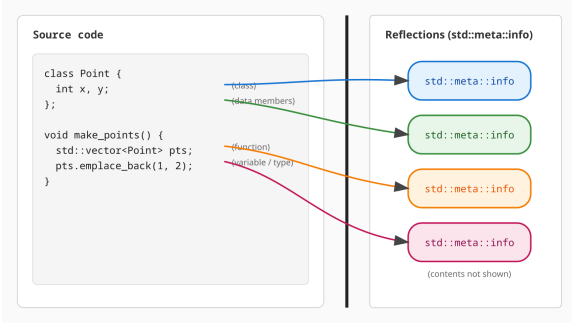
Reflection

- New fundamental type for reflection values. There is a unique null reflection. A non-null reflection can represent, e.g., a scalar value, an object, a variable, a function, a class, an enumerator, a template, a concept...
- The namespace `std::meta` is associated with reflections. Reflection values are represented by `std::meta::info`.
- Standard functions working with these reflection types are in the same namespace for ADL purposes
- `info` values are equality-comparable (but note that reflections of type aliases compare unequal with reflections of the aliased types)



Reflection

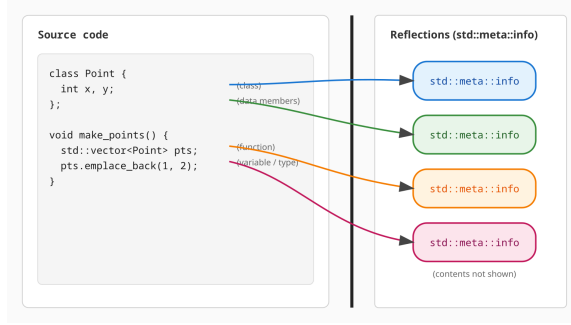
- New fundamental type for reflection values. There is a unique null reflection. A non-null reflection can represent, e.g., a scalar value, an object, a variable, a function, a class, an enumerator, a template, a concept...
- The namespace `std::meta` is associated with reflections. Reflection values are represented by `std::meta::info`.
- Standard functions working with these reflection types are in the same namespace for ADL purposes
- `info` values are equality-comparable (but note that reflections of type aliases compare unequal with reflections of the aliased types)
- The new unary operator `^^` gives us the reflection corresponding to a language construct. E.g., `^^Point` gives us an `std::meta::info` value representing the `Point` class



JÜLICH Forschungszentrum

Reflection

- New fundamental type for reflection values. There is a unique null reflection. A non-null reflection can represent, e.g., a scalar value, an object, a variable, a function, a class, an enumerator, a template, a concept...
- The namespace `std::meta` is associated with reflections. Reflection values are represented by `std::meta::info`.
- Standard functions working with these reflection types are in the same namespace for ADL purposes
- `info` values are equality-comparable (but note that reflections of type aliases compare unequal with reflections of the aliased types)
- The new unary operator `^^` gives us the reflection corresponding to a language construct. E.g., `^^Point` gives us an `std::meta::info` value representing the `Point` class



- The splicing operator `[: r :]` can be thought of as the opposite of `^^`, as it gives us the entity whose reflection is represented by `r`

JÜLICH Forschungszentrum

Inserting a reflection back into source code

```
1  // reflection/refl0.cc
2  #include <meta>
3  #include <concepts>
4  auto main() -> int
5  {
6      constexpr auto r = ^^const int;
7      using T = const [: r :] &;
8      T x = 4;
9      static_assert(std::same_as<const int&, T>);
10 }
```

- `-std=c++26 -freflection`
- `r` is a reflection of operand of `^^`, i.e., of `const int`
- `r` itself declared as a `constexpr` variable. It is not a typename, and can not be used to declare anything. But `[: r :]` is functionally equivalent to `const int`, and can be spliced into another declaration
- Functions receiving `std::meta::info` objects are immediate (`consteval`) functions. Reflective programming in C++ is a compile time technique, meant to improve expressiveness, but at no runtime cost.

Reflection

```cpp
// reflection/refl1.cc
#include <string_view>
#include <print>









enum class Colour { red, green, blue };
auto main() -> int
{
    std::println("{}", enum_to_string(Colour::red));
}
```

- Task: A function template to convert an enum to its name as a string. Can we do it more generally?

JÜLICH Forschungszentrum

Reflection

```
1  // reflection/refl1.cc
2  #include <string_view>
3  #include <print>
4  #include <meta>
5  template <class E>
6  requires std::is_enum_v<E>
7  constexpr auto enum_to_string(E value) -> std::string_view
8  {
9      namespace sm = std::meta;
10     template for (constexpr auto e:
11       std::define_static_array(sm::enumerators_of(^^ E))) {
12         if (value == [: e :]) {
13             return sm::display_string_of(e);
14         }
15     }
16     return "<unknown>";
17 }
18 enum class Colour { red, green, blue };
19 auto main() -> int
20 {
21     std::println("{}", enum_to_string(Colour::red));
22 }
```

- Task: A function template to convert an enum to its name as a string. Can we do it more generally?
- We *reflect* the typename of the enum type, which we receive as the template argument

JÜLICH Forschungszentrum

Reflection

```
#include <string_view>
#include <print>
#include <meta>
template <class E>
requires std::is_enum_v<E>
constexpr auto enum_to_string(E value) -> std::string_view
{
    namespace sm = std::meta;
    template for (constexpr auto e:
      std::define_static_array(sm::enumerators_of(^^ E))) {
        if (value == [: e :]) {
            return sm::display_string_of(e);
        }
    }
    return "<unknown>";
}
enum class Colour { red, green, blue };
auto main() -> int
{
    std::println("{}", enum_to_string(Colour::red));
}
```

- Task: A function template to convert an enum to its name as a string. Can we do it more generally?
- We *reflect* the typename of the enum type, which we receive as the template argument
- Ask for all enumerators of that enum and loop over their reflections

Reflection

```
#include <string_view>
#include <print>
#include <meta>
template <class E>
requires std::is_enum_v<E>
constexpr auto enum_to_string(E value) -> std::string_view
{
    namespace sm = std::meta;
    template for (constexpr auto e:
      std::define_static_array(sm::enumerators_of(^^ E))) {
        if (value == [: e :]) {
            return sm::display_string_of(e);
        }
    }
    return "<unknown>";
}
enum class Colour { red, green, blue };
auto main() -> int
{
    std::println("{}", enum_to_string(Colour::red));
}
```

- Task: A function template to convert an enum to its name as a string. Can we do it more generally?
- We *reflect* the typename of the enum type, which we receive as the template argument
- Ask for all enumerators of that enum and loop over their reflections
- Pick the matching one by splicing back the reflection from the current iteration of the loop into source code

Reflection

```
1  #include <string_view>
2  #include <print>
3  #include <meta>
4  template <class E>
5  requires std::is_enum_v<E>
6  constexpr auto enum_to_string(E value) -> std::string_view
7  {
8      namespace sm = std::meta;
9      template for (constexpr auto e:
10       std::define_static_array(sm::enumerators_of(^^ E))) {
11         if (value == [: e :]) {
12             return sm::display_string_of(e);
13         }
14     }
15     return "<unknown>";
16 }
17 enum class Colour { red, green, blue };
18 auto main() -> int
19 {
20     std::println("{}", enum_to_string(Colour::red));
21 }
```

- Task: A function template to convert an enum to its name as a string. Can we do it more generally?
- We *reflect* the typename of the enum type, which we receive as the template argument
- Ask for all enumerators of that enum and loop over their reflections
- Pick the matching one by splicing back the reflection from the current iteration of the loop into source code
- Return the string identifier from the reflection

Reflection

```
1  #include <string_view>
2  #include <print>
3  #include <meta>
4  template <class E>
5  requires std::is_enum_v<E>
6  constexpr auto enum_to_string(E value) -> std::string_view
7  {
8      namespace sm = std::meta;
9      template for (constexpr auto e:
10       std::define_static_array(
11           sm::enumerators_of(^^ E))) {
12         if (value == [: e :]) {
13             return sm::display_string_of(e);
14         }
15     }
16     return "<unknown>";
17 }
18 enum class Colour { red, green, blue };
19 auto main() -> int
20 {
21     std::println("{}", enum_to_string(Colour::red));
22 }
```

- Reflection of every kind of language construct creates a `std::meta::info`.

JÜLICH Forschungszentrum

Reflection

```cpp
#include <string_view>
#include <print>
#include <meta>
template <class E>
requires std::is_enum_v<E>
constexpr auto enum_to_string(E value) -> std::string_view
{
    namespace sm = std::meta;
    template for (constexpr auto e:
      std::define_static_array(
          sm::nonstatic_datamembers_of(^^ E))) {
        if (value == [: e :]) {
            return sm::display_string_of(e);
        }
    }
    return "<unknown>";
}
enum class Colour { red, green, blue };
auto main() -> int
{
    std::println("{}", enum_to_string(Colour::red));
}
```

- Reflection of every kind of language construct creates a `std::meta::info`.
- What if we ask a non-sensical question about the construct, like, the non-static datamembers of a **`concept`** or **`enum class`**?

JÜLICH Forschungszentrum

Reflection

```cpp
#include <string_view>
#include <print>
#include <meta>
template <class E>
requires std::is_enum_v<E>
constexpr auto enum_to_string(E value) -> std::string_view
{
    namespace sm = std::meta;
    template for (constexpr auto e:
      std::define_static_array(
          sm::nonstatic_datamembers_of(^^ E))) {
          // throws sm::exception
        if (value == [: e :]) {
            return sm::display_string_of(e);
        }
    }
    return "<unknown>";
}
enum class Colour { red, green, blue };
auto main() -> int
{
    std::println("{}", enum_to_string(Colour::red));
}
```

- Reflection of every kind of language construct creates a `std::meta::info`.
- What if we ask a non-sensical question about the construct, like, the non-static datamembers of a **`concept`** or **`enum class`**?
- → Throw an exception: `std::meta::exception`, which can only be created and accessed in constant evaluation

JÜLICH Forschungszentrum

Reflection

```cpp
// reflection/refl2.cc
template <class T> requires std::is_class_v<T>
void print_members(const T& t)
{
    using namespace std::meta;
    template for (constexpr auto m: std::define_static_array(
      nonstatic_data_members_of(^^T, access_context::unprivileged()))) {
        std::println("{} = {}", identifier_of(m), t.[:m:]);
    }
}
struct Point {
    private:
    int x = 3;
    public:
    double y = 3.44;
};
struct Vec3 { double x{}, y{}, z{}; };
auto main() -> int
{
    Point p{};
    Vec3 v{0.1, 0.2, 0.3};
    print_members(p);
    print_members(v);
}
```

Listing members

- Task: write the data members of an arbitrary type
- Reflect, ask for non static data members of reflection → reflection of data members.
- Splice back reflection of data-members into source code.
- Observe how expansion statements (`template for`) is used to create a strictly compile time loop

JÜLICH Forschungszentrum

Reflection: Summary

- Massive step for C++ meta-programming

Reflection: Summary

- Massive step for C++ meta-programming
- New possibilities regarding what can be expressed using standard C++ code

Reflection: Summary

- Massive step for C++ meta-programming
- New possibilities regarding what can be expressed using standard C++ code
- Reflection in C++ happens at compile time. No additional runtime cost when using code involving reflection